

Sri Lanka Institute of Information Technology



SE4040: Enterprise Application Development
Y4.S1.WE.SE.01

Assignment 1: Project Report

Smart Solar Microgrid Trading System

Client-Server Application (Web, Mobile and Web Service)

IT Number	Name	Role
IT23325050	Weerasinghe G.A.A.I (Avishka)	Group Leader
IT23334892	Amarasekara S W N (Nisal)	Member
IT23386136	Vinoshan A G S (Sampath)	Member
IT23135734	Dias M T D (Tharindu)	Member

BSc (Hons) in Information Technology Specialized in Software Engineering
Sri Lanka Institute of Information Technology (SLIIT)
September 2026

Table of Contents

Table of Contents	2
1. Introduction.....	3
1.1 Objectives	3
2. System Architecture	4
2.1 Technology Stack	4
3. Use Case Diagram	5
4. Data Flow Diagram (Level 1).....	6
5. Database Design	7
5.1 Users	7
5.2 SolarStationInfo	7
5.3 EnergyBookingSlots	8
5.4 EnergyReservation.....	9
6. Web Application Screenshots.....	10
7. Mobile Application Screenshots.....	14
8. Individual Contribution.....	26
8.1 Weerasinghe G.A.A.I (Avishka) (IT23325050): Identity & Access.....	26
8.2 Amarasekara S W N (Nisal) (IT23334892): Microgrid Nodes & Maps	26
8.3 Vinoshan A G S (Sampath) (IT23386136): Reservations & Booking Workflow	26
8.4 Dias M T D (Tharindu) (IT23135734): Grid Operator Operations & Service Integration	26
9. Challenges.....	27
10. References	28
11. Git Repository and Demo Video	29
Appendix A: Web API Source Code (C#)	30
Appendix B: Web Application Source Code (React)	61
Appendix C: Mobile Application Source Code (Kotlin)	100

1. Introduction

The Smart Solar Microgrid Trading System is an end-to-end client-server application that lets a network of solar microgrid hubs trade stored energy with property-owning “prosumers” (solar panel owners). The system has three tiers: a React + Tailwind CSS web application for Backoffice administrators and Grid Operators, a pure native Android application (Kotlin, SQLite) for Prosumers and Grid Operators, and a central C# ASP.NET Core Web API (hosted on IIS, backed by MongoDB) that holds every piece of business logic. Both clients are thin UI layers: neither performs business validation nor talks to the database directly, satisfying the FAT-service architecture the assignment specifies.

The Backoffice team registers and maintains microgrid nodes and their operating schedules, reviews and approves new accounts, and manages staff users. Grid Operators, who can use both the web and mobile apps, monitor power-trading bookings and keep battery-slot availability current, and finalise energy transfers in person by scanning a prosumer's QR code. Prosumers use the mobile app exclusively to register with their National Identity Card (NIC) as the primary key, reserve, modify or cancel energy drop-off/charging slots, track their booking history and dashboard, and locate nearby stations on a map.

1.1 Objectives

- Implement a FAT-service Web API where all business rules (activation workflow, the 7-day booking window, the 12-hour update/cancel notice, QR issue/verify/complete) are enforced server-side.
- Build a responsive Backoffice/Grid Operator web console using React and Tailwind CSS.
- Build a pure native Android application (no cross-platform framework) with SQLite used only for local session/reference caching.
- Persist all business data in MongoDB across four collections: Users, SolarStationInfo, EnergyBookingSlots and EnergyReservation.
- Host the API on IIS using the ASP.NET Core Hosting Bundle.

2. System Architecture

Both client applications communicate exclusively over RESTful JSON calls to the Web API, which is the single source of truth for business logic and the only component with a MongoDB connection. The Android app keeps a small SQLite database purely for local persistence: the signed-in session (JWT + profile) and a cache of the last-fetched nearby-station list, never as a substitute for server data.

Smart Solar Microgrid Trading System — High-Level Architecture

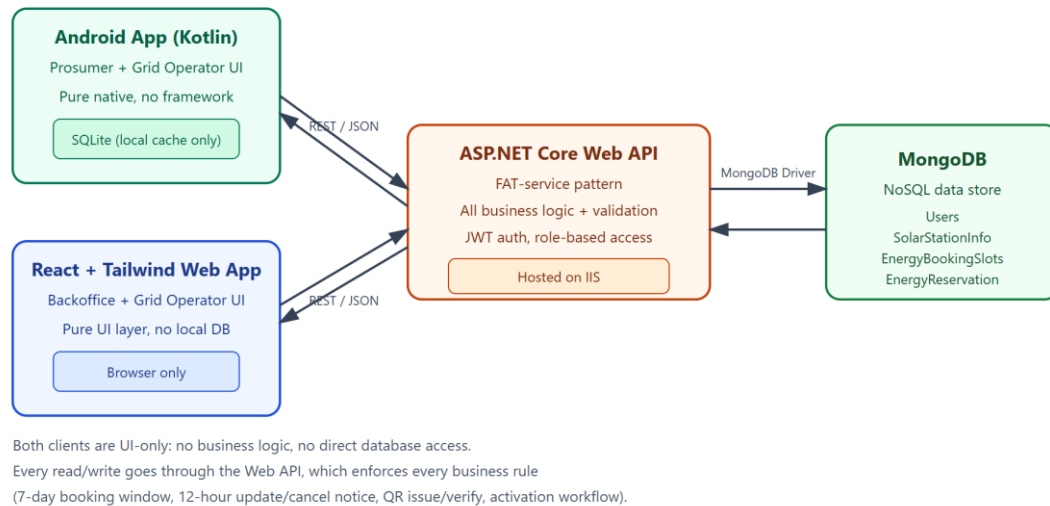


Figure 2.1: High-level architecture

2.1 Technology Stack

Layer	Technology
Web application	React (Vite), Tailwind CSS v4, React Router, Axios
Mobile application	Kotlin, native Android Views + ViewBinding, Retrofit/OkHttp, SQLite, Google Maps SDK, CameraX + ML Kit, ZXing
Web service	C# ASP.NET Core Web API, JWT bearer authentication, BCrypt.Net-Next, hosted on IIS
Database	MongoDB Atlas (cloud-hosted NoSQL), accessed via the official MongoDB.Driver

3. Use Case Diagram

Three actors interact with the system: the Prosumer (mobile only), the Backoffice officer (web only) and the Grid Operator (web and mobile). All three share the role-based Login use case; the remaining use cases are grouped by actor below.

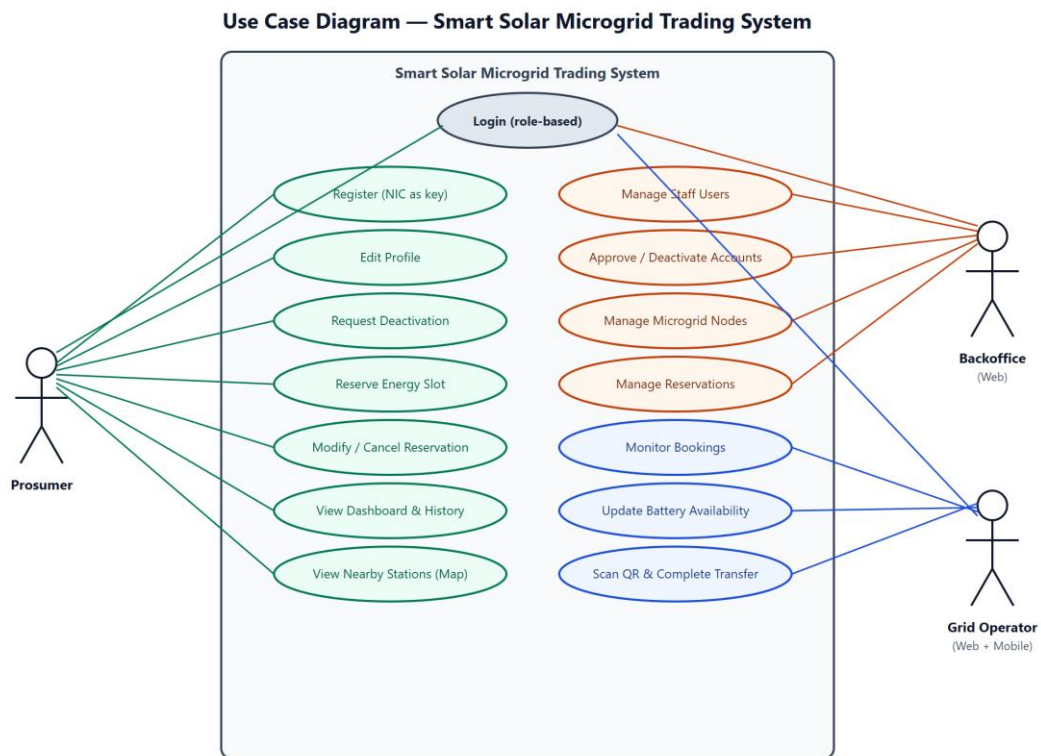


Figure 3.1: Use case diagram

4. Data Flow Diagram (Level 1)

The Level-1 DFD decomposes the system into four processes (Manage Users & Accounts, Manage Microgrid Nodes, Manage Energy Reservations and Process Energy Transfer), each reading and writing one of the four MongoDB collections that back it.

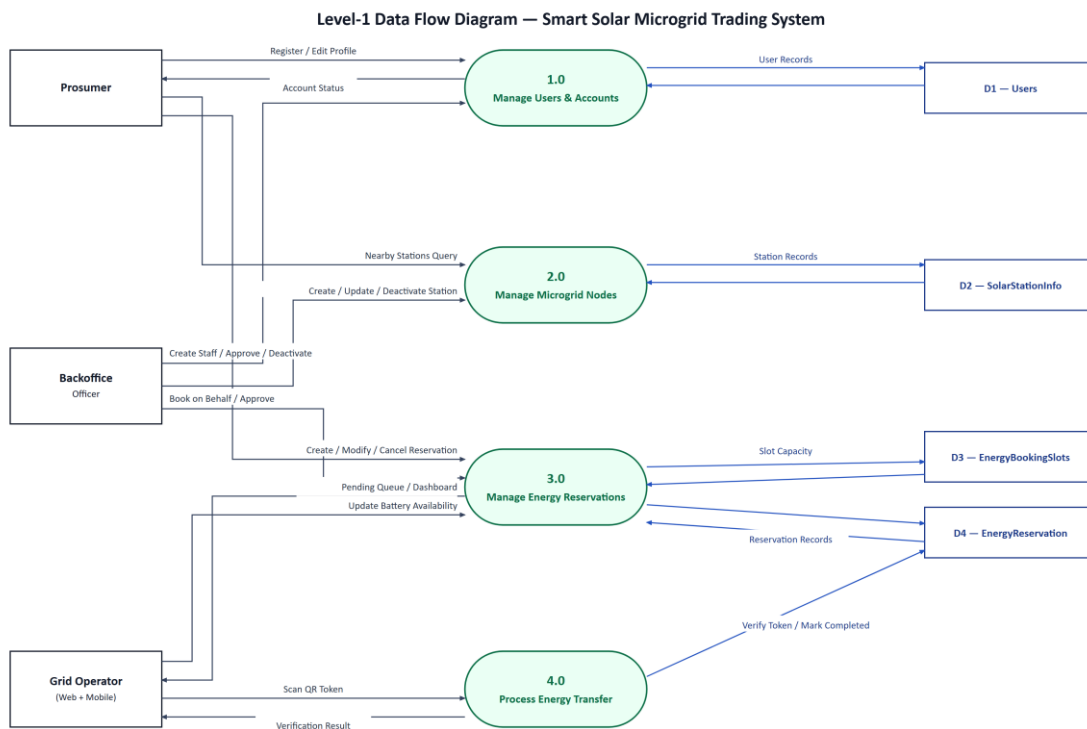


Figure 4.1: Level-1 Data Flow Diagram

5. Database Design

MongoDB (a NoSQL, document-oriented database) stores every business record behind the Web API, in exactly the four collections required by the assignment brief. The Users collection deliberately holds Backoffice, Grid Operator and Prosumer accounts together (discriminated by a UserType field) so that prosumer self-registration and Backoffice-created staff accounts can share one activation/deactivation workflow.

5.1 Users

Primary key: MongoDB ObjectId (_id). Prosumer accounts additionally use NIC as a unique natural key.

Field	Type	Description
UserType	enum	Backoffice GridOperator Prosumer
FullName, Email, Phone	string	Shared identity fields for every role
Username, PasswordHash	string	Staff login credentials (BCrypt-hashed password)
Nic, Address	string	Prosumer-only fields; Nic is the prosumer's primary key
Status	enum	PendingActivation Active Deactivated
CreatedBy, CreatedAt, UpdatedAt	string / date	Audit fields

Sample document:

```
{
  "_id": "6aaf7260b13c824a407b04c4",
  "UserType": "Backoffice (0)",
  "FullName": "System Administrator",
  "Email": "admin@solarmicrogrid.local",
  "Phone": "0000000000",
  "Username": "admin",
  "PasswordHash": "$2a$11$... (BCrypt hash)",
  "Nic": null,
  "Address": null,
  "Status": "Active (1)",
  "CreatedBy": "system-seed",
  "CreatedAt": "2026-09-20T05:42:55.801Z",
  "UpdatedAt": "2026-09-20T05:42:55.801Z"
}
```

5.2 SolarStationInfo

Represents a microgrid node / solar hub. Deactivation is blocked server-side while it has active (Pending or Approved) reservations.

Field	Type	Description
StationName	string	Display name
GpsLocation	{Lat, Lng}	Coordinates used for the mobile map and the nearby-stations query
CapacityKWh	double	Total generation/storage capacity
TotalBatterySlots, AvailableBatterySlots	int	Battery slot inventory, updated by Grid Operators

Schedule	array	List of {Day, OpenTime, CloseTime} operating-hours entries
Status	enum	Active Deactivated

Sample document:

```
{
  "_id": "6aaf72a0b13c824a407b04c8",
  "StationName": "Colombo Fort Hub",
  "GpsLocation": {
    "Lat": 6.9344,
    "Lng": 79.8428
  },
  "CapacityKWh": 75,
  "TotalBatterySlots": 12,
  "AvailableBatterySlots": 12,
  "Schedule": [
    {
      "Day": "Monday",
      "OpenTime": "07:00",
      "CloseTime": "19:00"
    }
  ],
  "Status": "Active (0)",
  "CreatedBy": "System Administrator",
  "CreatedAt": "2026-09-20T05:44:00.728Z",
  "UpdatedAt": "2026-09-20T05:44:00.728Z"
}
```

5.3 EnergyBookingSlots

Bookable time windows generated against a station; EnergyReservation documents reference one of these and decrement its CapacityAvailable. SlotService.GenerateFromScheduleAsync auto-generates the next 14 days of slots from a station's weekly Schedule whenever the station is created, and Backoffice can re-run it on demand (a “Generate slots” action on the Stations screen) to top up existing stations or extend the rolling window as time passes; it skips any date that already has a slot, so it is safe to call repeatedly.

Field	Type	Description
StationId	ObjectId ref	Owning station
Date, StartTime, EndTime	date / string	The slot's scheduled window
SlotType	enum	Charging DropOff
CapacityTotal, CapacityAvailable	int	Concurrent-booking capacity, decremented on each reservation
Status	enum	Open Full Closed

Sample document:

```
{
  "_id": "6aaf72a0b13c824a407b04cb",
  "StationId": "6aaf72a0b13c824a407b04c8",
  "Date": "2026-09-21T05:44:00.774Z",
  "StartTime": "09:00",
  "EndTime": "10:00",
  "SlotType": "Charging (0)",
  "CapacityTotal": 4,
  "CapacityAvailable": 3,
  "Status": "Open (0)",
  "CreatedAt": "2026-09-20T05:44:00.799Z",
  "UpdatedAt": "2026-09-20T05:44:00.882Z"
}
```

5.4 EnergyReservation

A prosumer's power-trading booking. Business rules enforced by ReservationService: ScheduledDateTime must fall within 7 days of creation; updates and cancellations require at least 12 hours' notice.

Field	Type	Description
ReservationCode	string	Short human-readable code, e.g. RSV-E69A06
ProsumerNic, StationId, SlotId	string / ObjectId ref	Links to the owning prosumer, station and slot
ScheduledDateTime, EnergyAmountKWh	date / double	When and how much energy is being transferred
Status	enum	Pending Approved Completed Cancelled
QrToken	string	HMAC-signed transaction token issued on approval, verified by the Grid Operator's scan
ApprovedBy/At, CompletedBy/At, CancelledAt, CancelReason	string / date	Workflow audit trail

Sample document:

```
{
  "_id": "6aaf72a0b13c824a407b04d0",
  "ReservationCode": "RSV-E69A06",
  "ProsumerNic": "199512345678",
  "StationId": "6aaf72a0b13c824a407b04c8",
  "SlotId": "6aaf72a0b13c824a407b04cb",
  "ScheduledDateTime": "2026-09-21T05:44:00.862Z",
  "EnergyAmountKWh": 6.5,
  "Status": "Pending (0)",
  "QrToken": null,
  "ApprovedBy": null,
  "ApprovedAt": null,
  "CompletedBy": null,
  "CompletedAt": null,
  "CancelledAt": null,
  "CancelReason": null,
  "CreatedAt": "2026-09-20T05:44:00.884Z",
  "UpdatedAt": "2026-09-20T05:44:00.884Z"
}
```

6. Web Application Screenshots

All screens below were captured against the live API and a freshly seeded MongoDB database (a scripted API smoke flow), confirmed rendering with zero browser console errors.

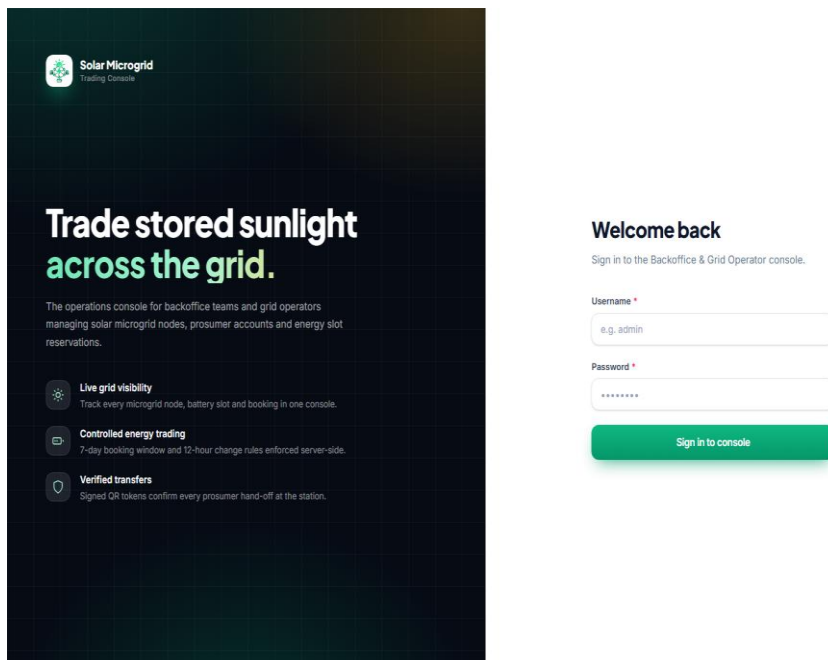


Figure 6.1: Web login page (shared Backoffice / Grid Operator sign-in)

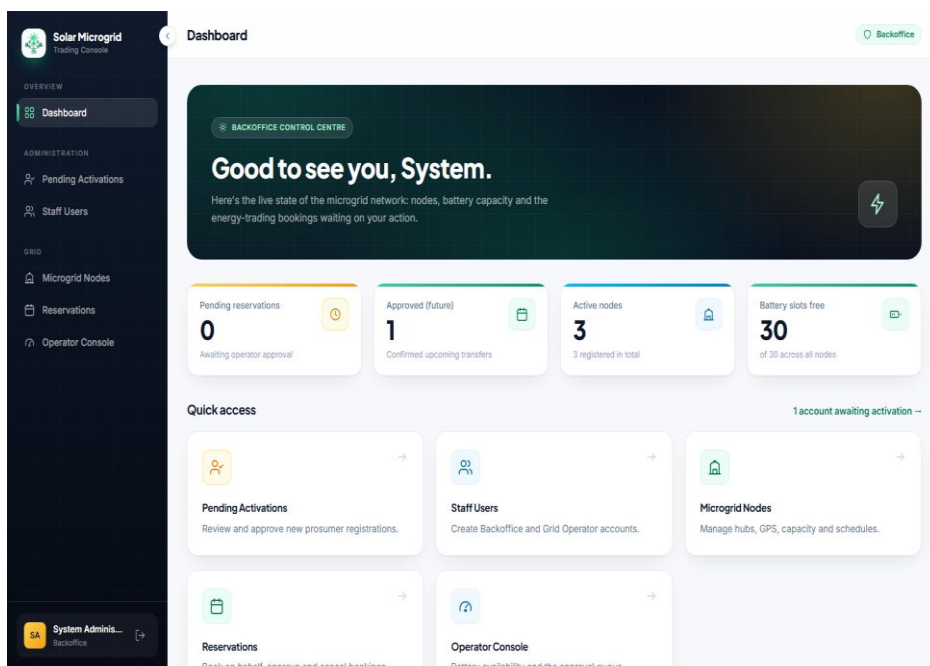


Figure 6.2: Backoffice: Dashboard (main landing screen)

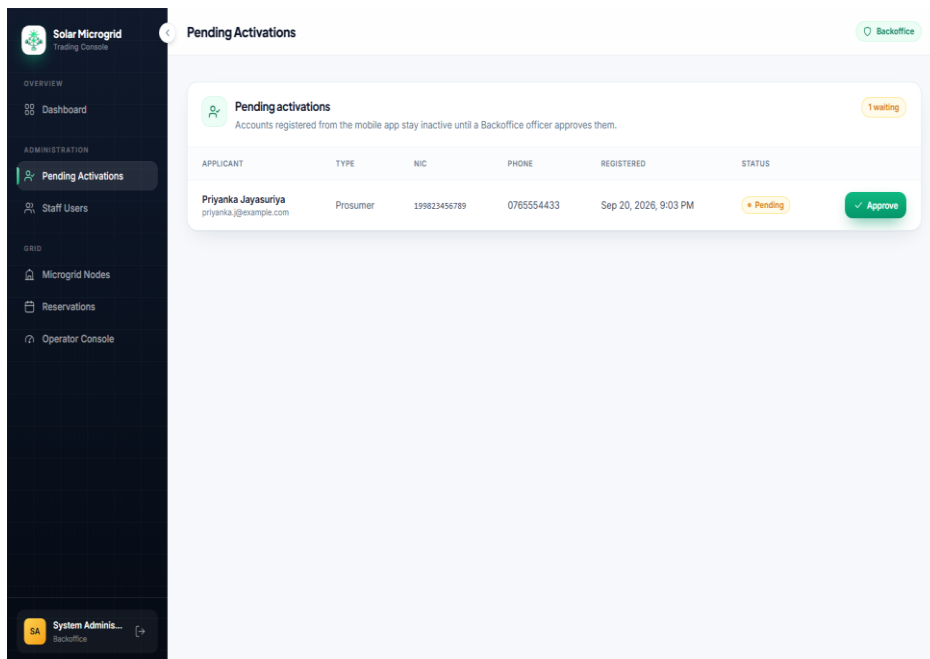


Figure 6.3: Backoffice: Pending Activations queue

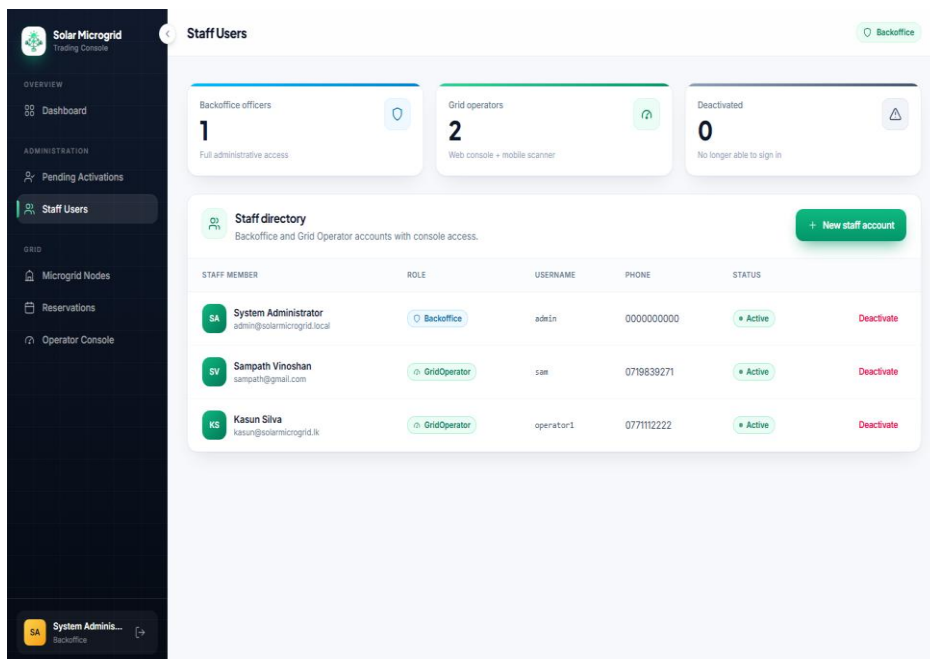


Figure 6.4: Backoffice: Staff Users management

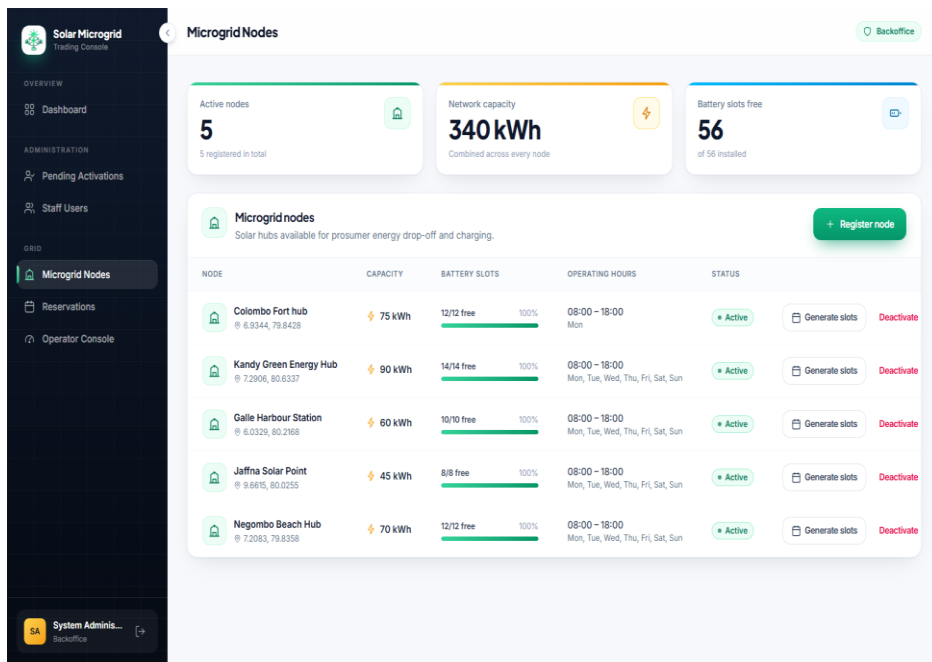


Figure 6.5: Backoffice: Microgrid Nodes management

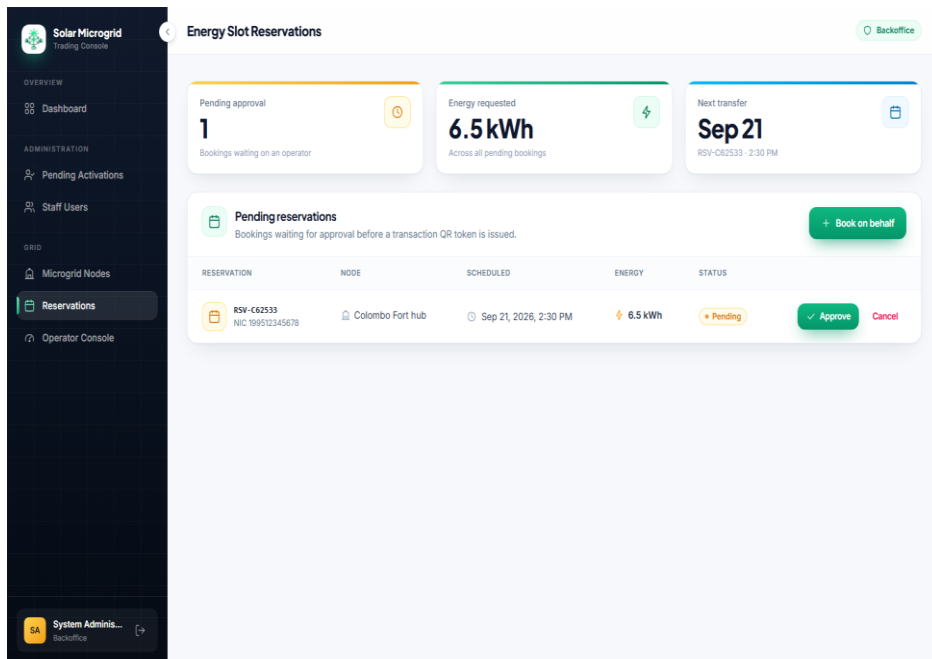


Figure 6.6: Backoffice: Energy Slot Reservations management

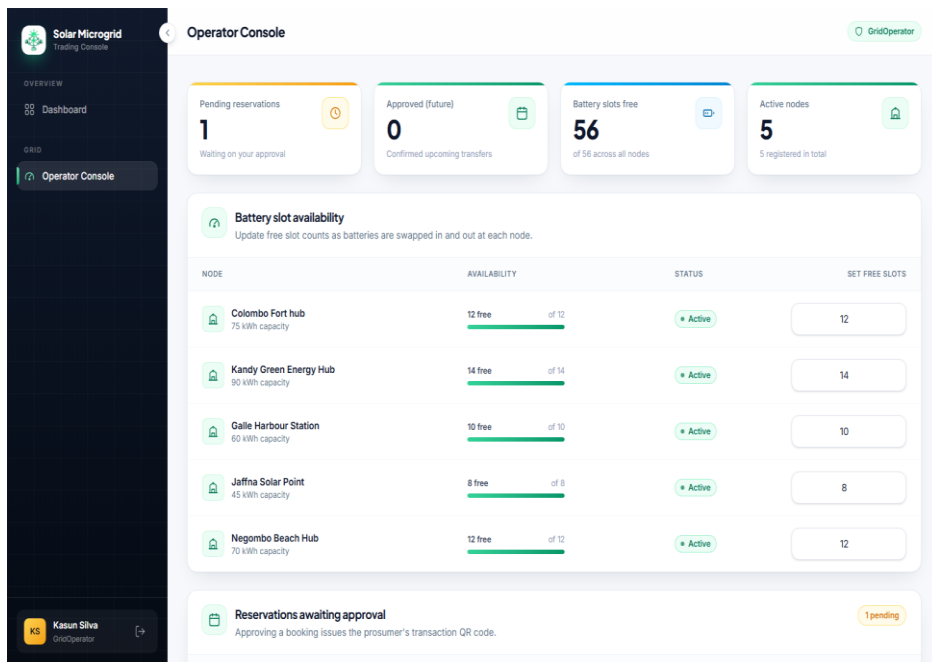


Figure 6.7: Grid Operator dashboard (web)

7. Mobile Application Screenshots

Captured on a physical Samsung Galaxy A04s (Android 14) running the debug build over USB, against the same seeded database used for the web screenshots above.

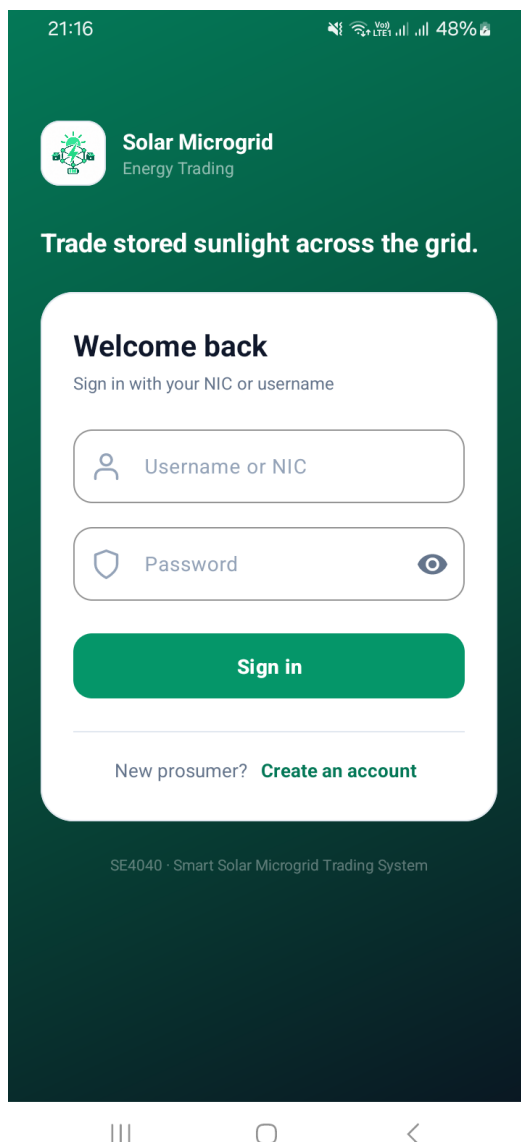


Figure 7.1: Mobile: shared Prosumer / Grid Operator login

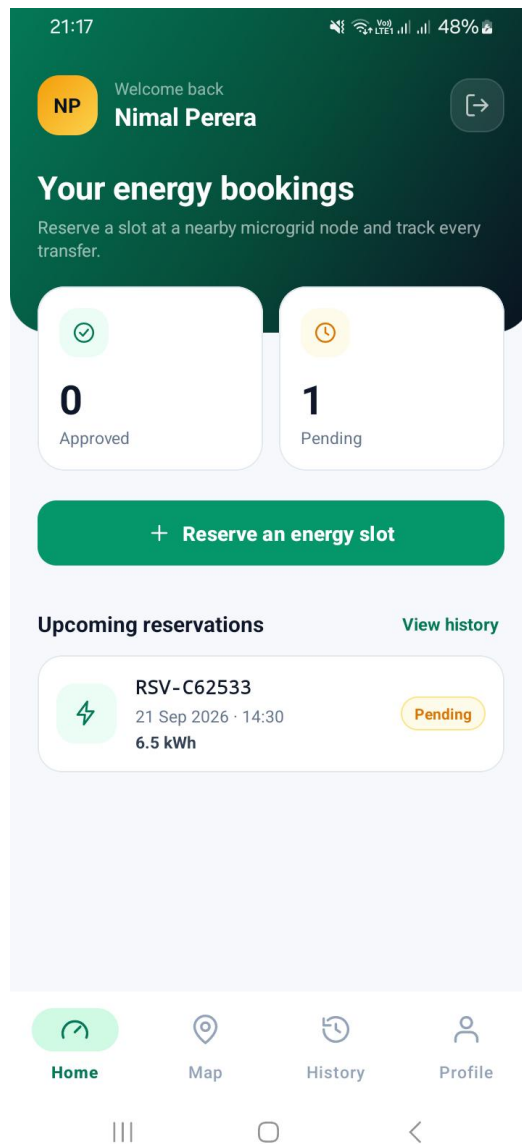


Figure 7.2: Prosumer dashboard (active/pending counts, upcoming bookings)

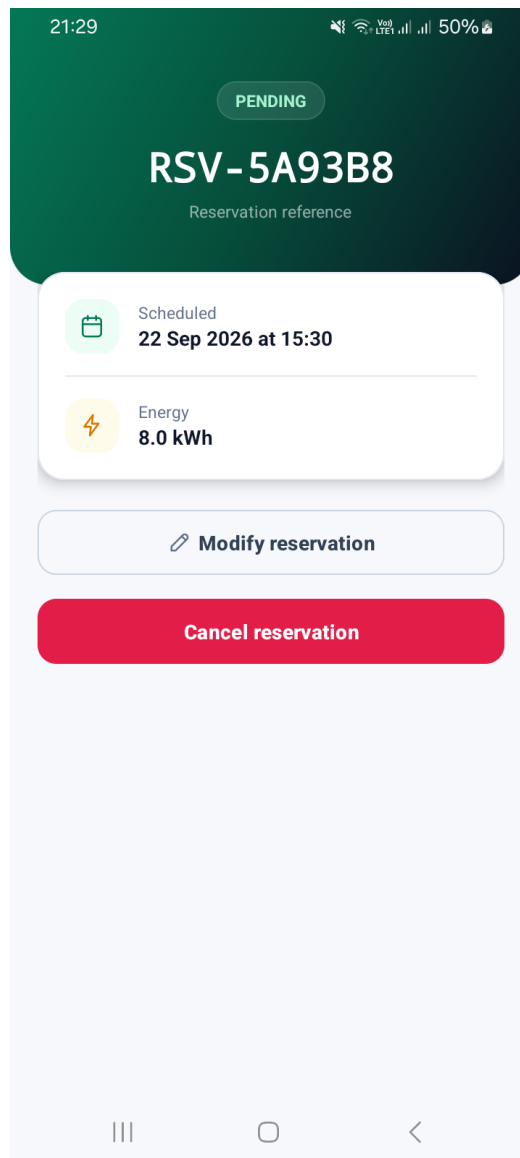


Figure 7.3: Reservation detail (Pending, with Modify/Cancel)

21:18

Vodafone LTE1

49%

New reservation

Pick a microgrid node and an available energy slot.

WHERE

Station

Colombo Fort hub

Available slot

2026-09-21 08:00-18:00 (11 left)

WHEN & HOW MUCH

Pick date & time

Energy amount (kWh)

Bookings must be within 7 days. Changes and cancellations need at least 12 hours' notice.

✓ Submit reservation

|||

<

Figure 7.4: New Reservation form (station/slot pickers, date/time)

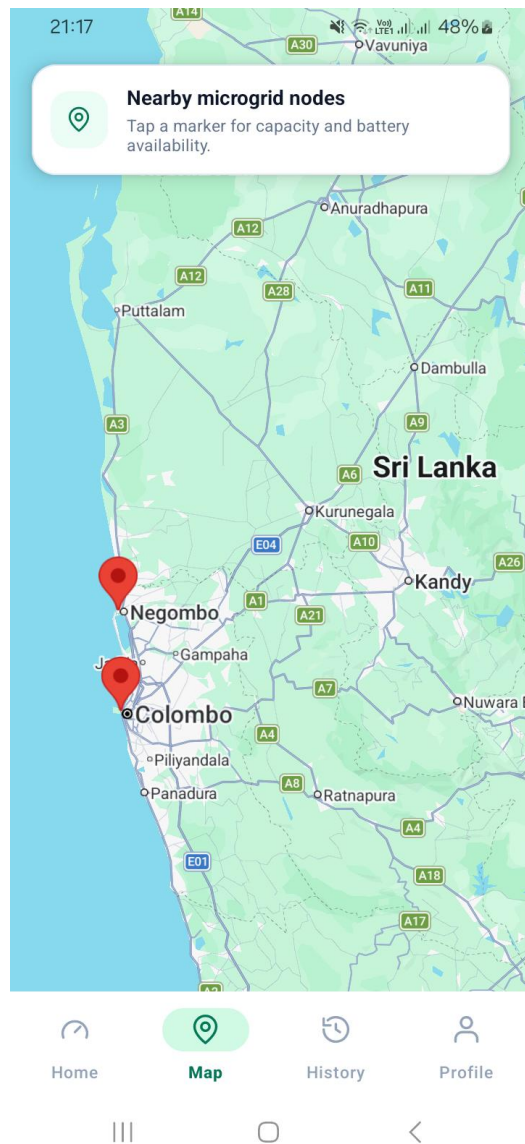


Figure 7.5: Nearby Microgrid Stations map, live with a registered Google Maps API key

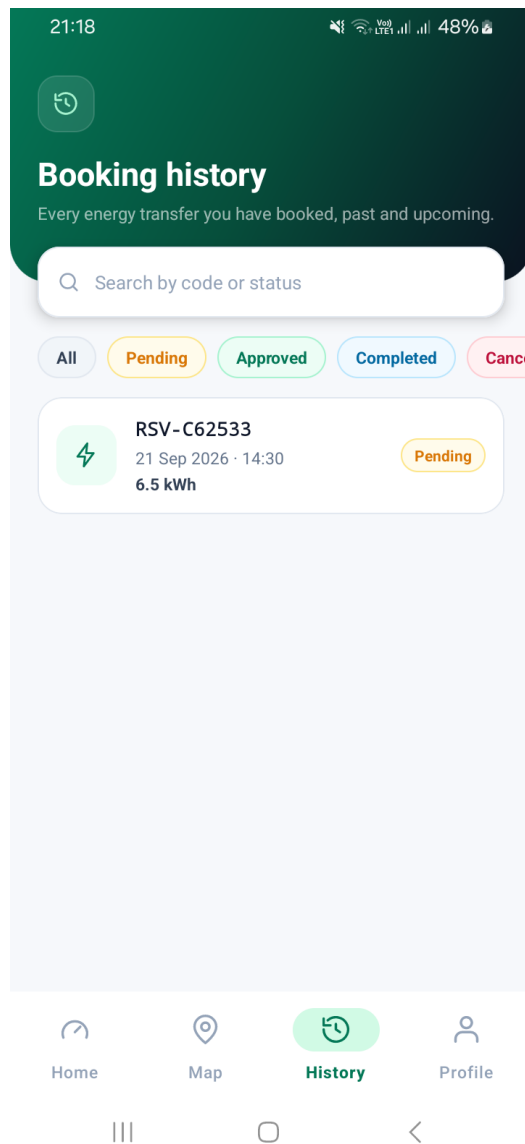


Figure 7.6: Booking History with search/filter, showing all four reservation states

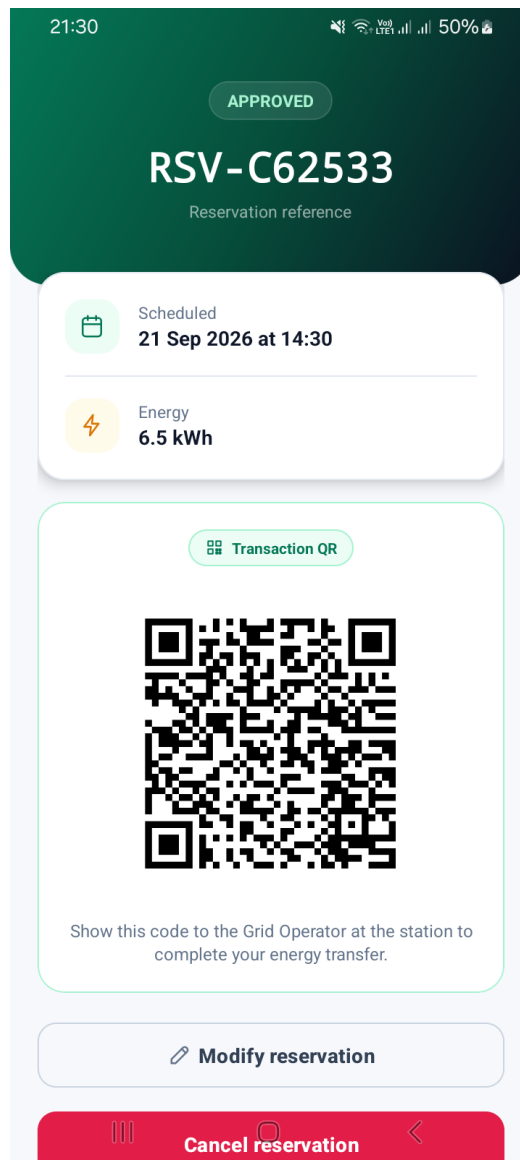


Figure 7.7: Approved reservation with generated transaction QR code

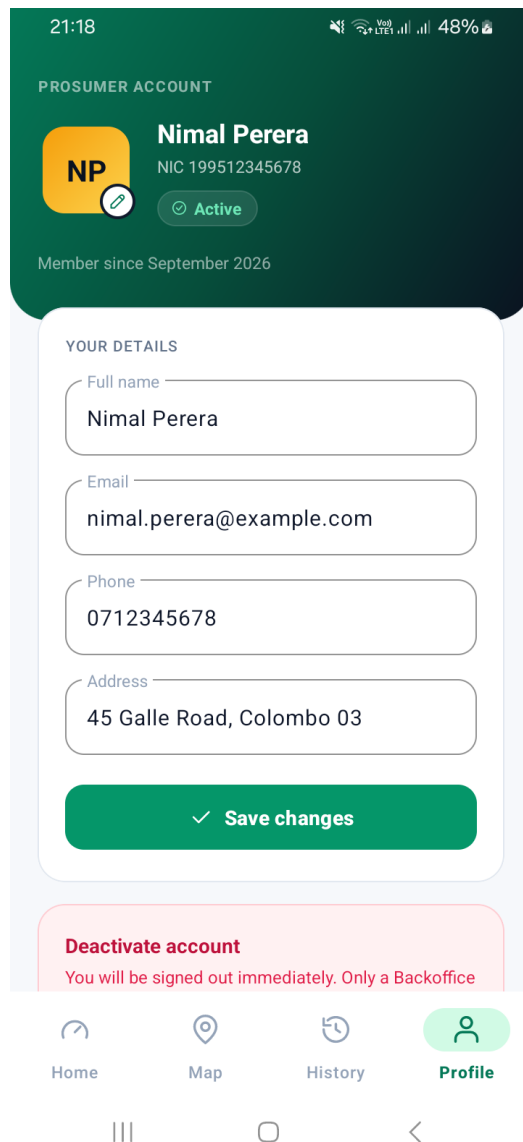


Figure 7.8: Prosumer profile (edit details / request deactivation)

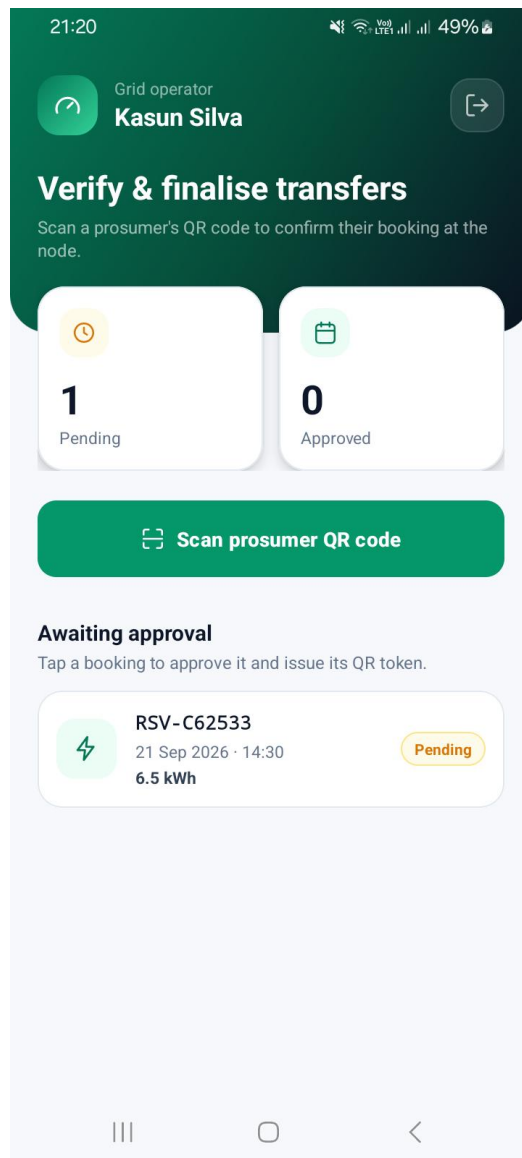


Figure 7.9: Grid Operator dashboard (mobile)

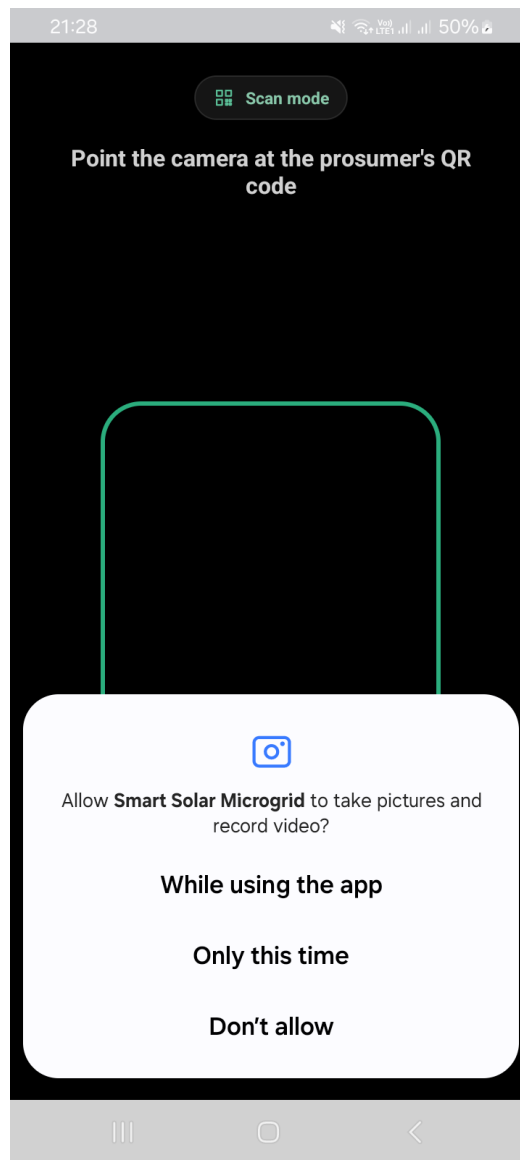


Figure 7.10: Runtime camera permission request (QR scan mode)

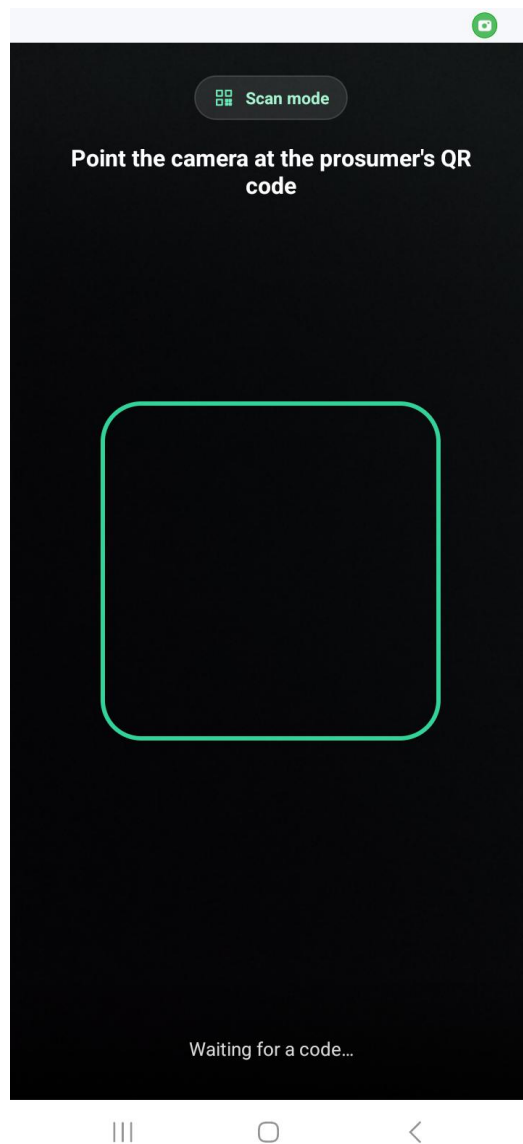


Figure 7.11: Live QR scanner (CameraX + ML Kit)

21:30

Vodafone

LTE1

50%

Create your account

Register as a solar prosumer to reserve energy slots at microgrid nodes.

IDENTITY

NIC (National Identity Card)

Full name

CONTACT

Email

Phone

Address

SECURITY

Password

Your account status pending until a Backoffice

Figure 7.12: Prosumer self-registration (NIC as primary key)

8. Individual Contribution

Each member owns a vertical slice spanning web, mobile, API and one MongoDB collection, so their contribution can be independently verified and defended at viva.

8.1 Weerasinghe G.A.A.I (Avishka) (IT23325050): Identity & Access

- Web: Login / role-based redirect, Backoffice staff user creation, Pending Activations review screen
- Mobile: NIC-based registration, login with role-based home routing, profile edit, request deactivation
- API: AuthController/AuthService (JWT issuing), UsersController/UserService, ProsumersController/ProsumerService
- Database: Users collection design and the PendingActivation to Active to Deactivated workflow
- Also led overall API architecture and hosting: Program.cs composition root, JWT/CORS configuration, IIS deployment script

8.2 Amarasekara S W N (Nisal) (IT23334892): Microgrid Nodes & Maps

- Web: Microgrid Node CRUD (GPS, capacity, battery slots, weekly schedule editor), deactivation guard
- Mobile: Nearby-stations map (Google Maps SDK + FusedLocationProvider), SQLite station cache for offline display
- API: StationsController/StationService, including the haversine-distance nearby-stations query
- Database: SolarStationInfo collection design

8.3 Vinoshan A G S (Sampath) (IT23386136): Reservations & Booking Workflow

- Web: Energy Slot Reservation management (create on behalf, approve, cancel) with 7-day/12-hour rule feedback
- Mobile: Reservation create/modify/cancel form, reservation detail with QR rendering, booking history with search, prosumer dashboard
- API: SlotsController/SlotService, ReservationsController/ReservationService (core CRUD and the two business rules), the dashboard aggregation endpoints
- Database: EnergyBookingSlots and EnergyReservation collection design

8.4 Dias M T D (Tharindu) (IT23135734): Grid Operator Operations & Service Integration

- Web: Grid Operator dashboard (pending/approved counts, battery-slot availability editor, pending-reservation monitor)
- Mobile: Operator dashboard, QR scan mode (CameraX + ML Kit), transfer-confirmation screen
- API: QR issue/verify/complete logic (QrTokenService, the approve/verify-qr/complete endpoints), the shared Retrofit/OkHttp client layer and SQLite session persistence on the mobile side
- Deployment: dotnet publish output verification and the elevated-PowerShell IIS deployment script

9. Challenges

- Android 15 (targetSdk 35) enforces edge-to-edge display by default. Our layouts did not account for this: the top header was drawn behind the status bar and the dashboard's bottom navigation bar was drawn behind the gesture-navigation bar, making it invisible. This only surfaced once the app was run on a real device (build success alone did not catch it). The first fix, adding `android:fitsSystemWindows="true"` to every activity's root layout, looked right in isolation but was actually wrong on screens where a `SwipeRefreshLayout/ScrollView` sits between the root and the gradient header: the root consumed the inset itself, pushing the whole screen down and exposing blank space above the header instead of extending the gradient under the status bar. The durable fix was to stop relying on the legacy attribute and instead call `WindowCompat.setDecorFitsSystemWindows(window, false)` and apply the status-bar/navigation-bar insets directly to the header and bottom-nav views in code (`EdgeToEdge.kt`), independent of whatever sits in between.
- A related CSS bug on the web console: Tailwind's `animation-fill-mode`: both on the sidebar/drawer entrance animations left a transform permanently applied to the animated element after the animation finished (`translateY(0)/translateX(0)` render identically to no transform, but the property itself lingers). A lingering transform on an ancestor makes that element a CSS containing block, which broke two unrelated things: modal/drawer overlays positioned with `position: fixed` were clipped to the transformed ancestor instead of the viewport, and, more surprising, a native `<select>` dropdown inside a transformed ancestor stopped opening at all in Chrome. Both were fixed by dropping the both `fill-mode` (or narrowing it to `backwards`), since the animation's final state and the element's natural CSS state are identical, so nothing needed to be retained.
- A related, subtler bug: even after the insets fix, the dashboard's Map/History/Profile buttons still rendered with invisible text. Dumping the view hierarchy with UI Automator confirmed the buttons existed with correct bounds, which pointed to a styling rather than layout problem: the platform style `?android:attr/borderlessButtonStyle` resolved an unreadable text colour under our `MaterialComponents` theme. Fixed by setting an explicit `textColor` on each affected button.
- Automating the emulator over adb (input tap/text) was flaky: taps issued during an activity transition sometimes "bled through" to the new screen, and the soft keyboard's autocorrect occasionally transposed characters in typed text. Both were mitigated by dumping the UI hierarchy for exact element bounds and verifying the resumed activity after every single action rather than chaining actions blindly.
- The 7-day booking-window and 12-hour update/cancel-notice rules had to be enforced in exactly one place (`ReservationService`) while still supporting two different callers (a prosumer booking for themselves from the mobile app, and Backoffice/Grid Operator staff booking on a prosumer's behalf from the web app) without duplicating the validation logic per caller type.
- Creating the IIS site and application pool requires Windows administrator elevation, which an automated development shell does not have. This was addressed by publishing the API and preparing a self-contained PowerShell script (`deploy-iis.ps1`) that a team member runs once in an elevated prompt.

10. References

- Microsoft, "ASP.NET Core documentation," <https://learn.microsoft.com/aspnet/core>
- Microsoft, "ASP.NET Core Module (IIS hosting)," <https://learn.microsoft.com/aspnet/core/host-and-deploy/aspnet-core-module>
- MongoDB Inc., "MongoDB C# Driver documentation," <https://www.mongodb.com/docs/drivers/csharp/current/>
- Auth0, "Introduction to JSON Web Tokens," <https://jwt.io/introduction>
- Meta, "React documentation," <https://react.dev/>
- Tailwind Labs, "Tailwind CSS documentation," <https://tailwindcss.com/docs>
- Vite, "Vite documentation," <https://vitejs.dev/guide/>
- Google, "Android Developers documentation," <https://developer.android.com/docs>
- Square, "Retrofit: A type-safe HTTP client for Android and Java," <https://square.github.io/retrofit/>
- Google, "Google Maps Platform: Maps SDK for Android," <https://developers.google.com/maps/documentation/android-sdk/overview>
- Google, "ML Kit: Barcode scanning," <https://developers.google.com/ml-kit/vision/barcode-scanning>
- ZXing Project, "ZXing ('Zebra Crossing') barcode image processing library," <https://github.com/zxing/zxing>
- Gradle Inc., "Gradle User Manual," <https://docs.gradle.org/current/userguide/userguide.html>

All source code listed in the appendices was written specifically for this project following the patterns described in the official documentation above; no third-party tutorial code was copied verbatim.

11. Git Repository and Demo Video

Repository link: https://github.com/Nisal2522/Solar_Microgrid.git

Individual contributions are also visible in the commit history and are summarised in Section 8 above.

Demo video (OneDrive, 4 min 49 s): https://mysliit-my.sharepoint.com/:f:/g/personal/it23334892_my_sliit_lk/IgCtGPX1-SZKTLuUgwzcuFFNAbd7-S9K6iZhpOfV5e6-ywM?e=BwX0Sj

The video walks through the Backoffice web console, the Solar Prosumer Android app and Grid Operator mode on both clients.

Appendix A: Web API Source Code (C#)

31 files. Every file carries a header comment block and inline comments before each method.

api/SolarMicrogrid.Api/Common/AppException.cs

```
// -----  
// File: AppException.cs  
// Purpose: Business-rule violation exception thrown by Services (e.g. the  
//          7-day booking window rule, the 12-hour notice rule). Caught by  
//          GlobalExceptionHandler and turned into a JSON error response.  
// Module owner: Member D (Service Integration)  
// -----  
namespace SolarMicrogrid.Api.Common;  
  
public class AppException : Exception  
{  
    public int StatusCode { get; }  
  
    // Creates a business-rule exception with an HTTP status code (default 400).  
    public AppException(string message, int statusCode = 400) : base(message)  
    {  
        StatusCode = statusCode;  
    }  
}
```

api/SolarMicrogrid.Api/Common/GlobalExceptionHandler.cs

```
// -----  
// File: GlobalExceptionHandler.cs  
// Purpose: Converts AppException (business-rule violations) and unexpected  
//          errors into consistent JSON error responses, so every Service  
//          can simply throw instead of building HTTP responses by hand.  
// Module owner: Member D (Service Integration)  
// -----  
using System.Text.Json;  
  
namespace SolarMicrogrid.Api.Common;  
  
public class GlobalExceptionHandler  
{  
    private readonly RequestDelegate _next;  
    private readonly ILogger<GlobalExceptionHandler> _logger;  
  
    public GlobalExceptionHandler(RequestDelegate next, ILogger<GlobalExceptionHandler> logger)  
    {  
        _next = next;  
        _logger = logger;  
    }  
  
    // Invokes the next middleware, catching AppException/unhandled errors into JSON.  
    public async Task InvokeAsync(HttpContext context)  
    {  
        try  
        {  
            await _next(context);  
        }  
        catch (AppException ex)  
        {  
            context.Response.ContentType = "application/json";  
            context.Response.StatusCode = ex.StatusCode;  
            await context.Response.WriteAsync(JsonSerializer.Serialize(new { message = ex.Message }));  
        }  
        catch (FormatException)  
        {  
            // Thrown by the Mongo driver when an id route/query parameter isn't a valid ObjectId.  
            context.Response.ContentType = "application/json";  
            context.Response.StatusCode = 400;  
            await context.Response.WriteAsync(JsonSerializer.Serialize(new { message = "Invalid identifier  
format." }));  
        }  
        catch (Exception ex)  
        {  
            _logger.LogError(ex, "Unhandled exception");  
            context.Response.StatusCode = 500;  
            await context.Response.WriteAsync(JsonSerializer.Serialize(new { message = "Internal server error" }));  
        }  
    }  
}
```

```

        {
            _logger.LogError(ex, "Unhandled exception");
            context.Response.ContentType = "application/json";
            context.Response.StatusCode = 500;
            await context.Response.WriteAsync(JsonSerializer.Serialize(new { message = "An unexpected error
occurred." }));
        }
    }
}

```

api/SolarMicrogrid.Api/Common/JwtTokenService.cs

```

// -----
// File: JwtTokenService.cs
// Purpose: Issues signed JWT access tokens carrying the user's id, role and
//          (for prosumers) NIC, used by both the web and mobile clients to
//          authenticate subsequent API calls.
// Module owner: Member A (Identity & Access)
// -----
using System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;
using System.Text;
using Microsoft.Extensions.Options;
using Microsoft.IdentityModel.Tokens;
using SolarMicrogrid.Api.Models;

namespace SolarMicrogrid.Api.Common;

public class JwtSettings
{
    public string Key { get; set; } = string.Empty;
    public string Issuer { get; set; } = string.Empty;
    public string Audience { get; set; } = string.Empty;
    public int ExpiryMinutes { get; set; } = 480;
}

public class JwtTokenService
{
    private readonly JwtSettings _settings;

    public JwtTokenService(IOptions<JwtSettings> settings)
    {
        _settings = settings.Value;
    }

    // Builds a signed JWT for the given user containing id/role/nic claims.
    public string GenerateToken(User user)
    {
        var claims = new List<Claim>
        {
            new(JwtRegisteredClaimNames.Sub, user.Id ?? string.Empty),
            new(ClaimTypes.Role, user.UserType.ToString()),
            new("fullName", user.FullName)
        };

        if (!string.IsNullOrEmpty(user.Nic))
        {
            claims.Add(new Claim("nic", user.Nic));
        }

        var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_settings.Key));
        var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

        var token = new JwtSecurityToken(
            issuer: _settings.Issuer,
            audience: _settings.Audience,
            claims: claims,
            expires: DateTime.UtcNow.AddMinutes(_settings.ExpiryMinutes),
            signingCredentials: creds);

        return new JwtSecurityTokenHandler().WriteToken(token);
    }
}

```

```
}
```

api/SolarMicrogrid.Api/Common/QrTokenService.cs

```
// -----  
// File: QrTokenService.cs  
// Purpose: Generates and verifies the secure transaction QR token embedded  
//          in an approved reservation, so the Grid Operator app can confirm  
//          a scanned code was genuinely issued by this server and matches  
//          the reservation being finalised.  
// Module owner: Member D (Grid Operator Verification)  
// -----  
using System.Security.Cryptography;  
using System.Text;  
using Microsoft.Extensions.Options;  
  
namespace SolarMicrogrid.Api.Common;  
  
public class QrTokenService  
{  
    private readonly JwtSettings _settings;  
  
    public QrTokenService(IOptions<JwtSettings> settings)  
    {  
        _settings = settings.Value;  
    }  
  
    // Builds a "reservationId.signature" token unique to this reservation.  
    public string GenerateToken(string reservationId, string reservationCode)  
    {  
        var payload = $"{{reservationId}}:{{reservationCode}}";  
        var signature = Sign(payload);  
        return $"{{payload}}.{{signature}}";  
    }  
  
    // Verifies a scanned token's signature and returns the embedded reservationId, or null if invalid.  
    public string? ValidateAndExtractReservationId(string token)  
    {  
        var lastDot = token.LastIndexOf('.');  
        if (lastDot <= 0) return null;  
  
        var payload = token[..lastDot];  
        var signature = token[(lastDot + 1)..];  
  
        if (Sign(payload) != signature) return null;  
  
        var parts = payload.Split(':');  
        return parts.Length == 2 ? parts[0] : null;  
    }  
  
    // Computes an HMAC-SHA256 signature of the payload using the shared JWT key.  
    private string Sign(string payload)  
    {  
        using var hmac = new HMACSHA256(Encoding.UTF8.GetBytes(_settings.Key));  
        var hash = hmac.ComputeHash(Encoding.UTF8.GetBytes(payload));  
        return Convert.ToString(hash);  
    }  
}
```

api/SolarMicrogrid.Api/Controllers/AuthController.cs

```
// -----  
// File: AuthController.cs  
// Purpose: Exposes the single shared login endpoint used by both the web  
//          app and the mobile app.  
// Module owner: Member A (Identity & Access)  
// -----  
using Microsoft.AspNetCore.Mvc;  
using SolarMicrogrid.Api.Dtos;  
using SolarMicrogrid.Api.Services;
```

```

namespace SolarMicrogrid.Api.Controllers;

[ApiController]
[Route("api/auth")]
public class AuthController : ControllerBase
{
    private readonly AuthService _authService;

    public AuthController(AuthService authService)
    {
        _authService = authService;
    }

    // Authenticates a staff (username) or prosumer (NIC) account and returns a JWT.
    [HttpPost("login")]
    public async Task<ActionResult<LoginResponse>> Login(LoginRequest request)
    {
        var result = await _authService.LoginAsync(request);
        return Ok(result);
    }
}

```

api/SolarMicrogrid.Api/Controllers/DashboardController.cs

```

// -----
// File: DashboardController.cs
// Purpose: Aggregated dashboard endpoints – active/pending counts and
//          upcoming bookings for the prosumer mobile dashboard, and the
//          pending queue + approved-future count for the Grid Operator
//          dashboard.
// Module owner: Member C (prosumer dashboard) / Member D (operator dashboard)
// -----
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using SolarMicrogrid.Api.Dtos;
using SolarMicrogrid.Api.Services;

namespace SolarMicrogrid.Api.Controllers;

[ApiController]
[Route("api/dashboard")]
[Authorize]
public class DashboardController : ControllerBase
{
    private readonly ReservationService _reservationService;

    public DashboardController(ReservationService reservationService)
    {
        _reservationService = reservationService;
    }

    // Returns the Grid Operator dashboard summary.
    [HttpGet("operator")]
    [Authorize(Roles = "Backoffice,GridOperator")]
    public async Task<ActionResult<OperatorDashboardResponse>> GetOperatorDashboard()
    {
        return Ok(await _reservationService.GetOperatorDashboardAsync());
    }

    // Returns a prosumer's mobile dashboard summary.
    [HttpGet("prosumer/{nic}")]
    public async Task<ActionResult<ProsumerDashboardResponse>> GetProsumerDashboard(string nic)
    {
        return Ok(await _reservationService.GetProsumerDashboardAsync(nic));
    }
}

```

api/SolarMicrogrid.Api/Controllers/ProsumersController.cs

```
// -----
// File: ProsumersController.cs
// Purpose: Prosumer self-registration (public) and self-service profile /
//          deactivation-request endpoints (authenticated prosumer only).
// Module owner: Member A (Identity & Access)
// -----
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using SolarMicrogrid.Api.Dtos;
using SolarMicrogrid.Api.Services;

namespace SolarMicrogrid.Api.Controllers;

[ApiController]
[Route("api/prosumers")]
public class ProsumersController : ControllerBase
{
    private readonly ProsumerService _prosumerService;

    public ProsumersController(ProsumerService prosumerService)
    {
        _prosumerService = prosumerService;
    }

    // Public endpoint: registers a new prosumer from the mobile app (starts PendingActivation).
    [HttpPost("register")]
    [AllowAnonymous]
    public async Task<ActionResult<UserResponse>> Register(RegisterProsumerRequest request)
    {
        var result = await _prosumerService.RegisterAsync(request);
        return Ok(result);
    }

    // Retrieves a prosumer profile by NIC.
    [HttpGet("{nic}")]
    [Authorize]
    public async Task<ActionResult<UserResponse>> GetByNic(string nic)
    {
        return Ok(await _prosumerService.GetByNicAsync(nic));
    }

    // Updates the prosumer's own profile.
    [HttpPut("{nic}")]
    [Authorize(Roles = "Prosumer")]
    public async Task<ActionResult<UserResponse>> UpdateProfile(string nic, UpdateProsumerRequest request)
    {
        return Ok(await _prosumerService.UpdateProfileAsync(nic, request));
    }

    // Uploads or replaces the prosumer's profile picture.
    [HttpPost("{nic}/photo")]
    [Authorize(Roles = "Prosumer")]
    [RequestSizeLimit(5 * 1024 * 1024)]
    public async Task<ActionResult<UserResponse>> UploadPhoto(string nic, IFormFile? file)
    {
        return Ok(await _prosumerService.UploadPhotoAsync(nic, file));
    }

    // Prosumer requests their own account be deactivated.
    [HttpPut("{nic}/request-deactivation")]
    [Authorize(Roles = "Prosumer")]
    public async Task<ActionResult<UserResponse>> RequestDeactivation(string nic)
    {
        return Ok(await _prosumerService.RequestDeactivationAsync(nic));
    }
}

```

api/SolarMicrogrid.Api/Controllers/ReservationsController.cs

```
// -----
// File: ReservationsController.cs
// Purpose: Create/update/cancel reservations (Prosumer self-service via

```

```

//      mobile, or Backoffice/GridOperator on a prosumer's behalf via
//      web), plus the approve, QR-verify and complete workflow used by
//      Grid Operators.
// Module owner: Member C (create/update/cancel/history) /
//      Member D (approve/verify-qr/complete)
// -----
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using SolarMicrogrid.Api.Dtos;
using SolarMicrogrid.Api.Services;

namespace SolarMicrogrid.Api.Controllers;

[ApiController]
[Route("api/reservations")]
[Authorize]
public class ReservationsController : ControllerBase
{
    private readonly ReservationService _reservationService;

    public ReservationsController(ReservationService reservationService)
    {
        _reservationService = reservationService;
    }

    // Retrieves a single reservation by id (used by the mobile detail/edit screens).
    [HttpGet("{id}")]
    public async Task<ActionResult<ReservationResponse>> GetById(string id)
    {
        return Ok(await _reservationService.GetByIdAsync(id));
    }

    // Creates a reservation; Prosumers book for themselves, staff book on a prosumer's behalf.
    [HttpPost]
    public async Task<ActionResult<ReservationResponse>> Create(CreateReservationRequest request)
    {
        var callerNic = User.IsInRole("Prosumer") ? User.FindFirst("nic")?.Value : null;
        return Ok(await _reservationService.CreateAsync(callerNic, request));
    }

    // Updates a reservation's slot/time/amount (12-hour notice rule enforced by the service).
    [HttpPut("{id}")]
    public async Task<ActionResult<ReservationResponse>> Update(string id, UpdateReservationRequest request)
    {
        var callerNic = User.IsInRole("Prosumer") ? User.FindFirst("nic")?.Value : null;
        return Ok(await _reservationService.UpdateAsync(id, callerNic, request));
    }

    // Cancels a reservation and frees its slot capacity (12-hour notice rule enforced by the service).
    [HttpPut("{id}/cancel")]
    public async Task<ActionResult<ReservationResponse>> Cancel(string id, CancelReservationRequest request)
    {
        var callerNic = User.IsInRole("Prosumer") ? User.FindFirst("nic")?.Value : null;
        return Ok(await _reservationService.CancelAsync(id, callerNic, request));
    }

    // Grid Operator/Backoffice approves a pending reservation, issuing its QR token.
    [HttpPut("{id}/approve")]
    [Authorize(Roles = "Backoffice,GridOperator")]
    public async Task<ActionResult<ReservationResponse>> Approve(string id)
    {
        var approvedBy = User.FindFirst("fullName")?.Value ?? "Operator";
        return Ok(await _reservationService.ApproveAsync(id, approvedBy));
    }

    // Grid Operator scans a prosumer's QR code; verifies it against the server record.
    [HttpPost("verify-qr")]
    [Authorize(Roles = "GridOperator")]
    public async Task<ActionResult<ReservationResponse>> VerifyQr(VerifyQrRequest request)
    {
        return Ok(await _reservationService.VerifyQrAsync(request));
    }

    // Grid Operator finalises the energy transfer after a successful QR verification.

```

```

[HttpPut("{id}/complete")]
[Authorize(Roles = "GridOperator")]
public async Task<ActionResult<ReservationResponse>> Complete(string id)
{
    var completedBy = User.FindFirst("fullName)?.Value ?? "Operator";
    return Ok(await _reservationService.CompleteAsync(id, completedBy));
}

// Lists a prosumer's full booking history (self, or Backoffice/GridOperator looking it up).
[HttpGet("prosumer/{nic}")]
public async Task<ActionResult<List<ReservationResponse>>> GetByProsumer(string nic)
{
    return Ok(await _reservationService.GetByProsumerAsync(nic));
}

// Lists all reservations awaiting approval.
[HttpGet("pending")]
[Authorize(Roles = "Backoffice,GridOperator")]
public async Task<ActionResult<List<ReservationResponse>>> GetPending()
{
    return Ok(await _reservationService.GetPendingAsync());
}
}

```

api/SolarMicrogrid.Api/Controllers/SlotsController.cs

```

// -----
// File: SlotsController.cs
// Purpose: Creates and lists bookable energy slots for a given station.
// Module owner: Member C (Reservations & Booking Workflow)
// -----
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using SolarMicrogrid.Api.Dtos;
using SolarMicrogrid.Api.Services;

namespace SolarMicrogrid.Api.Controllers;

[ApiController]
[Route("api/stations/{stationId}/slots")]
[Authorize]
public class SlotsController : ControllerBase
{
    private readonly SlotService _slotService;

    public SlotsController(SlotService slotService)
    {
        _slotService = slotService;
    }

    // Generates a new bookable slot for the station (Backoffice/GridOperator only).
    [HttpPost]
    [Authorize(Roles = "Backoffice,GridOperator")]
    public async Task<ActionResult<SlotResponse>> Create(string stationId, CreateSlotRequest request)
    {
        return Ok(await _slotService.CreateAsync(stationId, request));
    }

    // Lists all slots for the station, so prosumers can pick one when booking.
    [HttpGet]
    public async Task<ActionResult<List<SlotResponse>>> GetByStation(string stationId)
    {
        return Ok(await _slotService.GetByStationAsync(stationId));
    }
}

```

api/SolarMicrogrid.Api/Controllers/StationsController.cs

```

// -----
// File: StationsController.cs

```

```

// Purpose: Microgrid node (solar station) CRUD for Backoffice, plus the
//          public nearby-stations query used by the mobile map screen.
// Module owner: Member B (Microgrid Nodes & Maps)
// -----
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using SolarMicrogrid.Api.Dtos;
using SolarMicrogrid.Api.Services;

namespace SolarMicrogrid.Api.Controllers;

[ApiController]
[Route("api/stations")]
public class StationsController : ControllerBase
{
    private readonly StationService _stationService;

    public StationsController(StationService stationService)
    {
        _stationService = stationService;
    }

    // Creates a new microgrid node (Backoffice only).
    [HttpPost]
    [Authorize(Roles = "Backoffice")]
    public async Task<ActionResult<StationResponse>> Create(CreateStationRequest request)
    {
        var createdBy = User.FindFirst("fullName")?.Value ?? "Backoffice";
        return Ok(await _stationService.CreateAsync(request, createdBy));
    }

    // Lists all stations (any authenticated user).
    [HttpGet]
    [Authorize]
    public async Task<ActionResult<List<StationResponse>>> GetAll()
    {
        return Ok(await _stationService.GetAllAsync());
    }

    // Returns active stations within radiusKm of a GPS point, for the mobile map.
    [HttpGet("nearby")]
    [Authorize]
    public async Task<ActionResult<List<StationResponse>>> GetNearby([FromQuery] double lat, [FromQuery] double lng, [FromQuery] double radiusKm = 25)
    {
        return Ok(await _stationService.GetNearbyAsync(lat, lng, radiusKm));
    }

    // Retrieves a single station by id.
    [HttpGet("{id}")]
    [Authorize]
    public async Task<ActionResult<StationResponse>> GetById(string id)
    {
        return Ok(await _stationService.GetByIdAsync(id));
    }

    // Updates a station's details, capacity and schedule (Backoffice only).
    [HttpPut("{id}")]
    [Authorize(Roles = "Backoffice")]
    public async Task<ActionResult<StationResponse>> Update(string id, UpdateStationRequest request)
    {
        return Ok(await _stationService.UpdateAsync(id, request));
    }

    // Deactivates a station, blocked if active reservations exist (Backoffice only).
    [HttpPut("{id}/deactivate")]
    [Authorize(Roles = "Backoffice")]
    public async Task<ActionResult<StationResponse>> Deactivate(string id)
    {
        return Ok(await _stationService.DeactivateAsync(id));
    }

    // Tops up bookable slots from the station's schedule (Backoffice/GridOperator only).
    [HttpPost("{id}/generate-slots")]

```

```

[Authorize(Roles = "Backoffice,GridOperator")]
public async Task<ActionResult<GenerateSlotsResponse>> GenerateSlots(string id)
{
    var created = await _stationService.GenerateSlotsAsync(id);
    return Ok(new GenerateSlotsResponse { Created = created });
}
}

```

api/SolarMicrogrid.Api/Controllers/UsersController.cs

```

// -----
// File: UsersController.cs
// Purpose: Backoffice-only endpoints for staff user creation, the pending
//         activation queue, and activate/deactivate/reactivate actions.
// Module owner: Member A (Identity & Access)
// -----
using System.Security.Claims;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using SolarMicrogrid.Api.Dtos;
using SolarMicrogrid.Api.Services;

namespace SolarMicrogrid.Api.Controllers;

[ApiController]
[Route("api/users")]
[Authorize(Roles = "Backoffice")]
public class UsersController : ControllerBase
{
    private readonly UserService _userService;

    public UsersController(UserService userService)
    {
        _userService = userService;
    }

    // Creates a new Backoffice or Grid Operator staff account.
    [HttpPost]
    public async Task<ActionResult<UserResponse>> Create(CreateStaffUserRequest request)
    {
        var createdBy = User.FindFirstValue("fullName") ?? "Backoffice";
        var result = await _userService.CreateStaffUserAsync(request, createdBy);
        return Ok(result);
    }

    // Lists all staff (Backoffice/GridOperator) accounts.
    [HttpGet]
    public async Task<ActionResult<List<UserResponse>>> GetAll()
    {
        return Ok(await _userService.GetStaffUsersAsync());
    }

    // Lists accounts (staff or prosumer) awaiting activation.
    [HttpGet("pending")]
    public async Task<ActionResult<List<UserResponse>>> GetPending()
    {
        return Ok(await _userService.GetPendingActivationsAsync());
    }

    // Approves a pending account.
    [HttpPut("{id}/approve")]
    public async Task<ActionResult<UserResponse>> Approve(string id)
    {
        return Ok(await _userService.ApproveAsync(id));
    }

    // Deactivates an account.
    [HttpPut("{id}/deactivate")]
    public async Task<ActionResult<UserResponse>> Deactivate(string id)
    {
        return Ok(await _userService.DeactivateAsync(id));
    }
}

```

```

// Reactivates a previously deactivated account.
[HttpPut("{id}/reactivate")]
public async Task<ActionResult<UserResponse>> Reactivate(string id)
{
    return Ok(await _userService.ReactivateAsync(id));
}
}

```

api/SolarMicrogrid.Api/Data/DbSeeder.cs

```

// -----
// File: DbSeeder.cs
// Purpose: Ensures a first Backoffice administrator account exists on
//          startup, so the system is reachable before any manual data
//          entry – required to bootstrap login on a fresh MongoDB database.
// Module owner: Member D (Service Integration & Hosting)
// -----
using MongoDB.Driver;
using SolarMicrogrid.Api.Models;

namespace SolarMicrogrid.Api.Data;

public static class DbSeeder
{
    // Inserts a default "admin" Backoffice account if the Users collection is empty.
    public static async Task SeedAsync(MongoDbContext db)
    {
        var hasAnyUser = await db.Users.Find(_ => true).AnyAsync();
        if (hasAnyUser) return;

        var admin = new User
        {
            UserType = UserType.Backoffice,
            FullName = "System Administrator",
            Email = "admin@solarmicrogrid.local",
            Phone = "0000000000",
            Username = "admin",
            PasswordHash = BCrypt.Net.BCrypt.HashPassword("Admin@123"),
            Status = UserStatus.Active,
            CreatedBy = "system-seed"
        };

        await db.Users.InsertOneAsync(admin);
    }
}

```

api/SolarMicrogrid.Api/Data/MongoDbContext.cs

```

// -----
// File: MongoDbContext.cs
// Purpose: Central access point to the MongoDB database and its four typed
//          collections. Injected as a singleton into every Service.
// Module owner: Member D (Service Integration & Hosting)
// -----
using Microsoft.Extensions.Options;
using MongoDB.Driver;
using SolarMicrogrid.Api.Models;

namespace SolarMicrogrid.Api.Data;

public class MongoDbContext
{
    // Opens the Mongo client/database using the bound MongoDbSettings.
    public MongoDbContext(IOptions<MongoDbSettings> settings)
    {
        var client = new MongoClient(settings.Value.ConnectionString);
        var database = client.GetDatabase(settings.Value.DatabaseName);

        Users = database.GetCollection<User>("Users");
    }
}

```

```

        Stations = database.GetCollection<SolarStation>("SolarStationInfo");
        BookingSlots = database.GetCollection<EnergyBookingSlot>("EnergyBookingSlots");
        Reservations = database.GetCollection<EnergyReservation>("EnergyReservation");
    }

    public IMongoCollection<User> Users { get; }
    public IMongoCollection<SolarStation> Stations { get; }
    public IMongoCollection<EnergyBookingSlot> BookingSlots { get; }
    public IMongoCollection<EnergyReservation> Reservations { get; }
}

```

api/SolarMicrogrid.Api/Data/MongoDbSettings.cs

```

// -----
// File: MongoDbSettings.cs
// Purpose: Strongly-typed binding of the "MongoDb" section in appsettings.json
//          (connection string + database name).
// Module owner: Member D (Service Integration & Hosting)
// -----
namespace SolarMicrogrid.Api.Data;

public class MongoDbSettings
{
    public string ConnectionString { get; set; } = string.Empty;
    public string DatabaseName { get; set; } = string.Empty;
}

```

api/SolarMicrogrid.Api/Dtos/AuthDtos.cs

```

// -----
// File: AuthDtos.cs
// Purpose: Request/response payloads for the login endpoint shared by the
//          web app (Backoffice/GridOperator) and the mobile app
//          (Prosumer/GridOperator).
// Module owner: Member A (Identity & Access)
// -----
namespace SolarMicrogrid.Api.Dtos;

public class LoginRequest
{
    // Username for staff logins, or NIC for prosumer logins.
    public string Identifier { get; set; } = string.Empty;
    public string Password { get; set; } = string.Empty;
}

public class LoginResponse
{
    public string Token { get; set; } = string.Empty;
    public string UserId { get; set; } = string.Empty;
    public string UserType { get; set; } = string.Empty;
    public string FullName { get; set; } = string.Empty;
    public string? Nic { get; set; }
}

```

api/SolarMicrogrid.Api/Dtos/ProsumerDtos.cs

```

// -----
// File: ProsumerDtos.cs
// Purpose: Request payloads for prosumer self-registration (mobile) and
//          self-service profile edits.
// Module owner: Member A (Identity & Access)
// -----
namespace SolarMicrogrid.Api.Dtos;

public class RegisterProsumerRequest
{
    public string Nic { get; set; } = string.Empty;
    public string FullName { get; set; } = string.Empty;
    public string Email { get; set; } = string.Empty;
}

```

```

    public string Phone { get; set; } = string.Empty;
    public string Address { get; set; } = string.Empty;
    public string Password { get; set; } = string.Empty;
}

public class UpdateProsumerRequest
{
    public string FullName { get; set; } = string.Empty;
    public string Email { get; set; } = string.Empty;
    public string Phone { get; set; } = string.Empty;
    public string Address { get; set; } = string.Empty;
}

```

api/SolarMicrogrid.Api/Dtos/ReservationDtos.cs

```

// -----
// File: ReservationDtos.cs
// Purpose: Request/response payloads for creating, updating, cancelling and
//          viewing energy slot reservations, plus the QR-verification
//          payload used by the Grid Operator mobile mode.
// Module owner: Member C (Reservations & Booking Workflow) /
//               Member D (QR verification)
// -----
using SolarMicrogrid.Api.Models;

namespace SolarMicrogrid.Api.Dtos;

public class CreateReservationRequest
{
    // Only required when a Backoffice/GridOperator creates a booking on a prosumer's behalf;
    // ignored (the caller's own NIC is used) when a Prosumer creates their own booking.
    public string? ProsumerNic { get; set; }
    public string StationId { get; set; } = string.Empty;
    public string SlotId { get; set; } = string.Empty;
    public DateTime ScheduledDateTime { get; set; }
    public double EnergyAmountKWh { get; set; }
}

public class UpdateReservationRequest
{
    public string SlotId { get; set; } = string.Empty;
    public DateTime ScheduledDateTime { get; set; }
    public double EnergyAmountKWh { get; set; }
}

public class CancelReservationRequest
{
    public string Reason { get; set; } = string.Empty;
}

public class VerifyQrRequest
{
    public string QrToken { get; set; } = string.Empty;
}

public class ReservationResponse
{
    public string Id { get; set; } = string.Empty;
    public string ReservationCode { get; set; } = string.Empty;
    public string ProsumerNic { get; set; } = string.Empty;
    public string StationId { get; set; } = string.Empty;
    public string SlotId { get; set; } = string.Empty;
    public DateTime ScheduledDateTime { get; set; }
    public double EnergyAmountKWh { get; set; }
    public ReservationStatus Status { get; set; }
    public string? QrToken { get; set; }
    public DateTime CreatedAt { get; set; }

    // Maps an EnergyReservation document to the response shape.
    public static ReservationResponse FromModel(EnergyReservation r) => new()
    {
        Id = r.Id ?? string.Empty,

```

```

        ReservationCode = r.ReservationCode,
        ProsumerNic = r.ProsumerNic,
        StationId = r.StationId,
        SlotId = r.SlotId,
        ScheduledDateTime = r.ScheduledDateTime,
        EnergyAmountKWh = r.EnergyAmountKWh,
        Status = r.Status,
        QrToken = r.QrToken,
        CreatedAt = r.CreatedAt
    };
}

public class ProsumerDashboardResponse
{
    public int ActiveCount { get; set; }
    public int PendingCount { get; set; }
    public List<ReservationResponse> UpcomingReservations { get; set; } = new();
}

public class OperatorDashboardResponse
{
    public int PendingReservationsCount { get; set; }
    public int ApprovedFutureReservationsCount { get; set; }
    public List<ReservationResponse> PendingReservations { get; set; } = new();
}

```

api/SolarMicrogrid.Api/Dtos/SlotDtos.cs

```

// -----
// File: SlotDtos.cs
// Purpose: Request/response payloads for generating and listing energy
//          booking slots against a station.
// Module owner: Member C (Reservations & Booking Workflow)
// -----
using SolarMicrogrid.Api.Models;

namespace SolarMicrogrid.Api.Dtos;

public class CreateSlotRequest
{
    public DateTime Date { get; set; }
    public string StartTime { get; set; } = string.Empty;
    public string EndTime { get; set; } = string.Empty;
    public SlotType SlotType { get; set; }
    public int CapacityTotal { get; set; }
}

public class SlotResponse
{
    public string Id { get; set; } = string.Empty;
    public string StationId { get; set; } = string.Empty;
    public DateTime Date { get; set; }
    public string StartTime { get; set; } = string.Empty;
    public string EndTime { get; set; } = string.Empty;
    public SlotType SlotType { get; set; }
    public int CapacityTotal { get; set; }
    public int CapacityAvailable { get; set; }
    public SlotStatus Status { get; set; }

    // Maps an EnergyBookingSlot document to the response shape.
    public static SlotResponse FromModel(EnergyBookingSlot s) => new()
    {
        Id = s.Id ?? string.Empty,
        StationId = s.StationId,
        Date = s.Date,
        StartTime = s.StartTime,
        EndTime = s.EndTime,
        SlotType = s.SlotType,
        CapacityTotal = s.CapacityTotal,
        CapacityAvailable = s.CapacityAvailable,
        Status = s.Status
    };
}

```

api/SolarMicrogrid.Api/Dtos/StationDtos.cs

```
// -----  
// File: StationDtos.cs  
// Purpose: Request/response payloads for microgrid node (SolarStationInfo)  
//          management and the nearby-stations map query.  
// Module owner: Member B (Microgrid Nodes & Maps)  
// -----  
using SolarMicrogrid.Api.Models;  
  
namespace SolarMicrogrid.Api.Dtos;  
  
public class CreateStationRequest  
{  
    public string StationName { get; set; } = string.Empty;  
    public double Lat { get; set; }  
    public double Lng { get; set; }  
    public double CapacityKWh { get; set; }  
    public int TotalBatterySlots { get; set; }  
    public List<ScheduleEntry> Schedule { get; set; } = new();  
}  
  
public class UpdateStationRequest  
{  
    public string StationName { get; set; } = string.Empty;  
    public double Lat { get; set; }  
    public double Lng { get; set; }  
    public double CapacityKWh { get; set; }  
    public int TotalBatterySlots { get; set; }  
    public int AvailableBatterySlots { get; set; }  
    public List<ScheduleEntry> Schedule { get; set; } = new();  
}  
  
public class StationResponse  
{  
    public string Id { get; set; } = string.Empty;  
    public string StationName { get; set; } = string.Empty;  
    public double Lat { get; set; }  
    public double Lng { get; set; }  
    public double CapacityKWh { get; set; }  
    public int TotalBatterySlots { get; set; }  
    public int AvailableBatterySlots { get; set; }  
    public List<ScheduleEntry> Schedule { get; set; } = new();  
    public StationStatus Status { get; set; }  
  
    // Maps a SolarStation document to the response shape.  
    public static StationResponse FromModel(SolarStation s) => new()  
    {  
        Id = s.Id ?? string.Empty,  
        StationName = s.StationName,  
        Lat = s.GpsLocation.Lat,  
        Lng = s.GpsLocation.Lng,  
        CapacityKWh = s.CapacityKWh,  
        TotalBatterySlots = s.TotalBatterySlots,  
        AvailableBatterySlots = s.AvailableBatterySlots,  
        Schedule = s.Schedule,  
        Status = s.Status  
    };  
}  
  
public class GenerateSlotsResponse  
{  
    public int Created { get; set; }  
}
```

api/SolarMicrogrid.Api/Dtos/UserDtos.cs

```
// -----  
// File: UserDtos.cs  
// Purpose: Request/response payloads for Backoffice-managed staff user
```

```
//          accounts (Backoffice + Grid Operator) and the pending-activation
//          review list.
// Module owner: Member A (Identity & Access)
// -----
using SolarMicrogrid.Api.Models;

namespace SolarMicrogrid.Api.Dtos;

public class CreateStaffUserRequest
{
    public UserType UserType { get; set; } // Backoffice or GridOperator
    public string FullName { get; set; } = string.Empty;
    public string Email { get; set; } = string.Empty;
    public string Phone { get; set; } = string.Empty;
    public string Username { get; set; } = string.Empty;
    public string Password { get; set; } = string.Empty;
}

public class UserResponse
{
    public string Id { get; set; } = string.Empty;
    public UserType UserType { get; set; }
    public string FullName { get; set; } = string.Empty;
    public string Email { get; set; } = string.Empty;
    public string Phone { get; set; } = string.Empty;
    public string? Username { get; set; }
    public string? Nic { get; set; }
    public string? Address { get; set; }
    public string? PhotoUrl { get; set; }
    public UserStatus Status { get; set; }
    public DateTime CreatedAt { get; set; }

    // Maps a User document to the response shape, hiding the password hash.
    public static UserResponse FromModel(User user) => new()
    {
        Id = user.Id ?? string.Empty,
        UserType = user.UserType,
        FullName = user.FullName,
        Email = user.Email,
        Phone = user.Phone,
        Username = user.Username,
        Nic = user.Nic,
        Address = user.Address,
        PhotoUrl = user.PhotoUrl,
        Status = user.Status,
        CreatedAt = user.CreatedAt
    };
}
```

api/SolarMicrogrid.Api/Models/EnergyBookingSlot.cs

```
// -----
// File: EnergyBookingSlot.cs
// Purpose: MongoDB document model for the "EnergyBookingSlots" collection –
//          bookable time windows generated against a SolarStation that
//          prosumers reserve via EnergyReservation documents.
// Module owner: Member C (Reservations & Booking Workflow)
// -----
using MongoDB.Bson;
using MongoDB.Bson.Serialization.Attributes;

namespace SolarMicrogrid.Api.Models;

public enum SlotType
{
    Charging,
    DropOff
}

public enum SlotStatus
{
    Open,
    Full,
}
```

```

        Closed
    }

    public class EnergyBookingSlot
    {
        [BsonId]
        [BsonRepresentation(BsonType.ObjectId)]
        public string? Id { get; set; }

        [BsonRepresentation(BsonType.ObjectId)]
        public string StationId { get; set; } = string.Empty;

        public DateTime Date { get; set; }
        public string StartTime { get; set; } = string.Empty;
        public string EndTime { get; set; } = string.Empty;
        public SlotType SlotType { get; set; }

        public int CapacityTotal { get; set; }
        public int CapacityAvailable { get; set; }

        public SlotStatus Status { get; set; } = SlotStatus.Open;

        public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
        public DateTime UpdatedAt { get; set; } = DateTime.UtcNow;
    }

```

api/SolarMicrogrid.Api/Models/EnergyReservation.cs

```

// -----
// File: EnergyReservation.cs
// Purpose: MongoDB document model for the "EnergyReservation" collection – a
//          prosumer's power-trading booking against an EnergyBookingSlot,
//          including the QR-dispatch and grid-operator verification fields.
// Module owner: Member C (Reservations & Booking Workflow) /
//               Member D (QR verification, completion)
// -----
using MongoDB.Bson;
using MongoDB.Bson.Serialization.Attributes;

namespace SolarMicrogrid.Api.Models;

public enum ReservationStatus
{
    Pending,
    Approved,
    Completed,
    Cancelled
}

public class EnergyReservation
{
    [BsonId]
    [BsonRepresentation(BsonType.ObjectId)]
    public string? Id { get; set; }

    public string ReservationCode { get; set; } = string.Empty;
    public string ProsumerNic { get; set; } = string.Empty;

    [BsonRepresentation(BsonType.ObjectId)]
    public string StationId { get; set; } = string.Empty;

    [BsonRepresentation(BsonType.ObjectId)]
    public string SlotId { get; set; } = string.Empty;

    public DateTime ScheduledDateTime { get; set; }
    public double EnergyAmountKWh { get; set; }

    public ReservationStatus Status { get; set; } = ReservationStatus.Pending;
}

```

```

    public string? QrToken { get; set; }

    public string? ApprovedBy { get; set; }
    public DateTime? ApprovedAt { get; set; }
    public string? CompletedBy { get; set; }
    public DateTime? CompletedAt { get; set; }
    public DateTime? CancelledAt { get; set; }
    public string? CancelReason { get; set; }

    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
    public DateTime UpdatedAt { get; set; } = DateTime.UtcNow;
}

```

api/SolarMicrogrid.Api/Models/SolarStation.cs

```

// -----
// File: SolarStation.cs
// Purpose: MongoDB document model for the "SolarStationInfo" collection -
//         the physical microgrid hub/node (GPS location, capacity, battery
//         slots, weekly operating schedule).
// Module owner: Member B (Microgrid Nodes & Maps)
// -----
using MongoDB.Bson;
using MongoDB.Bson.Serialization.Attributes;

namespace SolarMicrogrid.Api.Models;

public enum StationStatus
{
    Active,
    Deactivated
}

public class GpsLocation
{
    public double Lat { get; set; }
    public double Lng { get; set; }
}

public class ScheduleEntry
{
    // Day of week, e.g. "Monday"
    public string Day { get; set; } = string.Empty;
    public string OpenTime { get; set; } = "08:00";
    public string CloseTime { get; set; } = "18:00";
}

public class SolarStation
{
    [BsonId]
    [BsonRepresentation(BsonType.ObjectId)]
    public string? Id { get; set; }

    public string StationName { get; set; } = string.Empty;
    public GpsLocation GpsLocation { get; set; } = new();
    public double CapacityKWh { get; set; }
    public int TotalBatterySlots { get; set; }
    public int AvailableBatterySlots { get; set; }
    public List<ScheduleEntry> Schedule { get; set; } = new();

    public StationStatus Status { get; set; } = StationStatus.Active;

    public string? CreatedBy { get; set; }
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
    public DateTime UpdatedAt { get; set; } = DateTime.UtcNow;
}

```

api/SolarMicrogrid.Api/Models/User.cs

```
// -----
// File: User.cs
// Purpose: MongoDB document model for the "Users" collection. Represents
//          Backoffice staff, Grid Operator staff, and Prosumer accounts in a
//          single role-discriminated collection.
// Module owner: Member A (Identity & Access)
// -----
using MongoDB.Bson;
using MongoDB.Bson.Serialization.Attributes;

namespace SolarMicrogrid.Api.Models;

public enum UserType
{
    Backoffice,
    GridOperator,
    Prosumer
}

public enum UserStatus
{
    PendingActivation,
    Active,
    Deactivated
}

public class User
{
    [BsonId]
    [BsonRepresentation(BsonType.ObjectId)]
    public string? Id { get; set; }

    public UserType UserType { get; set; }

    // Shared identity fields
    public string FullName { get; set; } = string.Empty;
    public string Email { get; set; } = string.Empty;
    public string Phone { get; set; } = string.Empty;

    // Staff (Backoffice / GridOperator) login fields
    public string? Username { get; set; }
    public string? PasswordHash { get; set; }

    // Prosumer fields – NIC is the prosumer's natural primary key
    public string? Nic { get; set; }
    public string? Address { get; set; }
    public string? PhotoUrl { get; set; }

    public UserStatus Status { get; set; } = UserStatus.Active;

    public string? CreatedBy { get; set; }
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
    public DateTime UpdatedAt { get; set; } = DateTime.UtcNow;
}

```

api/SolarMicrogrid.Api/Program.cs

```
// -----
// File: Program.cs
// Purpose: Application composition root – configures MongoDB, JWT
//          authentication/authorization, CORS for the web/mobile clients,
//          dependency injection for every Service, and the middleware
//          pipeline (FAT-service Web API entry point).
// Module owner: Member D (Service Integration & Hosting)
// -----
using System.Text;
using System.Text.Json.Serialization;
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.IdentityModel.Tokens;
using SolarMicrogrid.Api.Common;
using SolarMicrogrid.Api.Data;

```

```

using SolarMicrogrid.Api.Services;

var builder = WebApplication.CreateBuilder(args);

// Ensure the uploads folder exists before the static-file provider initializes.
Directory.CreateDirectory(Path.Combine(builder.Environment.WebRootPath ?? Path.Combine(builder.Environment.ContentRootPath, "wwwroot"), "uploads", "prosumers"));

// Bind MongoDB and JWT settings from appsettings.json.
builder.Services.Configure<MongoDbSettings>(builder.Configuration.GetSection("MongoDb"));
builder.Services.Configure<JwtSettings>(builder.Configuration.GetSection("Jwt"));

// Register the Mongo context and every FAT-service business-logic service.
builder.Services.AddSingleton<MongoDbContext>();
builder.Services.AddSingleton<JwtTokenService>();
builder.Services.AddSingleton<QrTokenService>();
builder.Services.AddScoped<AuthService>();
builder.Services.AddScoped<UserService>();
builder.Services.AddScoped<ProsumerService>();
builder.Services.AddScoped<StationService>();
builder.Services.AddScoped<SlotService>();
builder.Services.AddScoped<ReservationService>();

// Serialize enums (UserType, SlotType, ReservationStatus, ...) as their string names, not ints.
builder.Services.AddControllers()
    .AddJsonOptions(options => options.JsonSerializerOptions.Converters.Add(new JsonStringEnumConverter()));
builder.Services.AddOpenApi();

// Allow the React web app (Vite dev server) and the Android emulator to call the API.
builder.Services.AddCors(options =>
{
    options.AddPolicy("ClientApps", policy =>
    {
        policy.AllowAnyHeader().AllowAnyMethod().SetIsOriginAllowed(_ => true);
    });
});

var jwtSettings = builder.Configuration.GetSection("Jwt").Get<JwtSettings>() ?? new JwtSettings();

builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,
            ValidIssuer = jwtSettings.Issuer,
            ValidAudience = jwtSettings.Audience,
            IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwtSettings.Key))
        };
    });
builder.Services.AddAuthorization();

var app = builder.Build();

// Seed the default Backoffice admin account on a fresh database.
await DbSeeder.SeedAsync(app.Services.GetRequiredService<MongoDbContext>());

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    app.MapOpenApi();
}

app.UseMiddleware<GlobalExceptionMiddleware>();

app.UseHttpsRedirection();

// Serves uploaded prosumer photos from wwwroot/uploads.
app.UseStaticFiles();

```

```

app.UseCors("ClientApps");

app.UseAuthentication();
app.UseAuthorization();

app.MapControllers();

app.Run();

```

api/SolarMicrogrid.Api/Services/AuthService.cs

```

// -----
// File: AuthService.cs
// Purpose: Verifies staff (username) and prosumer (NIC) credentials against
//          the Users collection and issues a JWT on success. All login
//          business logic (status checks, password verification) lives here
//          per the FAT-service pattern.
// Module owner: Member A (Identity & Access)
// -----
using MongoDB.Driver;
using SolarMicrogrid.Api.Common;
using SolarMicrogrid.Api.Data;
using SolarMicrogrid.Api.Dtos;
using SolarMicrogrid.Api.Models;

namespace SolarMicrogrid.Api.Services;

public class AuthService
{
    private readonly MongoDbContext _db;
    private readonly JwtTokenService _tokenService;

    public AuthService(MongoDbContext db, JwtTokenService tokenService)
    {
        _db = db;
        _tokenService = tokenService;
    }

    // Authenticates by username (staff) or NIC (prosumer) and returns a signed token.
    public async Task<LoginResponse> LoginAsync(LoginRequest request)
    {
        var filter = Builders<User>.Filter.Or(
            Builders<User>.Filter.Eq(u => u.Username, request.Identifier),
            Builders<User>.Filter.Eq(u => u.Nic, request.Identifier));

        var user = await _db.Users.Find(filter).FirstOrDefaultAsync()
            ?? throw new AppException("Invalid credentials.", 401);

        if (string.IsNullOrEmpty(user.PasswordHash) || !BCrypt.Net.BCrypt.Verify(request.Password, user.PasswordHash))
        {
            throw new AppException("Invalid credentials.", 401);
        }

        if (user.Status == UserStatus.PendingActivation)
        {
            throw new AppException("Your account is pending Backoffice activation.", 403);
        }

        if (user.Status == UserStatus.Deactivated)
        {
            throw new AppException("Your account has been deactivated. Contact Backoffice to reactivate.",
403);
        }

        var token = _tokenService.GenerateToken(user);

        return new LoginResponse
        {

```

```

        Token = token,
        UserId = user.Id ?? string.Empty,
        UserType = user.UserType.ToString(),
        FullName = user.FullName,
        Nic = user.Nic
    };
}
}

```

api/SolarMicrogrid.Api/Services/ProsumerService.cs

```

// -----
// File: ProsumerService.cs
// Purpose: Prosumer self-registration (NIC as primary key, starts
//         PendingActivation), profile lookup/edit, and self-service
//         deactivation requests from the mobile app.
// Module owner: Member A (Identity & Access)
// -----
using MongoDB.Driver;
using SolarMicrogrid.Api.Common;
using SolarMicrogrid.Api.Data;
using SolarMicrogrid.Api.Dtos;
using SolarMicrogrid.Api.Models;

namespace SolarMicrogrid.Api.Services;

public class ProsumerService
{
    private static readonly string[] AllowedPhotoContentTypes = { "image/jpeg", "image/png", "image/webp" };
    private const long MaxPhotoSizeBytes = 5 * 1024 * 1024;

    private readonly MongoDBContext _db;
    private readonly IWebHostEnvironment _env;

    public ProsumerService(MongoDbContext db, IWebHostEnvironment env)
    {
        _db = db;
        _env = env;
    }

    // Registers a new prosumer using NIC as the primary key; starts PendingActivation.
    public async Task<UserResponse> RegisterAsync(RegisterProsumerRequest request)
    {
        var exists = await _db.Users.Find(u => u.Nic == request.Nic).AnyAsync();
        if (exists)
        {
            throw new AppException("A prosumer with this NIC is already registered.");
        }

        var user = new User
        {
            UserType = UserType.Prosumer,
            Nic = request.Nic,
            FullName = request.FullName,
            Email = request.Email,
            Phone = request.Phone,
            Address = request.Address,
            PasswordHash = BCrypt.Net.BCrypt.HashPassword(request.Password),
            Status = UserStatus.PendingActivation
        };

        await _db.Users.InsertOneAsync(user);
        return UserResponse.FromModel(user);
    }

    // Retrieves a prosumer profile by NIC.
    public async Task<UserResponse> GetByNicAsync(string nic)
    {
        var user = await FindByNicOrThrow(nic);
        return UserResponse.FromModel(user);
    }
}

```

```

// Updates the prosumer's own editable profile fields.
public async Task<UserResponse> UpdateProfileAsync(string nic, UpdateProsumerRequest request)
{
    var user = await FindByNicOrThrow(nic);
    user.FullName = request.FullName;
    user.Email = request.Email;
    user.Phone = request.Phone;
    user.Address = request.Address;
    user.UpdatedAt = DateTime.UtcNow;

    await _db.Users.ReplaceOneAsync(u => u.Id == user.Id, user);
    return UserResponse.FromModel(user);
}

// Uploads (or replaces) the prosumer's profile picture and returns the updated profile.
public async Task<UserResponse> UploadPhotoAsync(string nic, IFormFile? file)
{
    if (file == null || file.Length == 0)
    {
        throw new ApplicationException("No image file was uploaded.");
    }

    if (!AllowedPhotoContentTypes.Contains(file.ContentType))
    {
        throw new ApplicationException("Only JPEG, PNG, or WEBP images are allowed.");
    }

    if (file.Length > MaxPhotoSizeBytes)
    {
        throw new ApplicationException("Image must be smaller than 5 MB.");
    }

    var user = await FindByNicOrThrow(nic);

    var webRoot = _env.WebRootPath ?? Path.Combine(_env.ContentRootPath, "wwwroot");
    var uploadsDir = Path.Combine(webRoot, "uploads", "prosumers");
    Directory.CreateDirectory(uploadsDir);

    if (!string.IsNullOrEmpty(user.PhotoUrl))
    {
        var previousPath = Path.Combine(webRoot, user.PhotoUrl.TrimStart('/').Replace('/', Path.DirectorySeparatorChar));
        if (File.Exists(previousPath))
        {
            File.Delete(previousPath);
        }
    }

    var extension = file.ContentType switch
    {
        "image/png" => ".png",
        "image/webp" => ".webp",
        _ => ".jpg"
    };
    var fileName = $"{user.Id}-{Guid.NewGuid():N}{extension}";
    var filePath = Path.Combine(uploadsDir, fileName);

    await using (var stream = new FileStream(filePath, FileMode.Create))
    {
        await file.CopyToAsync(stream);
    }

    user.PhotoUrl = $"/uploads/prosumers/{fileName}";
    user.UpdatedAt = DateTime.UtcNow;
    await _db.Users.ReplaceOneAsync(u => u.Id == user.Id, user);
    return UserResponse.FromModel(user);
}

// Prosumer self-service: requests account deactivation immediately.
public async Task<UserResponse> RequestDeactivationAsync(string nic)
{
    var user = await FindByNicOrThrow(nic);
    user.Status = UserStatus.Deactivated;
}

```

```

        user.UpdatedAt = DateTime.UtcNow;
        await _db.Users.ReplaceOneAsync(u => u.Id == user.Id, user);
        return UserResponse.FromModel(user);
    }

    // Fetches a prosumer by NIC or throws a 404 AppException.
    private async Task<User> FindByNicOrThrow(string nic)
    {
        return await _db.Users.Find(u => u.Nic == nic && u.UserType == UserType.Prosumer).FirstOrDefaultAsync();
    }
    ?? throw new AppException("Prosumer not found.", 404);
}

```

api/SolarMicrogrid.Api/Services/ReservationService.cs

```

// -----
// File: ReservationService.cs
// Purpose: Core power-trading reservation workflow – create (within a 7-day
//         window), update/cancel (at least 12 hours' notice), approve, QR
//         verification + completion, and the prosumer/operator dashboard
//         aggregations. All rule enforcement lives here (FAT-service).
// Module owner: Member C (create/update/cancel/dashboards) /
//               Member D (approve/verify-qr/complete)
// -----
using MongoDB.Driver;
using SolarMicrogrid.Api.Common;
using SolarMicrogrid.Api.Data;
using SolarMicrogrid.Api.Dtos;
using SolarMicrogrid.Api.Models;

namespace SolarMicrogrid.Api.Services;

public class ReservationService
{
    private readonly MongoDbContext _db;
    private readonly SlotService _slotService;
    private readonly QrTokenService _qrTokenService;

    private const int MaxAdvanceBookingDays = 7;
    private const int MinNoticeHours = 12;

    public ReservationService(MongoDbContext db, SlotService slotService, QrTokenService qrTokenService)
    {
        _db = db;
        _slotService = slotService;
        _qrTokenService = qrTokenService;
    }

    // Creates a reservation; enforces the 7-day advance-booking window and slot capacity.
    // callerNic is the authenticated prosumer's NIC, or null when a staff member is booking on
    // a prosumer's behalf (in which case request.ProsumerNic must be supplied).
    public async Task<ReservationResponse> CreateAsync(string? callerNic, CreateReservationRequest request)
    {
        var prosumerNic = callerNic ?? request.ProsumerNic
            ?? throw new AppException("ProsumerNic is required when booking on a prosumer's behalf.");

        ValidateWithinBookingWindow(request.ScheduledDateTime);

        var slot = await _slotService.GetByIdOrThrow(request.SlotId);
        await _slotService.ReserveCapacityAsync(slot);

        var reservation = new EnergyReservation
        {
            ReservationCode = GenerateReservationCode(),
            ProsumerNic = prosumerNic,
            StationId = request.StationId,
            SlotId = request.SlotId,
            ScheduledDateTime = request.ScheduledDateTime,
            EnergyAmountKWh = request.EnergyAmountKWh,
            Status = ReservationStatus.Pending
        };
    }
}

```

```

        await _db.Reservations.InsertOneAsync(reservation);
        return ReservationResponse.FromModel(reservation);
    }

    // Updates a reservation's slot/time/amount; requires at least 12 hours' notice.
    // callerNic is null when a staff member (Backoffice/GridOperator) is performing the update.
    public async Task<ReservationResponse> UpdateAsync(string id, string? callerNic, UpdateReservationRequest request)
    {
        var reservation = callerNic is null
            ? await GetByIdOrThrow(id)
            : await GetOwnedByProsumerOrThrow(id, callerNic);
        ValidateMinimumNotice(reservation.ScheduledDateTime);
        ValidateWithinBookingWindow(request.ScheduledDateTime);

        reservation.SlotId = request.SlotId;
        reservation.ScheduledDateTime = request.ScheduledDateTime;
        reservation.EnergyAmountKWh = request.EnergyAmountKWh;
        reservation.UpdatedAt = DateTime.UtcNow;

        await _db.Reservations.ReplaceOneAsync(r => r.Id == id, reservation);
        return ReservationResponse.FromModel(reservation);
    }

    // Cancels a reservation and releases its slot capacity; requires at least 12 hours' notice.
    // callerNic is null when a staff member (Backoffice/GridOperator) is performing the cancellation.
    public async Task<ReservationResponse> CancelAsync(string id, string? callerNic, CancelReservationRequest request)
    {
        var reservation = callerNic is null
            ? await GetByIdOrThrow(id)
            : await GetOwnedByProsumerOrThrow(id, callerNic);
        ValidateMinimumNotice(reservation.ScheduledDateTime);

        reservation.Status = ReservationStatus.Cancelled;
        reservation.CancelledAt = DateTime.UtcNow;
        reservation.CancelReason = request.Reason;
        reservation.UpdatedAt = DateTime.UtcNow;

        await _db.Reservations.ReplaceOneAsync(r => r.Id == id, reservation);

        var slot = await _slotService.GetByIdOrThrow(reservation.SlotId);
        await _slotService.ReleaseCapacityAsync(slot);

        return ReservationResponse.FromModel(reservation);
    }

    // Grid Operator/Backoffice approves a Pending reservation and issues its QR token.
    public async Task<ReservationResponse> ApproveAsync(string id, string approvedBy)
    {
        var reservation = await GetByIdOrThrow(id);
        if (reservation.Status != ReservationStatus.Pending)
        {
            throw new AppException("Only pending reservations can be approved.");
        }

        reservation.Status = ReservationStatus.Approved;
        reservation.ApprovedBy = approvedBy;
        reservation.ApprovedAt = DateTime.UtcNow;
        reservation.QrToken = _qrTokenService.GenerateToken(reservation.Id!, reservation.ReservationCode);
        reservation.UpdatedAt = DateTime.UtcNow;

        await _db.Reservations.ReplaceOneAsync(r => r.Id == id, reservation);
        return ReservationResponse.FromModel(reservation);
    }

    // Grid Operator scans a QR code; verifies its signature and returns the matching reservation.
    public async Task<ReservationResponse> VerifyQrAsync(VerifyQrRequest request)
    {
        var reservationId = _qrTokenService.ValidateAndExtractReservationId(request.QrToken)
            ?? throw new AppException("Invalid or tampered QR code.");
    }

```

```

        var reservation = await GetByIdOrThrow(reservationId);

        if (reservation.QrToken != request.QrToken)
        {
            throw new AppException("QR code does not match the reservation record.");
        }

        if (reservation.Status != ReservationStatus.Approved)
        {
            throw new AppException("Reservation is not in an approved, ready-to-transfer state.");
        }

        return ReservationResponse.FromModel(reservation);
    }

    // Grid Operator finalises the energy transfer after a successful QR verification.
    public async Task<ReservationResponse> CompleteAsync(string id, string completedBy)
    {
        var reservation = await GetByIdOrThrow(id);
        if (reservation.Status != ReservationStatus.Approved)
        {
            throw new AppException("Only approved reservations can be completed.");
        }

        reservation.Status = ReservationStatus.Completed;
        reservation.CompletedBy = completedBy;
        reservation.CompletedAt = DateTime.UtcNow;
        reservation.UpdatedAt = DateTime.UtcNow;

        await _db.Reservations.ReplaceOneAsync(r => r.Id == id, reservation);
        return ReservationResponse.FromModel(reservation);
    }

    // Retrieves a single reservation by id (mobile detail/edit screens).
    public async Task<ReservationResponse> GetByIdAsync(string id)
    {
        var reservation = await GetByIdOrThrow(id);
        return ReservationResponse.FromModel(reservation);
    }

    // Lists a prosumer's full reservation/booking history.
    public async Task<List<ReservationResponse>> GetByProsumerAsync(string prosumerNic)
    {
        var reservations = await _db.Reservations.Find(r => r.ProsumerNic == prosumerNic)
            .SortByDescending(r => r.ScheduledDateTime).ToListAsync();
        return reservations.Select(ReservationResponse.FromModel).ToList();
    }

    // Lists all reservations awaiting Grid Operator approval.
    public async Task<List<ReservationResponse>> GetPendingAsync()
    {
        var reservations = await _db.Reservations.Find(r => r.Status == ReservationStatus.Pending)
            .SortBy(r => r.ScheduledDateTime).ToListAsync();
        return reservations.Select(ReservationResponse.FromModel).ToList();
    }

    // Builds the prosumer's mobile dashboard: active/pending counts + upcoming bookings.
    public async Task<ProsumerDashboardResponse> GetProsumerDashboardAsync(string prosumerNic)
    {
        var reservations = await _db.Reservations.Find(r => r.ProsumerNic == prosumerNic).ToListAsync();

        return new ProsumerDashboardResponse
        {
            ActiveCount = reservations.Count(r => r.Status == ReservationStatus.Approved),
            PendingCount = reservations.Count(r => r.Status == ReservationStatus.Pending),
            UpcomingReservations = reservations
                .Where(r => r.Status is ReservationStatus.Pending or ReservationStatus.Approved)
                .OrderBy(r => r.ScheduledDateTime)
                .Select(ReservationResponse.FromModel)
                .ToList()
        };
    }
}

```

```

// Builds the Grid Operator dashboard: pending queue + count of approved future reservations.
public async Task<OperatorDashboardResponse> GetOperatorDashboardAsync()
{
    var reservations = await _db.Reservations.Find(_ => true).ToListAsync();
    var now = DateTime.UtcNow;

    return new OperatorDashboardResponse
    {
        PendingReservationsCount = reservations.Count(r => r.Status == ReservationStatus.Pending),
        ApprovedFutureReservationsCount = reservations.Count(r => r.Status == ReservationStatus.Approved
&& r.ScheduledDateTime > now),
        PendingReservations = reservations
            .Where(r => r.Status == ReservationStatus.Pending)
            .OrderBy(r => r.ScheduledDateTime)
            .Select(ReservationResponse.FromModel)
            .ToList()
    };
}

// Rejects scheduling more than 7 days out, or in the past.
private static void ValidateWithinBookingWindow(DateTime scheduledDateTime)
{
    var now = DateTime.UtcNow;
    if (scheduledDateTime < now)
    {
        throw new AppException("Reservation time must be in the future.");
    }
    if (scheduledDateTime > now.AddDays(MaxAdvanceBookingDays))
    {
        throw new AppException($"Reservations can only be scheduled within {MaxAdvanceBookingDays}
days.");
    }
}

// Rejects update/cancel when less than 12 hours remain before the scheduled time.
private static void ValidateMinimumNotice(DateTime scheduledDateTime)
{
    if (scheduledDateTime - DateTime.UtcNow < TimeSpan.FromHours(MinNoticeHours))
    {
        throw new AppException($"Updates and cancellations require at least {MinNoticeHours} hours' no-
tice.");
    }
}

// Generates a short human-readable reservation code (e.g. RSV-A1B2C3).
private static string GenerateReservationCode()
{
    return $"RSV-{Guid.NewGuid().ToString("N")[..6].ToUpperInvariant()}";
}

// Fetches a reservation by id or throws a 404 AppException.
private async Task<EnergyReservation> GetByIdOrThrow(string id)
{
    return await _db.Reservations.Find(r => r.Id == id).FirstOrDefaultAsync()
        ?? throw new AppException("Reservation not found.", 404);
}

// Fetches a reservation by id, ensuring it belongs to the requesting prosumer.
private async Task<EnergyReservation> GetOwnedByProsumerOrThrow(string id, string prosumerNic)
{
    var reservation = await GetByIdOrThrow(id);
    if (reservation.ProsumerNic != prosumerNic)
    {
        throw new AppException("You do not own this reservation.", 403);
    }
    return reservation;
}
}

```

api/SolarMicrogrid.Api/Services/SlotService.cs

// -----

```

// File: SlotService.cs
// Purpose: Creates and lists bookable energy time-slots for a station, and
//          adjusts remaining capacity as reservations are made/cancelled.
// Module owner: Member C (Reservations & Booking Workflow)
// -----
using MongoDB.Driver;
using SolarMicrogrid.Api.Common;
using SolarMicrogrid.Api.Data;
using SolarMicrogrid.Api.Dtos;
using SolarMicrogrid.Api.Models;

namespace SolarMicrogrid.Api.Services;

public class SlotService
{
    private readonly MongoDBContext _db;

    public SlotService(MongoDbContext db)
    {
        _db = db;
    }

    // Creates a new bookable slot for a station.
    public async Task<SlotResponse> CreateAsync(string stationId, CreateSlotRequest request)
    {
        var stationExists = await _db.Stations.Find(s => s.Id == stationId).AnyAsync();
        if (!stationExists)
        {
            throw new ApplicationException("Station not found.", 404);
        }

        var slot = new EnergyBookingSlot
        {
            StationId = stationId,
            Date = request.Date,
            StartTime = request.StartTime,
            EndTime = request.EndTime,
            SlotType = request.SlotType,
            CapacityTotal = request.CapacityTotal,
            CapacityAvailable = request.CapacityTotal,
            Status = SlotStatus.Open
        };

        await _db.BookingSlots.InsertOneAsync(slot);
        return SlotResponse.FromModel(slot);
    }

    // Lists all slots for a station.
    public async Task<List<SlotResponse>> GetByStationAsync(string stationId)
    {
        var slots = await _db.BookingSlots.Find(s => s.StationId == stationId).ToListAsync();
        return slots.Select(SlotResponse.FromModel).ToList();
    }

    // Generates one bookable slot per day that matches the station's weekly
    // schedule, for the next `days` days. Safe to call repeatedly – dates that
    // already have a slot are skipped, so this also tops up the rolling window
    // as time passes. Returns how many new slots were created.
    public async Task<int> GenerateFromScheduleAsync(SolarStation station, int days = 14)
    {
        if (station.Schedule.Count == 0)
        {
            return 0;
        }

        var today = DateTime.UtcNow.Date;
        var existingDates = (await _db.BookingSlots.Find(s => s.StationId == station.Id).ToListAsync())
            .Select(s => s.Date.Date)
            .ToHashSet();

        var newSlots = new List<EnergyBookingSlot>();
        for (var offset = 0; offset < days; offset++)
        {
            var date = today.AddDays(offset);

```

```

        if (existingDates.Contains(date))
        {
            continue;
        }

        var scheduleEntry = station.Schedule.FirstOrDefault(e => e.Day == date.DayOfWeek.ToString());
        if (scheduleEntry == null)
        {
            continue;
        }

        newSlots.Add(new EnergyBookingSlot
        {
            StationId = station.Id!,
            Date = date,
            StartTime = scheduleEntry.OpenTime,
            EndTime = scheduleEntry.CloseTime,
            SlotType = SlotType.Charging,
            CapacityTotal = station.TotalBatterySlots,
            CapacityAvailable = station.TotalBatterySlots,
            Status = SlotStatus.Open
        });
    }

    if (newSlots.Count > 0)
    {
        await _db.BookingSlots.InsertManyAsync(newSlots);
    }
    return newSlots.Count;
}

// Fetches a slot by id or throws a 404 AppException.
internal async Task<EnergyBookingSlot> GetByIdOrThrow(string id)
{
    return await _db.BookingSlots.Find(s => s.Id == id).FirstOrDefaultAsync()
        ?? throw new AppException("Booking slot not found.", 404);
}

// Decrements available capacity by one when a reservation is placed; flips to Full at zero.
internal async Task ReserveCapacityAsync(EnergyBookingSlot slot)
{
    if (slot.CapacityAvailable <= 0)
    {
        throw new AppException("This slot is fully booked.");
    }

    slot.CapacityAvailable -= 1;
    slot.Status = slot.CapacityAvailable == 0 ? SlotStatus.Full : SlotStatus.Open;
    slot.UpdatedAt = DateTime.UtcNow;
    await _db.BookingSlots.ReplaceOneAsync(s => s.Id == slot.Id, slot);
}

// Restores one unit of capacity when a reservation is cancelled.
internal async Task ReleaseCapacityAsync(EnergyBookingSlot slot)
{
    slot.CapacityAvailable = Math.Min(slot.CapacityAvailable + 1, slot.CapacityTotal);
    slot.Status = SlotStatus.Open;
    slot.UpdatedAt = DateTime.UtcNow;
    await _db.BookingSlots.ReplaceOneAsync(s => s.Id == slot.Id, slot);
}
}

```

api/SolarMicrogrid.Api/Services/StationService.cs

```

// -----
// File: StationService.cs
// Purpose: CRUD for microgrid nodes (solar stations), the nearby-stations
//          query used by the mobile map, and the deactivation guard that
//          blocks removal while active energy reservations exist.
// Module owner: Member B (Microgrid Nodes & Maps)
// -----
using MongoDB.Driver;
using SolarMicrogrid.Api.Common;

```

```

using SolarMicrogrid.Api.Data;
using SolarMicrogrid.Api.Dtos;
using SolarMicrogrid.Api.Models;

namespace SolarMicrogrid.Api.Services;

public class StationService
{
    private readonly MongoDBContext _db;
    private readonly SlotService _slotService;

    public StationService(MongoDbContext db, SlotService slotService)
    {
        _db = db;
        _slotService = slotService;
    }

    // Creates a new microgrid node/hub and generates its first two weeks of bookable slots.
    public async Task<StationResponse> CreateAsync(CreateStationRequest request, string createdBy)
    {
        var station = new SolarStation
        {
            StationName = request.StationName,
            GpsLocation = new GpsLocation { Lat = request.Lat, Lng = request.Lng },
            CapacityKWh = request.CapacityKWh,
            TotalBatterySlots = request.TotalBatterySlots,
            AvailableBatterySlots = request.TotalBatterySlots,
            Schedule = request.Schedule,
            Status = StationStatus.Active,
            CreatedBy = createdBy
        };

        await _db.Stations.InsertOneAsync(station);
        await _slotService.GenerateFromScheduleAsync(station);
        return StationResponse.FromModel(station);
    }

    // Tops up bookable slots for an existing station from its current schedule
    // (covers stations registered before auto-generation existed, and extends
    // the rolling window as time passes). Returns how many slots were created.
    public async Task<int> GenerateSlotsAsync(string id)
    {
        var station = await GetByIdOrThrow(id);
        return await _slotService.GenerateFromScheduleAsync(station);
    }

    // Lists all stations (active and deactivated) for the Backoffice admin screen.
    public async Task<List<StationResponse>> GetAllAsync()
    {
        var stations = await _db.Stations.Find(_ => true).ToListAsync();
        return stations.Select(StationResponse.FromModel).ToList();
    }

    // Retrieves a single station by id.
    public async Task<StationResponse> GetByIdAsync(string id)
    {
        var station = await GetByIdOrThrow(id);
        return StationResponse.FromModel(station);
    }

    // Updates capacity, battery slot counts and the operating schedule.
    public async Task<StationResponse> UpdateAsync(string id, UpdateStationRequest request)
    {
        var station = await GetByIdOrThrow(id);
        station.StationName = request.StationName;
        station.GpsLocation = new GpsLocation { Lat = request.Lat, Lng = request.Lng };
        station.CapacityKWh = request.CapacityKWh;
        station.TotalBatterySlots = request.TotalBatterySlots;
        station.AvailableBatterySlots = request.AvailableBatterySlots;
        station.Schedule = request.Schedule;
        station.UpdatedAt = DateTime.UtcNow;

        await _db.Stations.ReplaceOneAsync(s => s.Id == id, station);
        return StationResponse.FromModel(station);
    }
}

```

```

}

// Deactivates a station, but only if it has no active (Pending/Approved) reservations.
public async Task<StationResponse> DeactivateAsync(string id)
{
    var station = await GetByIdOrThrow(id);

    var hasActiveReservations = await _db.Reservations.Find(r =>
        r.StationId == id &&
        (r.Status == ReservationStatus.Pending || r.Status == ReservationStatus.Approved)
    ).AnyAsync();

    if (hasActiveReservations)
    {
        throw new ApplicationException("Cannot deactivate a station with active energy reservations.");
    }

    station.Status = StationStatus.Deactivated;
    station.UpdatedAt = DateTime.UtcNow;
    await _db.Stations.ReplaceOneAsync(s => s.Id == id, station);
    return StationResponse.FromModel(station);
}

// Returns active stations within radiusKm of the given point (haversine, in-memory filter).
public async Task<List<StationResponse>> GetNearbyAsync(double lat, double lng, double radiusKm)
{
    var stations = await _db.Stations.Find(s => s.Status == StationStatus.Active).ToListAsync();

    var nearby = stations.Where(s => HaversineKm(lat, lng, s.GpsLocation.Lat, s.GpsLocation.Lng) <= radiusKm);
    return nearby.Select(StationResponse.FromModel).ToList();
}

// Computes great-circle distance in kilometres between two GPS points.
private static double HaversineKm(double lat1, double lon1, double lat2, double lon2)
{
    const double earthRadiusKm = 6371;
    var dLat = DegreesToRadians(lat2 - lat1);
    var dLon = DegreesToRadians(lon2 - lon1);
    var a = Math.Sin(dLat / 2) * Math.Sin(dLat / 2) +
        Math.Cos(DegreesToRadians(lat1)) * Math.Cos(DegreesToRadians(lat2)) *
        Math.Sin(dLon / 2) * Math.Sin(dLon / 2);
    var c = 2 * Math.Atan2(Math.Sqrt(a), Math.Sqrt(1 - a));
    return earthRadiusKm * c;
}

// Converts degrees to radians.
private static double DegreesToRadians(double degrees) => degrees * Math.PI / 180;

// Fetches a station by id or throws a 404 ApplicationException.
private async Task<SolarStation> GetByIdOrThrow(string id)
{
    return await _db.Stations.Find(s => s.Id == id).FirstOrDefaultAsync()
        ?? throw new ApplicationException("Station not found.", 404);
}
}

```

api/SolarMicrogrid.Api/Services/UserService.cs

```

// -----
// File: UserService.cs
// Purpose: Backoffice-only staff account management (create Backoffice /
//          Grid Operator users), plus the pending-activation review queue
//          and activate/deactivate/reactivate workflow shared by staff and
//          prosumer accounts.
// Module owner: Member A (Identity & Access)
// -----
using MongoDB.Driver;
using SolarMicrogrid.Api.Common;
using SolarMicrogrid.Api.Data;
using SolarMicrogrid.Api.Dtos;
using SolarMicrogrid.Api.Models;

```

```

namespace SolarMicrogrid.Api.Services;

public class UserService
{
    private readonly MongoDBContext _db;

    public UserService(MongoDbContext db)
    {
        _db = db;
    }

    // Creates a Backoffice or Grid Operator staff account (active immediately).
    public async Task<UserResponse> CreateStaffUserAsync(CreateStaffUserRequest request, string createdBy)
    {
        if (request.UserType == UserType.Prosumer)
        {
            throw new AppException("Use the prosumer registration endpoint for Prosumer accounts.");
        }

        var exists = await _db.Users.Find(u => u.Username == request.Username).AnyAsync();
        if (exists)
        {
            throw new AppException("Username is already taken.");
        }

        var user = new User
        {
            UserType = request.UserType,
            FullName = request.FullName,
            Email = request.Email,
            Phone = request.Phone,
            Username = request.Username,
            PasswordHash = BCrypt.Net.BCrypt.HashPassword(request.Password),
            Status = UserStatus.Active,
            CreatedBy = createdBy
        };

        await _db.Users.InsertOneAsync(user);
        return UserResponse.FromModel(user);
    }

    // Lists all staff (Backoffice/GridOperator) accounts for the admin screen.
    public async Task<List<UserResponse>> GetStaffUsersAsync()
    {
        var filter = Builders<User>.Filter.In(u => u.UserType, new[] { UserType.Backoffice, UserType.GridOperator });
        var users = await _db.Users.Find(filter).ToListAsync();
        return users.Select(UserResponse.FromModel).ToList();
    }

    // Returns every account (staff or prosumer) awaiting Backoffice activation.
    public async Task<List<UserResponse>> GetPendingActivationsAsync()
    {
        var users = await _db.Users.Find(u => u.Status == UserStatus.PendingActivation).ToListAsync();
        return users.Select(UserResponse.FromModel).ToList();
    }

    // Approves a pending account, moving it to Active.
    public async Task<UserResponse> ApproveAsync(string id)
    {
        var user = await GetByIdOrThrow(id);
        user.Status = UserStatus.Active;
        user.UpdatedAt = DateTime.UtcNow;
        await _db.Users.ReplaceOneAsync(u => u.Id == id, user);
        return UserResponse.FromModel(user);
    }

    // Deactivates an account (Backoffice-initiated or approving a prosumer's own request).
    public async Task<UserResponse> DeactivateAsync(string id)
    {
        var user = await GetByIdOrThrow(id);
        user.Status = UserStatus.Deactivated;
    }
}

```

```

        user.UpdatedAt = DateTime.UtcNow;
        await _db.Users.ReplaceOneAsync(u => u.Id == id, user);
        return UserResponse.FromModel(user);
    }

    // Reactivates a deactivated account – Backoffice officers only (enforced by controller authorization).
    public async Task<UserResponse> ReactivateAsync(string id)
    {
        var user = await GetByIdOrThrow(id);
        if (user.Status != UserStatus.Deactivated)
        {
            throw new AppException("Only deactivated accounts can be reactivated.");
        }
        user.Status = UserStatus.Active;
        user.UpdatedAt = DateTime.UtcNow;
        await _db.Users.ReplaceOneAsync(u => u.Id == id, user);
        return UserResponse.FromModel(user);
    }

    // Fetches a user by id or throws a 404 AppException.
    private async Task<User> GetByIdOrThrow(string id)
    {
        return await _db.Users.Find(u => u.Id == id).FirstOrDefaultAsync()
            ?? throw new AppException("User not found.", 404);
    }
}

```

Appendix B: Web Application Source Code (React)

25 files.

web/src/App.jsx

```

// -----
// File: App.jsx
// Purpose: Application root – auth provider, toast provider, router, and the
//          route table. Authenticated routes render inside AppShell; the login
//          screen renders standalone.
// Module owner: shared infrastructure
// -----
import { BrowserRouter, Routes, Route, Navigate } from "react-router-dom";
import { AuthProvider } from "../context/AuthContext";
import { ToastProvider } from "../components/ui/Toast";
import AppShell from "../components/AppShell";
import ProtectedRoute from "../components/ProtectedRoute";
import LoginPage from "../pages/LoginPage";
import HomePage from "../pages/HomePage";
import UsersPage from "../pages/backoffice/UsersPage";
import PendingActivationsPage from "../pages/backoffice/PendingActivationsPage";
import StationsPage from "../pages/backoffice/StationsPage";
import ReservationsPage from "../pages/backoffice/ReservationsPage";
import OperatorDashboardPage from "../pages/operator/OperatorDashboardPage";

// Wraps a page in the auth guard and the application shell.
function Shell({ roles, children }) {
    return (
        <ProtectedRoute allowedRoles={roles}>
            <AppShell>{children}</AppShell>
        </ProtectedRoute>
    );
}

const BACKOFFICE = ["Backoffice"];
const STAFF = ["Backoffice", "GridOperator"];

// Application root: providers + router.
export default function App() {
    return (
        <AuthProvider>
            <ToastProvider>
                <BrowserRouter>

```

```

    <Routes>
      <Route path="/login" element={<LoginPage />} />
      <Route path="/" element={<Shell roles={STAFF}><HomePage /></Shell>} />
      <Route path="/backoffice/users" element={<Shell roles={BACKOFFICE}><UsersPage /></Shell>} />
      <Route path="/backoffice/pending" element={<Shell roles={BACKOFFICE}><PendingActivationsPage
/></Shell>} />
      <Route path="/backoffice/stations" element={<Shell roles={BACKOFFICE}><StationsPage /></Shell>} />
      <Route path="/backoffice/reservations" element={<Shell roles={BACKOFFICE}><ReservationsPage
/></Shell>} />
      <Route path="/operator" element={<Shell roles={STAFF}><OperatorDashboardPage /></Shell>} />
      <Route path="*" element={<Navigate to="/" replace />} />
    </Routes>
  </BrowserRouter>
</ToastProvider>
</AuthProvider>
);
}

```

web/src/api/client.js

```

// -----
// File: client.js
// Purpose: Shared Axios instance pointed at the Web API. Attaches the JWT
//          from localStorage to every request and redirects to /login on a
//          401 response, so individual pages never handle auth headers.
// Module owner: Member A (Identity & Access) / shared infrastructure
// -----
import axios from "axios";

// Base URL is read from Vite env (falls back to the local dev API port).
const API_BASE_URL = import.meta.env.VITE_API_BASE_URL || "http://localhost:5080/api";

const apiClient = axios.create({ baseURL: API_BASE_URL });

// Attaches the stored JWT (if any) to every outgoing request.
apiClient.interceptors.request.use((config) => {
  const token = localStorage.getItem("token");
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

// Clears the session and bounces to /login whenever the API rejects the token.
apiClient.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      localStorage.removeItem("token");
      localStorage.removeItem("user");
      if (window.location.pathname !== "/login") {
        window.location.href = "/login";
      }
    }
    return Promise.reject(error);
  }
);

export default apiClient;

```

web/src/components/AppShell.jsx

```

// -----
// File: AppShell.jsx
// Purpose: The console's application frame – a dark fixed sidebar with
//          role-aware navigation, a sticky top bar, and a full-width content
//          area. Collapses to an overlay drawer on small screens.
// Module owner: shared infrastructure
// -----
import { useState } from "react";
import { Link, NavLink, useLocation, useNavigate } from "react-router-dom";

```

```

import { useAuth } from "../context/AuthContext";
import {
  GaugeIcon, GridIcon, LogoutIcon, MenuIcon, StationIcon,
  UserCheckIcon, UsersIcon, CalendarIcon, XIcon, ShieldIcon, ChevronLeftIcon,
} from "../ui/Icons";

const NAV = [
  {
    section: "Overview",
    items: [{ to: "/", label: "Dashboard", icon: GridIcon, roles: ["Backoffice", "GridOperator"], end: true }],
  },
  {
    section: "Administration",
    items: [
      { to: "/backoffice/pending", label: "Pending Activations", icon: UserCheckIcon, roles: ["Backoffice"] },
      { to: "/backoffice/users", label: "Staff Users", icon: UsersIcon, roles: ["Backoffice"] },
    ],
  },
  {
    section: "Grid",
    items: [
      { to: "/backoffice/stations", label: "Microgrid Nodes", icon: StationIcon, roles: ["Backoffice"] },
      { to: "/backoffice/reservations", label: "Reservations", icon: CalendarIcon, roles: ["Backoffice"] },
      { to: "/operator", label: "Operator Console", icon: GaugeIcon, roles: ["Backoffice", "GridOperator"] },
    ],
  },
];

// Returns the user's initials for the avatar chip.
function initials(name = "") {
  return name.split(" ").filter(Boolean).slice(0, 2).map((w) => w[0]).join("").toUpperCase();
}

// Sidebar navigation; `onNavigate` lets the mobile drawer close after a click.
function SidebarNav({ role, onNavigate, collapsed }) {
  const sections = NAV.map((s) => ({
    ...s,
    items: s.items.filter((i) => i.roles.includes(role)),
  })).filter((s) => s.items.length > 0);

  return (
    <nav className="flex-1 space-y-7 overflow-y-auto overflow-x-hidden px-4 py-6">
      {sections.map((section) => (
        <div key={section.section}>
          {!collapsed && (
            <p className="mb-2 px-3 text-[10px] font-bold uppercase tracking-[0.14em] text-white/35">
              {section.section}
            </p>
          )}
          <div className="space-y-1">
            {section.items.map((item) => (
              <NavLink
                key={item.to}
                to={item.to}
                end={item.end}
                onClick={onNavigate}
                title={collapsed ? item.label : undefined}
                className={({ isActive }) =>
                  `group relative flex items-center gap-3 rounded-xl px-3 py-2.5 text-sm font-medium transi-
tion-all duration-150 ${
                    collapsed ? "justify-center" : ""
                  } ${
                    isActive
                      ? "bg-white/10 text-white shadow-[inset_0_1px_0_rgb(255_255_255/0.08)]"
                      : "text-white/60 hover:bg-white/5 hover:text-white"
                  }`}
              >
                <>
                  {isActive && (
                    <span className="absolute left-0 top-1/2 h-6 w-1 -translate-y-1/2 rounded-r-full bg-gra-
dient-to-b from-brand-300 to-brand-500" />
                  )}
                  <item.icon className={`h-[18px] w-[18px] shrink-0 ${isActive ? "text-brand-300" : ""}`} />
                </>
              </div>
            )}
          </div>
        </div>
      ))}
    </nav>
  );
}

```

```

        {!collapsed && <span className="truncate">{item.label}</span>}
      </>
    )}
  </NavLink>
  )}
</div>
</div>
  )}
</nav>
);
}

// Brand lockup shown at the top of the sidebar.
function Brand({ collapsed }) {
  return (
    <Link to="/" className={`flex items-center gap-3 px-6 pb-2 pt-6 ${collapsed ? "justify-center px-0" : ""}`>
      <span className="grid h-10 w-10 shrink-0 place-items-center overflow-hidden rounded-xl bg-white shadow-[0_10px_24px_-8px_rgba(16_185_129/0.9)]">
        
      </span>
      {!collapsed && (
        <span className="min-w-0">
          <span className="block truncate font-display text-[15px] font-extrabold leading-tight text-white">
            Solar Microgrid
          </span>
          <span className="block text-[11px] font-medium tracking-wide text-white/45">Trading Console</span>
        </span>
      )}
    </Link>
  );
}

// User card pinned to the bottom of the sidebar with a sign-out action.
function UserCard({ user, onLogout, collapsed }) {
  return (
    <div className="border-t border-white/10 p-4">
      <div className={`flex items-center gap-3 rounded-xl bg-white/5 p-3 ${collapsed ? "flex-col" : ""}`>
        <span className="grid h-9 w-9 shrink-0 place-items-center rounded-lg bg-gradient-to-br from-solar-300 to-solar-500 text-[12px] font-bold text-ink-900">
          {initials(user.fullName)}
        </span>
        {!collapsed && (
          <span className="min-w-0 flex-1">
            <span className="block truncate text-[13px] font-semibold text-white">{user.fullName}</span>
            <span className="block truncate text-[11px] text-white/45">{user.userType}</span>
          </span>
        )}
        <button
          onClick={onLogout}
          title="Sign out"
          className="rounded-lg p-2 text-white/50 transition-colors hover:bg-white/10 hover:text-white"
        >
          <LogoutIcon className="h-[18px] w-[18px]" />
        </button>
      </div>
    </div>
  );
}

// Derives the page title shown in the top bar from the current route.
const TITLES = {
  "/": "Dashboard",
  "/backoffice/pending": "Pending Activations",
  "/backoffice/users": "Staff Users",
  "/backoffice/stations": "Microgrid Nodes",
  "/backoffice/reservations": "Energy Slot Reservations",
  "/operator": "Operator Console",
};

// Application frame: sidebar + top bar + content outlet.
export default function AppShell({ children }) {
  const { user, logout } = useAuth();
  const navigate = useNavigate();
  const location = useLocation();
  const [drawerOpen, setDrawerOpen] = useState(false);

```

```

const [collapsed, setCollapsed] = useState(false);

// Clears the session and returns to the sign-in screen.
function handleLogout() {
  logout();
  navigate("/login");
}

// The grid texture lives on its own layer: both it and the gradient set
// background-image, so they cannot share an element.
function renderSidebar(isCollapsed, onNavigate) {
  return (
    <div className="relative h-full overflow-hidden bg-gradient-to-b from-ink-800 via-ink-900 to-ink-950">
      <div className="bg-grid pointer-events-none absolute inset-0 opacity-50" />
      <div className="relative flex h-full flex-col">
        <Brand collapsed={isCollapsed} />
        <SidebarNav role={user.userType} onNavigate={onNavigate} collapsed={isCollapsed} />
        <UserCard user={user} onLogout={handleLogout} collapsed={isCollapsed} />
      </div>
    </div>
  );
}

return (
  <div className="min-h-screen bg-[#f6f8fb]">
    <div /* Desktop sidebar */>
      <aside
        className={`fixed inset-y-0 left-0 z-40 hidden transition-[width] duration-200 lg:block ${
          collapsed ? "w-[84px]" : "w-[264px]"
        }`}
      >
        {renderSidebar(collapsed, () => {})}
        <button
          onClick={() => setCollapsed((c) => !c)}
          title={collapsed ? "Expand sidebar" : "Collapse sidebar"}
          aria-label={collapsed ? "Expand sidebar" : "Collapse sidebar"}
          className="absolute -right-3.5 top-5 grid h-7 w-7 place-items-center rounded-full border border-
slate-200 bg-white text-slate-500 shadow-md transition-colors hover:text-slate-900"
        >
          <ChevronLeftIcon className={`h-4 w-4 transition-transform duration-200 ${collapsed ? "rotate-180" :
""}`} />
        </button>
      </aside>

      <div /* Mobile drawer */>
        {drawerOpen && (
          <div className="fixed inset-0 z-50 lg:hidden">
            <div className="absolute inset-0 bg-ink-900/60 backdrop-blur-sm onClick={() => setDrawerO-
pen(false)} />
            <aside className="animate-rise absolute inset-y-0 left-0 w-[264px]">
              <button
                onClick={() => setDrawerOpen(false)}
                className="absolute -right-11 top-4 rounded-lg bg-white/10 p-2 text-white backdrop-blur"
                aria-label="Close navigation"
              >
                <XIcon className="h-5 w-5" />
              </button>
              {renderSidebar(false, () => setDrawerOpen(false))}
            </aside>
          </div>
        )}

        <div className={`transition-[padding] duration-200 ${collapsed ? "lg:pl-[84px]" : "lg:pl-[264px]"}`>
          <div /* Top bar */>
            <header className="sticky top-0 z-30 border-b border-slate-200/70 bg-white/85 backdrop-blur-xl">
              <div className="flex h-16 items-center gap-4 px-5 sm:px-8">
                <button
                  onClick={() => setDrawerOpen(true)}
                  className="rounded-lg p-2 text-slate-600 transition-colors hover:bg-slate-100 lg:hidden"
                  aria-label="Open navigation"
                >
                  <MenuIcon className="h-5 w-5" />
                </button>

                <div className="min-w-0 flex-1">

```

```

        <h1 className="truncate font-display text-[19px] font-extrabold tracking-tight text-slate-900">
          {TITLES[location.pathname] || "Console"}
        </h1>
      </div>

      <span className="hidden items-center gap-2 rounded-full bg-brand-50 px-3 py-1.5 text-xs font-semi-
bold text-brand-700 ring-1 ring-inset ring-brand-100 sm:inline-flex">
        <ShieldIcon className="h-3.5 w-3.5" />
        {user.userType}
      </span>
    </div>
  </header>

  <main className="animate-rise px-5 py-7 sm:px-8 sm:py-9">{children}</main>
</div>
</div>
);
}

```

web/src/components/ProtectedRoute.jsx

```

// -----
// File: ProtectedRoute.jsx
// Purpose: Route guard – redirects to /login when no session exists, and shows
//          a styled "not authorised" screen when the signed-in role is not
//          allowed on that route.
// Module owner: Member A (Identity & Access)
// -----
import { Link, Navigate } from "react-router-dom";
import { useAuth } from "../context/AuthContext";
import { ShieldIcon } from "../ui/Icons";
import Button from "../ui/Button";

// Renders children only for an authenticated, role-permitted user.
export default function ProtectedRoute({ children, allowedRoles }) {
  const { user } = useAuth();

  if (!user) {
    return <Navigate to="/login" replace />;
  }

  if (allowedRoles && !allowedRoles.includes(user.userType)) {
    return (
      <div className="flex min-h-screen items-center justify-center bg-[#f6f8fb] px-6">
        <div className="w-full max-w-md rounded-2xl border border-slate-200 bg-white p-8 text-center shadow-
[0_1px_2px_rgb(15_23_42/0.04),0_18px_40px_-18px_rgb(15_23_42/0.25)]">
          <span className="mx-auto mb-5 grid h-14 w-14 place-items-center rounded-2xl bg-rose-50 text-rose-600
ring-1 ring-inset ring-rose-100">
            <ShieldIcon className="h-7 w-7" />
          </span>
          <h2 className="font-display text-xl font-bold text-slate-900">Not authorised</h2>
          <p className="mt-2 text-sm leading-relaxed text-slate-500">
            Your {user.userType} account doesn't have access to this area of the console.
          </p>
          <Link to="/" className="mt-6 inline-block">
            <Button variant="subtle">Back to dashboard</Button>
          </Link>
        </div>
      </div>
    );
  }

  return children;
}

```

web/src/components/ui/Badge.jsx

```

// -----
// File: Badge.jsx
// Purpose: Status pills. StatusBadge maps the API's status strings
//          (Pending/Approved/Active/Deactivated/...) onto a consistent colour

```

```
// language so the same state always looks the same everywhere.
// Module owner: shared infrastructure
// -----

const TONES = {
  emerald: "bg-brand-50 text-brand-700 ring-brand-200",
  amber: "bg-solar-50 text-solar-600 ring-solar-200",
  slate: "bg-slate-100 text-slate-600 ring-slate-200",
  rose: "bg-rose-50 text-rose-700 ring-rose-200",
  sky: "bg-sky-50 text-sky-700 ring-sky-200",
};

// Generic coloured pill.
export function Badge({ tone = "slate", children, className = "" }) {
  return (
    <span
      className={`inline-flex items-center gap-1.5 rounded-full px-2.5 py-1 text-xs font-semibold ring-1 ring-
inset ${TONES[tone]} ${className}`}
    >
      {children}
    </span>
  );
}

const STATUS_TONES = {
  Active: "emerald",
  Approved: "emerald",
  Completed: "sky",
  Open: "emerald",
  Pending: "amber",
  PendingActivation: "amber",
  Full: "amber",
  Cancelled: "rose",
  Deactivated: "rose",
  Closed: "slate",
};

const STATUS_LABELS = {
  PendingActivation: "Pending",
};

// Status pill that colours itself from the API status string.
export function StatusBadge({ status }) {
  const tone = STATUS_TONES[status] || "slate";
  return (
    <Badge tone={tone}>
      <span className="h-1.5 w-1.5 rounded-full bg-current opacity-70" />
      {STATUS_LABELS[status] || status}
    </Badge>
  );
}

export default Badge;
```

web/src/components/ui/Button.jsx

```
// -----
// File: Button.jsx
// Purpose: Single button primitive with brand/solar/ghost/danger variants so
//          every action in the console shares one interaction language.
// Module owner: shared infrastructure
// -----

const VARIANTS = {
  primary:
    "bg-gradient-to-b from-brand-500 to-brand-600 text-white shadow-[0_10px_24px_-10px_rgb(5_150_105/0.9)]
hover:from-brand-400 hover:to-brand-600 active:translate-y-px",
  solar:
    "bg-gradient-to-b from-solar-400 to-solar-500 text-ink-900 shadow-[0_10px_24px_-10px_rgb(217_119_6/0.8)]
hover:from-solar-300 hover:to-solar-500 active:translate-y-px",
  subtle:
    "bg-white text-slate-700 ring-1 ring-slate-200 hover:bg-slate-50 hover:ring-slate-300",
  ghost: "text-slate-600 hover:bg-slate-100 hover:text-slate-900",
};
```

```

    danger:
      "bg-gradient-to-b from-rose-500 to-rose-600 text-white shadow-[0_10px_24px_-10px_rgb(225_29_72/0.8)]
  hover:from-rose-400 hover:to-rose-600 active:translate-y-px",
  dangerGhost: "text-rose-600 hover:bg-rose-50",
};

const SIZES = {
  sm: "h-9 px-3.5 text-[13px] gap-1.5",
  md: "h-11 px-5 text-sm gap-2",
  lg: "h-12 px-6 text-[15px] gap-2",
};

// Renders a styled button; `icon` is an optional leading icon component.
export default function Button({
  variant = "primary",
  size = "md",
  icon: LeadingIcon,
  className = "",
  children,
  ...props
}) {
  return (
    <button
      className={`inline-flex items-center justify-center rounded-xl font-semibold tracking-tight transition-
all duration-150 focus:outline-none focus-visible:ring-2 focus-visible:ring-brand-500/60 focus-visible:ring-
offset-2 disabled:cursor-not-allowed disabled:opacity-55 ${VARIANTS[variant]} ${SIZES[size]} ${className}`}
      {...props}
    >
      {LeadingIcon && <LeadingIcon className="h-4 w-4" />}
      {children}
    </button>
  );
}

```

web/src/components/ui/Card.jsx

```

// -----
// File: Card.jsx
// Purpose: Surface primitives – the base Card container and a CardHeader with
//         optional eyebrow/title/description/actions used by every page panel.
// Module owner: shared infrastructure
// -----

// Base elevated surface used to group related content.
export function Card({ className = "", children, ...props }) {
  return (
    <div
      className={`rounded-2xl border border-slate-200/80 bg-white shadow-
[0_1px_2px_rgb(15_23_42/0.04),0_12px_32px_-16px_rgb(15_23_42/0.16)] ${className}`}
      {...props}
    >
      {children}
    </div>
  );
}

// Title block for a card, with an optional right-aligned actions slot.
export function CardHeader({ title, description, icon: LeadIcon, actions }) {
  return (
    <div className="flex flex-wrap items-start justify-between gap-4 border-b border-slate-100 px-6 py-5">
      <div className="flex items-start gap-3.5">
        {LeadIcon && (
          <span className="grid h-10 w-10 shrink-0 place-items-center rounded-xl bg-brand-50 text-brand-700
ring-1 ring-brand-100">
            <LeadIcon className="h-5 w-5" />
          </span>
        )}
      </div>
      <div>
        <h2 className="text-[17px] font-bold text-slate-900">{title}</h2>
        {description && <p className="mt-0.5 text-sm text-slate-500">{description}</p>}
      </div>
      <div>
        {actions && <div className="flex items-center gap-2">{actions}</div>}
      </div>
    </div>
  );
}

```

```

    });
  }

  // Section heading used between cards on a page.
  export function SectionTitle({ children, hint }) {
    return (
      <div className="mb-4 flex items-baseline justify-between gap-4">
        <h2 className="text-lg font-bold tracking-tight text-slate-900">{children}</h2>
        {hint && <span className="text-sm text-slate-500">{hint}</span>}
      </div>
    );
  }

  export default Card;

```

web/src/components/ui/DataTable.jsx

```

// -----
// File: DataTable.jsx
// Purpose: Shared table shell with sticky-styled header, row hover, loading
//          skeletons and an empty state, so every listing screen matches.
// Module owner: shared infrastructure
// -----
import EmptyState from "../EmptyState";

// Renders `columns` against `rows`; each column is { key, header, render, className }.
export default function DataTable({ columns, rows, loading, empty, rowKey = (r) => r.id }) {
  return (
    <div className="overflow-x-auto">
      <table className="w-full min-w-[640px] border-collapse text-left">
        <thead>
          <tr className="border-b border-slate-100 bg-slate-50/60">
            {columns.map((c) => (
              <th
                key={c.key}
                className={`whitespace-nowrap px-6 py-3.5 text-[11px] font-bold uppercase tracking-wider text-slate-500 ${c.className || ""}`}
              >
                {c.header}
              </th>
            ))}
          </tr>
        </thead>
        <tbody>
          {loading &&
            Array.from({ length: 3 }).map((_, i) => (
              <tr key={i} className="border-b border-slate-50">
                {columns.map((c) => (
                  <td key={c.key} className="px-6 py-4">
                    <div className="h-4 w-24 animate-pulse rounded bg-slate-100" />
                  </td>
                ))}
              </tr>
            ))}
          {!loading &&
            rows.map((row) => (
              <tr
                key={rowKey(row)}
                className="border-b border-slate-50 transition-colors last:border-0 hover:bg-brand-50/40"
              >
                {columns.map((c) => (
                  <td key={c.key} className={`px-6 py-4 text-sm text-slate-700 ${c.className || ""}`} >
                    {c.render ? c.render(row) : row[c.key]}
                  </td>
                ))}
              </tr>
            ))}
        </tbody>
      </table>

      {!loading && rows.length === 0 && empty && (
        <EmptyState {...empty} />
      )}
    </div>
  );
}

```

```

    })
  </div>
);
}

```

web/src/components/ui/Drawer.jsx

```

// -----
// File: Drawer.jsx
// Purpose: Slide-in side panel used for create/edit forms, so a long form gets
//          room to breathe instead of being cramped into a centred dialog.
//          Same open/onClose/Escape/backdrop-click behaviour as Modal.
// Module owner: shared infrastructure
// -----
import { useEffect } from "react";
import { createPortal } from "react-dom";
import { XIcon } from "../Icons";

const SIZES = { md: "max-w-md", lg: "max-w-xl", xl: "max-w-2xl" };

// Renders a right-anchored sliding panel over a blurred backdrop when `open` is true.
export default function Drawer({ open, onClose, title, description, icon: LeadIcon, tone = "brand", size =
"md", children, footer }) {
  useEffect(() => {
    if (!open) return undefined;
    const onKey = (e) => e.key === "Escape" && onClose?();
    document.addEventListener("keydown", onKey);
    document.body.style.overflow = "hidden";
    return () => {
      document.removeEventListener("keydown", onKey);
      document.body.style.overflow = "";
    };
  }, [open, onClose]);

  if (!open) return null;

  const toneClasses =
    tone === "danger"
      ? "bg-rose-50 text-rose-600 ring-rose-100"
      : "bg-brand-50 text-brand-700 ring-brand-100";

  return createPortal(
    <div className="fixed inset-0 z-50 flex justify-end">
      <div
        className="animate-fade absolute inset-0 bg-ink-900/55 backdrop-blur-sm"
        onClick={onClose}
        aria-hidden="true"
      />
      <div
        role="dialog"
        aria-modal="true"
        className={`animate-slide-in relative flex h-full w-full ${SIZES[size]} flex-col bg-white shadow-[-
32px_0_80px_-20px_rgb(15_23_42/0.45)]`}
      >
        <div className="flex shrink-0 items-start gap-4 border-b border-slate-100 px-6 pb-5 pt-6">
          {LeadIcon && (
            <span className={`grid h-11 w-11 shrink-0 place-items-center rounded-xl ring-1 ring-inset ${tone-
Classes}`}>
              <LeadIcon className="h-5 w-5" />
            </span>
          )}
          <div className="min-w-0 flex-1">
            <h3 className="text-lg font-bold text-slate-900">{title}</h3>
            {description && <p className="mt-1 text-sm text-slate-500">{description}</p>}
          </div>
          <button
            onClick={onClose}
            className="rounded-lg p-1.5 text-slate-400 transition-colors hover:bg-slate-100 hover:text-slate-
600"
            aria-label="Close panel"
          >
            <XIcon className="h-5 w-5" />
          </button>
        </div>
      </div>
    </div>
  );

```

```

    </div>

    {children && <div className="flex-1 overflow-y-auto px-6 py-5">{children}</div>}

    {footer && (
      <div className="flex shrink-0 items-center justify-end gap-2.5 border-t border-slate-100 bg-slate-
50/70 px-6 py-4">
        {footer}
      </div>
    )}
  </div>
</div>,
document.body
);
}

```

web/src/components/ui/EmptyState.jsx

```

// -----
// File: EmptyState.jsx
// Purpose: Friendly placeholder shown when a list has no rows yet, instead of
//          a bare "no data" line.
// Module owner: shared infrastructure
// -----

// Renders a centred icon, title, description and optional action.
export default function EmptyState({ icon: LeadIcon, title, description, action }) {
  return (
    <div className="flex flex-col items-center justify-center px-6 py-14 text-center">
      {LeadIcon && (
        <span className="mb-4 grid h-14 w-14 place-items-center rounded-2xl bg-slate-50 text-slate-300 ring-1
ring-slate-100">
          <LeadIcon className="h-7 w-7" />
        </span>
      )}
      <p className="text-[15px] font-semibold text-slate-700">{title}</p>
      {description && <p className="mt-1 max-w-sm text-sm text-slate-500">{description}</p>}
      {action && <div className="mt-5">{action}</div>}
    </div>
  );
}

```

web/src/components/ui/Field.jsx

```

// -----
// File: Field.jsx
// Purpose: Form primitives – labelled Input and Select wrappers that share one
//          focus ring, sizing and error treatment across every form.
// Module owner: shared infrastructure
// -----

const CONTROL =
  "h-11 w-full rounded-xl border border-slate-200 bg-white px-3.5 text-sm text-slate-900 shadow-sm transition-
colors placeholder:text-slate-400 focus:border-brand-500 focus:outline-none focus:ring-4 focus:ring-brand-
500/10 disabled:bg-slate-50 disabled:text-slate-400";

// Label + control wrapper providing consistent vertical rhythm.
export function Field({ label, hint, required, children, className = "" }) {
  return (
    <label className={`block ${className}`}>
      {label && (
        <span className="mb-1.5 flex items-center gap-1 text-[13px] font-semibold text-slate-700">
          {label}
          {required && <span className="text-rose-500">*</span>}
        </span>
      )}
      {children}
      {hint && <span className="mt-1 block text-xs text-slate-400">{hint}</span>}
    </label>
  );
}

```

```
// Text/number/date input styled to the design system.
export function Input({ className = "", ...props }) {
  return <input className={`${CONTROL} ${className}`} {...props} />;
}

// Native select with a custom chevron so it matches Input height/rounding.
export function Select({ className = "", children, ...props }) {
  return (
    <div className="relative">
      <select
        className={`${CONTROL} appearance-none pr-10 ${className}`}
        {...props}
      >
        {children}
      </select>
      <svg
        viewBox="0 0 24 24"
        fill="none"
        stroke="currentColor"
        strokeWidth="2"
        strokeLinecap="round"
        strokeLinejoin="round"
        className="pointer-events-none absolute right-3.5 top-1/2 h-4 w-4 -translate-y-1/2 text-slate-400"
        aria-hidden="true"
      >
        <path d="m6 9 6 6 6-6" />
      </svg>
    </div>
  );
}

export default Field;
```

web/src/components/ui/Icons.jsx

```
// -----
// File: Icons.jsx
// Purpose: Inline stroke-based SVG icon set used across the console. Kept
//          inline (rather than an icon package) so the bundle stays small and
//          every icon inherits currentColor from its parent.
// Module owner: shared infrastructure
// -----

// Wraps a 24x24 stroke icon with shared defaults so call sites only pass paths.
function Icon({ children, className = "h-5 w-5", strokeWidth = 1.75, ...rest }) {
  return (
    <svg
      viewBox="0 0 24 24"
      fill="none"
      stroke="currentColor"
      strokeWidth={strokeWidth}
      strokeLinecap="round"
      strokeLinejoin="round"
      className={className}
      aria-hidden="true"
      {...rest}
    >
      {children}
    </svg>
  );
}

export const BoltIcon = (p) => (
  <Icon {...p}><path d="M13 2 4.5 13.5H11l-1 8.5 8.5-11.5H12l11-8.5Z" /></Icon>
);
export const GridIcon = (p) => (
  <Icon {...p}><rect x="3" y="3" width="7" height="7" rx="2" /><rect x="14" y="3" width="7" height="7" rx="2" /><rect x="3" y="14" width="7" height="7" rx="2" /><rect x="14" y="14" width="7" height="7" rx="2" /></Icon>
);
export const UsersIcon = (p) => (
  <Icon {...p}><path d="M16 21v-2a4 4 0 0 0-4-4H6a4 4 0 0 0-4 4v2" /><circle cx="9" cy="7" r="4" /><path d="M22 21v-2a4 4 0 0 0-3-3.87M16 3.13a4 4 0 0 1 0 7.75" /></Icon>
);
```

```

);
export const UserCheckIcon = (p) => (
  <Icon {...p}><path d="M16 21v-2a4 4 0 0 0-4H6a4 4 0 0 0-4 4v2" /><circle cx="9" cy="7" r="4" /><path
d="m16 11 2 2 4-4" /></Icon>
);
export const StationIcon = (p) => (
  <Icon {...p}><path d="M3 21h18" /><path d="M5 21V8l7-5 7 5v13" /><path d="M9 21v-6h6v6" /><path d="M12 3v3"
/></Icon>
);
export const CalendarIcon = (p) => (
  <Icon {...p}><rect x="3" y="4" width="18" height="18" rx="3" /><path d="M16 2v4M8 2v4M3 10h18" /></Icon>
);
export const GaugeIcon = (p) => (
  <Icon {...p}><path d="M12 14 16 10" /><path d="M3.5 18a9 9 0 1 1 17 0" /><circle cx="12" cy="18" r="1.5"
/></Icon>
);
export const LogoutIcon = (p) => (
  <Icon {...p}><path d="M9 21H5a2 2 0 0 1-2-2V5a2 2 0 0 1 2-2h4" /><path d="m16 17 5-5-5-5" /><path d="M21
12H9" /></Icon>
);
export const BatteryIcon = (p) => (
  <Icon {...p}><rect x="2" y="7" width="16" height="10" rx="2" /><path d="M22 11v2" /><path d="M6 11v2M10
11v2" /></Icon>
);
export const ClockIcon = (p) => (
  <Icon {...p}><circle cx="12" cy="12" r="9" /><path d="M12 7v5l3 2" /></Icon>
);
export const CheckIcon = (p) => (
  <Icon {...p}><path d="m5 12 5 5L20 7" /></Icon>
);
export const XIcon = (p) => (
  <Icon {...p}><path d="M18 6 6 18M6 6l12 12" /></Icon>
);
export const PlusIcon = (p) => (
  <Icon {...p}><path d="M12 5v14M5 12h14" /></Icon>
);
export const SearchIcon = (p) => (
  <Icon {...p}><circle cx="11" cy="11" r="7" /><path d="m20 20-3.5-3.5" /></Icon>
);
export const ArrowRightIcon = (p) => (
  <Icon {...p}><path d="M5 12h14" /><path d="m13 6 6 6-6 6" /></Icon>
);
export const SunIcon = (p) => (
  <Icon {...p}><circle cx="12" cy="12" r="4" /><path d="M12 2v2M12 20v2M4.9 4.9l11.4 1.4M17.7 17.7l11.4 1.4M2
12h2M20 12h2M4.9 19.1l11.4-1.4M17.7 6.3l11.4-1.4" /></Icon>
);
export const TrendIcon = (p) => (
  <Icon {...p}><path d="m3 17 6-6 4 4 8-8" /><path d="M15 7h6v6" /></Icon>
);
export const MapPinIcon = (p) => (
  <Icon {...p}><path d="M20 10c0 6-8 12-8 12s-8-6-8-12a8 8 0 0 1 16 0Z" /><circle cx="12" cy="10" r="3"
/></Icon>
);
export const AlertIcon = (p) => (
  <Icon {...p}><path d="M12 9v4M12 17h.01" /><path d="M10.3 3.9 1.8 18a2 2 0 0 0 1.7 3h17a2 2 0 0 0 1.7-3L13.7
3.9a2 2 0 0 0-3.4 0Z" /></Icon>
);
export const MenuIcon = (p) => (
  <Icon {...p}><path d="M4 6h16M4 12h16M4 18h16" /></Icon>
);
export const ShieldIcon = (p) => (
  <Icon {...p}><path d="M12 22s8-4 8-10V5l-8-3-8 3v7c0 6 8 10 8 10Z" /></Icon>
);
export const ChevronLeftIcon = (p) => (
  <Icon {...p}><path d="m15 18-6-6 6-6" /></Icon>
);

export default Icon;

```

web/src/components/ui/Modal.jsx

```

// -----
// File: Modal.jsx
// Purpose: Accessible dialog used for confirmations and short forms, replacing
// the browser's window.prompt/alert so the console keeps one visual

```

```

// language. Closes on Escape and on backdrop click.
// Module owner: shared infrastructure
// -----
import { useEffect } from "react";
import { createPortal } from "react-dom";
import { XIcon } from "../Icons";

const SIZES = { md: "max-w-lg", lg: "max-w-2xl", xl: "max-w-3xl" };

// Renders a centred dialog over a blurred backdrop when `open` is true.
export default function Modal({ open, onClose, title, description, icon: LeadIcon, tone = "brand", size = "md", children, footer }) {
  useEffect(() => {
    if (!open) return undefined;
    const onKey = (e) => e.key === "Escape" && onClose?();
    document.addEventListener("keydown", onKey);
    document.body.style.overflow = "hidden";
    return () => {
      document.removeEventListener("keydown", onKey);
      document.body.style.overflow = "";
    };
  }, [open, onClose]);

  if (!open) return null;

  const toneClasses =
    tone === "danger"
      ? "bg-rose-50 text-rose-600 ring-rose-100"
      : "bg-brand-50 text-brand-700 ring-brand-100";

  return createPortal(
    <div className="fixed inset-0 z-50 flex items-end justify-center p-4 sm:items-center">
      <div
        className="absolute inset-0 bg-ink-900/55 backdrop-blur-sm"
        onClick={onClose}
        aria-hidden="true"
      />
      <div
        role="dialog"
        aria-modal="true"
        className={`animate-rise relative max-h-[92vh] w-full ${SIZES[size]} overflow-y-auto rounded-2xl bg-white shadow-[0_32px_80px_-20px_rgb(15_23_42/0.45)]`}
      >
        <div className="flex items-start gap-4 px-6 pb-4 pt-6">
          {LeadIcon && (
            <span className={`grid h-11 w-11 shrink-0 place-items-center rounded-xl ring-1 ring-inset ${toneClasses}`}>
              <LeadIcon className="h-5 w-5" />
            </span>
          )}
          <div className="min-w-0 flex-1">
            <h3 className="text-lg font-bold text-slate-900">{title}</h3>
            {description && <p className="mt-1 text-sm text-slate-500">{description}</p>}
          </div>
          <button
            onClick={onClose}
            className="rounded-lg p-1.5 text-slate-400 transition-colors hover:bg-slate-100 hover:text-slate-600"
            aria-label="Close dialog"
          >
            <XIcon className="h-5 w-5" />
          </button>
        </div>

        {children && <div className="px-6 pb-2">{children}</div>}

        {footer && (
          <div className="mt-4 flex items-center justify-end gap-2.5 border-t border-slate-100 bg-slate-50/70 px-6 py-4">
            {footer}
          </div>
        )}
      </div>
    </div>,
    document.body
  );

```

```

    });
}

```

web/src/components/ui/StatCard.jsx

```

// -----
// File: StatCard.jsx
// Purpose: KPI tile used on the dashboards – large value, label, optional
//          trailing hint and a tinted icon badge.
// Module owner: shared infrastructure
// -----

const TONES = {
  emerald: { ring: "ring-brand-100", bg: "bg-brand-50", fg: "text-brand-700", bar: "from-brand-400 to-brand-600" },
  amber: { ring: "ring-solar-200", bg: "bg-solar-50", fg: "text-solar-600", bar: "from-solar-300 to-solar-500" },
  sky: { ring: "ring-sky-100", bg: "bg-sky-50", fg: "text-sky-700", bar: "from-sky-400 to-sky-600" },
  slate: { ring: "ring-slate-200", bg: "bg-slate-100", fg: "text-slate-700", bar: "from-slate-400 to-slate-600" },
};

// Renders one KPI tile. `value` is displayed large; `hint` sits under it.
export default function StatCard({ label, value, hint, icon: LeadIcon, tone = "emerald", loading = false }) {
  const t = TONES[tone];
  return (
    <div className="group relative overflow-hidden rounded-2xl border border-slate-200/80 bg-white p-5 shadow-[0_1px_2px_rgb(15_23_42/0.04),0_12px_32px_-16px_rgb(15_23_42/0.16)] transition-all duration-200 hover:-translate-y-0.5 hover:shadow-[0_2px_4px_rgb(15_23_42/0.05),0_22px_44px_-20px_rgb(15_23_42/0.28)]">
      <span className={`absolute inset-x-0 top-0 h-1 bg-gradient-to-r ${t.bar}`} />
      <div className="flex items-start justify-between gap-4">
        <div className="min-w-0">
          <p className="text-[13px] font-medium text-slate-500">{label}</p>
          {loading ? (
            <div className="mt-2 h-9 w-16 animate-pulse rounded-lg bg-slate-100" />
          ) : (
            <p className="mt-1 font-display text-4xl font-extrabold tracking-tight text-slate-900">{value}</p>
          )}
          {hint && <p className="mt-1.5 truncate text-xs text-slate-400">{hint}</p>}
        </div>
        {LeadIcon && (
          <span className={`grid h-11 w-11 shrink-0 place-items-center rounded-xl ring-1 ring-inset ${t.bg} ${t.fg} ${t.ring}`}>
            <LeadIcon className="h-5 w-5" />
          </span>
        )}
      </div>
    </div>
  );
}

```

web/src/components/ui/Toast.jsx

```

// -----
// File: Toast.jsx
// Purpose: Lightweight toast system (provider + useToast hook) so success and
//          error feedback appears consistently instead of via alert().
// Module owner: shared infrastructure
// -----

import { createContext, useCallback, useContext, useState } from "react";
import { AlertIcon, CheckIcon, XIcon } from "../Icons";

const ToastContext = createContext(null);

const TONES = {
  success: { ring: "ring-brand-200", bg: "bg-brand-50", fg: "text-brand-700", Icon: CheckIcon },
  error: { ring: "ring-rose-200", bg: "bg-rose-50", fg: "text-rose-700", Icon: AlertIcon },
};

// Provides toast state to the tree and renders the stack in the corner.
export function ToastProvider({ children }) {

```

```

const [toasts, setToasts] = useState([]);

// Adds a toast and schedules its removal.
const push = useCallback((message, tone = "success") => {
  const id = Date.now() + Math.random();
  setToasts((prev) => [...prev, { id, message, tone }]);
  setTimeout(() => setToasts((prev) => prev.filter((t) => t.id !== id)), 4500);
}, []);

const api = {
  success: useCallback((m) => push(m, "success"), [push]),
  error: useCallback((m) => push(m, "error"), [push]),
};

return (
  <ToastContext.Provider value={api}>
    {children}
    <div className="pointer-events-none fixed bottom-5 right-5 z-[60] flex w-full max-w-sm flex-col gap-
2.5">
      {toasts.map((t) => {
        const tone = TONES[t.tone];
        return (
          <div
            key={t.id}
            className="animate-rise pointer-events-auto flex items-start gap-3 rounded-xl border border-
slate-200 bg-white p-3.5 shadow-[0_18px_40px_-14px_rgb(15_23_42/0.35)]"
            >
            <span className={`grid h-8 w-8 shrink-0 place-items-center rounded-lg ring-1 ring-inset
${tone.bg} ${tone.fg} ${tone.ring}`}>
              <tone.Icon className="h-4 w-4" />
            </span>
            <p className="flex-1 pt-1 text-sm font-medium text-slate-700">{t.message}</p>
            <button
              onClick={() => setToasts((prev) => prev.filter((x) => x.id !== t.id))}
              className="rounded p-1 text-slate-400 transition-colors hover:bg-slate-100 hover:text-slate-
600"
              aria-label="Dismiss notification"
            >
              <XIcon className="h-4 w-4" />
            </button>
          </div>
        );
      })}
    </div>
  </ToastContext.Provider>
);
}

// Hook returning { success, error } notification helpers.
export function useToast() {
  return useContext(ToastContext);
}

```

web/src/context/AuthContext.jsx

```

// -----
// File: AuthContext.jsx
// Purpose: Global auth state (current user + token), backed by localStorage
//          so a page refresh keeps the session. Exposes login()/logout()
//          used by every protected route and role guard.
// Module owner: Member A (Identity & Access) / shared infrastructure
// -----
import { createContext, useContext, useEffect, useState } from "react";
import apiClient from "../api/client";

const AuthContext = createContext(null);

// Wraps the app, loading any previously-saved session from localStorage on mount.
export function AuthProvider({ children }) {
  const [user, setUser] = useState(() => {
    const stored = localStorage.getItem("user");
    return stored ? JSON.parse(stored) : null;
  });

```

```

useEffect(() => {
  if (user) {
    localStorage.setItem("user", JSON.stringify(user));
  } else {
    localStorage.removeItem("user");
  }
}, [user]);

// Calls the login endpoint, stores the JWT + user profile on success.
async function login(identifier, password) {
  const { data } = await apiClient.post("/auth/login", { identifier, password });
  localStorage.setItem("token", data.token);
  const profile = { id: data.userId, userType: data.userType, fullName: data.fullName, nic: data.nic };
  setUser(profile);
  return profile;
}

// Clears the session and forgets the JWT.
function logout() {
  localStorage.removeItem("token");
  setUser(null);
}

return (
  <AuthContext.Provider value={{ user, login, logout }}>
    {children}
  </AuthContext.Provider>
);
}

// Convenience hook for consuming the auth context.
export function useAuth() {
  return useContext(AuthContext);
}

```

web/src/index.css

```

@import "tailwindcss";

/* -----
Design tokens – "Solar Nature" system.
Deep forest/emerald greens (grid, sustainability) paired with solar amber
(energy, sunlight), on a cool slate neutral base.
----- */
@theme {
  --color-brand-50: #ecfdf5;
  --color-brand-100: #d1fae5;
  --color-brand-200: #a7f3d0;
  --color-brand-300: #6ee7b7;
  --color-brand-400: #34d399;
  --color-brand-500: #10b981;
  --color-brand-600: #059669;
  --color-brand-700: #047857;
  --color-brand-800: #065f46;
  --color-brand-900: #064e3b;
  --color-brand-950: #022c22;

  --color-solar-50: #ffffeb;
  --color-solar-100: #fef3c7;
  --color-solar-200: #fde68a;
  --color-solar-300: #fcd34d;
  --color-solar-400: #fbbf24;
  --color-solar-500: #f59e0b;
  --color-solar-600: #d97706;

  --color-ink-700: #1e293b;
  --color-ink-800: #101a2c;
  --color-ink-900: #0a1220;
  --color-ink-950: #060b14;

  --font-display: "Plus Jakarta Sans", ui-sans-serif, system-ui, sans-serif;

```

```

--font-sans: "Inter", ui-sans-serif, system-ui, sans-serif;

--shadow-soft: 0 1px 2px rgb(15 23 42 / 0.04), 0 8px 24px -8px rgb(15 23 42 / 0.10);
--shadow-lift: 0 2px 4px rgb(15 23 42 / 0.04), 0 18px 40px -12px rgb(15 23 42 / 0.18);
--shadow-glow: 0 18px 40px -14px rgb(5 150 105 / 0.55);
}

html {
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
}

body {
  margin: 0;
  font-family: var(--font-sans);
  background: #f6f8fb;
  color: #0f172a;
}

h1, h2, h3, h4 {
  font-family: var(--font-display);
  letter-spacing: -0.02em;
}

/* Layered mesh background used by the auth screen and page headers. */
.bg-aurora {
  background-image:
    radial-gradient(60rem 40rem at 10% -10%, rgb(16 185 129 / 0.25), transparent 60%),
    radial-gradient(45rem 30rem at 95% 0%, rgb(251 191 36 / 0.20), transparent 55%),
    radial-gradient(40rem 30rem at 50% 110%, rgb(4 120 87 / 0.35), transparent 60%);
}

.bg-grid {
  background-image:
    linear-gradient(to right, rgb(255 255 255 / 0.06) 1px, transparent 1px),
    linear-gradient(to bottom, rgb(255 255 255 / 0.06) 1px, transparent 1px);
  background-size: 44px 44px;
}

/* Subtle entrance animation for dashboard content. */
@keyframes rise {
  from { opacity: 0; transform: translateY(10px); }
  to { opacity: 1; transform: translateY(0); }
}
/* No fill-mode: a lingering `transform` after the animation ends (from "both"/
"forwards") makes this element a CSS containing block forever, which breaks
position:fixed descendants and can stop native <select>/<input type=date>
popups from opening in some browsers. The end state is visually identical
either way (translateY(0) === no transform), so dropping fill-mode is free. */
.animate-rise { animation: rise 0.45s cubic-bezier(0.16, 1, 0.3, 1); }

/* Backdrop fade and side-panel slide-in, used by the Drawer component. */
@keyframes fade {
  from { opacity: 0; }
  to { opacity: 1; }
}
.animate-fade { animation: fade 0.25s ease-out both; }

@keyframes slide-in {
  from { transform: translateX(100%); }
  to { transform: translateX(0); }
}
/* "backwards" (not "both"): holds the off-screen start position through any
delay so there's no flash, but drops the transform once the animation ends
instead of retaining it forever – see the .animate-rise comment above. */
.animate-slide-in { animation: slide-in 0.32s cubic-bezier(0.16, 1, 0.3, 1) backwards; }

::-webkit-scrollbar { width: 10px; height: 10px; }
::-webkit-scrollbar-track { background: transparent; }
::-webkit-scrollbar-thumb { background: #cbd5e1; border-radius: 9999px; border: 2px solid #f6f8fb; }
::-webkit-scrollbar-thumb:hover { background: #94a3b8; }

```

web/src/main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

web/src/pages/HomePage.jsx

```
// -----
// File: HomePage.jsx
// Purpose: Main landing dashboard – a welcome hero, live KPI tiles pulled from
//          the API, and quick-access cards for each area the signed-in role can
//          reach. This is the console's opening screen after sign-in.
// Module owner: shared infrastructure
// -----
import { useEffect, useState } from "react";
import { Link } from "react-router-dom";
import apiClient from "../api/client";
import { useAuth } from "../context/AuthContext";
import StatCard from "../components/ui/StatCard";
import { Card } from "../components/ui/Card";
import {
  ArrowRightIcon, BatteryIcon, BoltIcon, CalendarIcon, ClockIcon, GaugeIcon,
  StationIcon, SunIcon, UserCheckIcon, UsersIcon,
} from "../components/ui/Icons";

const QUICK_LINKS = {
  Backoffice: [
    { to: "/backoffice/pending", label: "Pending Activations", copy: "Review and approve new prosumer registrations.", icon: UserCheckIcon, tone: "amber" },
    { to: "/backoffice/users", label: "Staff Users", copy: "Create Backoffice and Grid Operator accounts.", icon: UsersIcon, tone: "sky" },
    { to: "/backoffice/stations", label: "Microgrid Nodes", copy: "Manage hubs, GPS, capacity and schedules.", icon: StationIcon, tone: "emerald" },
    { to: "/backoffice/reservations", label: "Reservations", copy: "Book on behalf, approve and cancel bookings.", icon: CalendarIcon, tone: "emerald" },
    { to: "/operator", label: "Operator Console", copy: "Battery availability and the approval queue.", icon: GaugeIcon, tone: "sky" },
  ],
  GridOperator: [
    { to: "/operator", label: "Operator Console", copy: "Battery availability and the approval queue.", icon: GaugeIcon, tone: "emerald" },
  ],
};

const TONE_RING = {
  emerald: "bg-brand-50 text-brand-700 ring-brand-100",
  amber: "bg-solar-50 text-solar-600 ring-solar-200",
  sky: "bg-sky-50 text-sky-700 ring-sky-100",
};

// Renders the role-aware dashboard landing page.
export default function HomePage() {
  const { user } = useAuth();
  const [stats, setStats] = useState(null);
  const [loading, setLoading] = useState(true);

  // Loads the counters shown in the KPI row (dashboard + stations + pending queue).
  useEffect(() => {
    let cancelled = false;
    async function load() {
      try {
        const requests = [apiClient.get("/dashboard/operator"), apiClient.get("/stations")];
        if (user.userType === "Backoffice") requests.push(apiClient.get("/users/pending"));
        const [dash, stations, pending] = await Promise.all(requests);
        if (cancelled) return;
        setStats({

```

```

    pendingReservations: dash.data.pendingReservationsCount,
    approvedFuture: dash.data.approvedFutureReservationsCount,
    activeStations: stations.data.filter((s) => s.status === "Active").length,
    totalStations: stations.data.length,
    batteryAvailable: stations.data.reduce((sum, s) => sum + s.availableBatterySlots, 0),
    batteryTotal: stations.data.reduce((sum, s) => sum + s.totalBatterySlots, 0),
    pendingAccounts: pending ? pending.data.length : null,
  });
} finally {
  if (!cancelled) setLoading(false);
}
}
load();
return () => { cancelled = true; };
}, [user.userType]);

const links = QUICK_LINKS[user.userType] || [];
const firstName = user.fullName?.split(" ")[0] || "there";

return (
  <div className="space-y-8">
    {/* Hero */}
    <section className="bg-aurora relative overflow-hidden rounded-3xl bg-ink-900 px-7 py-9 sm:px-10 sm:py-
11">
      <div className="bg-grid absolute inset-0 opacity-60" />
      <div className="relative flex flex-wrap items-end justify-between gap-6">
        <div className="max-w-2xl">
          <span className="inline-flex items-center gap-2 rounded-full bg-white/10 px-3 py-1.5 text-[11px]
font-semibold uppercase tracking-wider text-brand-200 ring-1 ring-inset ring-white/15">
            <SunIcon className="h-3.5 w-3.5" />
            {user.userType === "Backoffice" ? "Backoffice control centre" : "Grid operations"}
          </span>
          <h2 className="mt-4 font-display text-[32px] font-extrabold leading-tight tracking-tight text-
white sm:text-[38px]">
            Good to see you, {firstName}.
          </h2>
          <p className="mt-2.5 max-w-xl text-[15px] leading-relaxed text-white/60">
            Here's the live state of the microgrid network: nodes, battery capacity and the
            energy-trading bookings waiting on your action.
          </p>
        </div>
        <div>
          <span className="grid h-16 w-16 place-items-center rounded-2xl bg-white/10 text-brand-200 ring-1
ring-inset ring-white/15 backdrop-blur">
            <BoltIcon className="h-8 w-8" />
          </span>
        </div>
      </div>
    </section>

    {/* KPIs */}
    <section className="grid gap-5 sm:grid-cols-2 xl:grid-cols-4">
      <StatCard
        label="Pending reservations"
        value={stats?.pendingReservations ?? 0}
        hint="Awaiting operator approval"
        icon={ClockIcon}
        tone="amber"
        loading={loading}
      />
      <StatCard
        label="Approved (future)"
        value={stats?.approvedFuture ?? 0}
        hint="Confirmed upcoming transfers"
        icon={CalendarIcon}
        tone="emerald"
        loading={loading}
      />
      <StatCard
        label="Active nodes"
        value={stats?.activeStations ?? 0}
        hint={stats ? `${stats.totalStations} registered in total` : ""}
        icon={StationIcon}
        tone="sky"
        loading={loading}
      />
      <StatCard
        label="Battery slots free"
        value={stats?.batteryAvailable ?? 0}

```

```

        hint={stats ? `of ${stats.batteryTotal} across all nodes` : ""}
        icon={BatteryIcon}
        tone="emerald"
        loading={loading}
      />
    </section>

    { /* Quick access */ }
    <section>
      <div className="mb-4 flex items-baseline justify-between gap-4">
        <h3 className="font-display text-lg font-bold tracking-tight text-slate-900">Quick access</h3>
        {stats?.pendingAccounts > 0 && (
          <Link to="/backoffice/pending" className="text-sm font-semibold text-brand-700 hover:text-brand-800">
            {stats.pendingAccounts} account{stats.pendingAccounts === 1 ? "" : "s"} awaiting activation →
          </Link>
        )}
      </div>

      <div className="grid gap-5 sm:grid-cols-2 xl:grid-cols-3">
        {links.map((link) => (
          <Link key={link.to} to={link.to} className="group">
            <Card className="h-full p-6 transition-all duration-200 hover:-translate-y-0.5 hover:border-brand-200 hover:shadow-[0_2px_4px_rgb(15_23_42/0.05),0_22px_44px_-20px_rgb(5_150_105/0.35)]">
              <div className="flex items-start justify-between gap-4">
                <span className={`grid h-12 w-12 shrink-0 place-items-center rounded-xl ring-1 ring-inset ${TONE_RING[link.tone]}`}>
                  <link.icon className="h-6 w-6" />
                </span>
                <ArrowRightIcon className="h-5 w-5 text-slate-300 transition-all duration-200 group-hover:translate-x-0.5 group-hover:text-brand-600" />
              </div>
              <h4 className="mt-5 text-[15px] font-bold text-slate-900">{link.label}</h4>
              <p className="mt-1.5 text-sm leading-relaxed text-slate-500">{link.copy}</p>
            </Card>
          </Link>
        )})}
      </div>
    </section>
  </div>
);
}

```

web/src/pages/LoginPage.jsx

```

// -----
// File: LoginPage.jsx
// Purpose: Split-screen sign-in for Backoffice and Grid Operator staff – a
//          branded aurora panel on the left, the credential form on the right.
//          Redirects to the role-appropriate landing page on success.
// Module owner: Member A (Identity & Access)
// -----
import { useState } from "react";
import { useNavigate } from "react-router-dom";
import { useAuth } from "../../context/AuthContext";
import Button from "../../components/ui/Button";
import { Field, Input } from "../../components/ui/Field";
import { AlertIcon, BatteryIcon, ShieldIcon, SunIcon } from "../../components/ui/Icons";

const HIGHLIGHTS = [
  { icon: SunIcon, title: "Live grid visibility", copy: "Track every microgrid node, battery slot and booking in one console." },
  { icon: BatteryIcon, title: "Controlled energy trading", copy: "7-day booking window and 12-hour change rules enforced server-side." },
  { icon: ShieldIcon, title: "Verified transfers", copy: "Signed QR tokens confirm every prosumer hand-off at the station." },
];

// Renders the branded sign-in experience.
export default function LoginPage() {
  const [identifier, setIdentifier] = useState("");
  const [password, setPassword] = useState("");
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);

```

```

const { login } = useAuth();
const navigate = useNavigate();

// Submits credentials and routes by the role the API returns.
async function handleSubmit(e) {
  e.preventDefault();
  setError("");
  setLoading(true);
  try {
    const profile = await login(identifier, password);
    navigate(profile.userType === "Backoffice" ? "/" : "/operator");
  } catch (err) {
    setError(err.response?.data?.message || "Login failed. Please check your credentials.");
  } finally {
    setLoading(false);
  }
}

return (
  <div className="grid min-h-screen lg:grid-cols-[1.05fr_1fr]">
    {/ * Brand panel */}
    <div className="bg-aurora relative hidden overflow-hidden bg-ink-950 lg:block">
      <div className="bg-grid absolute inset-0 opacity-70" />
      <div className="relative flex h-full flex-col justify-between p-12 xl:p-16">
        <div className="flex items-center gap-3">
          <span className="grid h-11 w-11 shrink-0 place-items-center overflow-hidden rounded-xl bg-white shadow-[0_14px_30px_-10px_rgba(16_185_129/0.9)]">
            
          </span>
          <span>
            <span className="block font-display text-base font-extrabold leading-tight text-white">Solar Microgrid</span>
            <span className="block text-[11px] font-medium tracking-wide text-white/45">Trading Console</span>
          </span>
        </div>

        <div className="max-w-lg">
          <h1 className="font-display text-[44px] font-extrabold leading-[1.08] tracking-tight text-white xl:text-[52px]">
            Trade stored sunlight
            <span className="block bg-gradient-to-r from-brand-300 via-brand-200 to-solar-300 bg-clip-text text-transparent">
              across the grid.
            </span>
          </h1>
          <p className="mt-5 text-[15px] leading-relaxed text-white/60">
            The operations console for backoffice teams and grid operators managing solar microgrid nodes, prosumer accounts and energy slot reservations.
          </p>

          <div className="mt-10 space-y-5">
            {HIGHLIGHTS.map((h) => (
              <div key={h.title} className="flex items-start gap-4">
                <span className="grid h-10 w-10 shrink-0 place-items-center rounded-xl bg-white/10 text-brand-200 ring-1 ring-inset ring-white/10">
                  <h.icon className="h-5 w-5" />
                </span>
                <div>
                  <p className="text-sm font-semibold text-white">{h.title}</p>
                  <p className="mt-0.5 text-[13px] leading-relaxed text-white/50">{h.copy}</p>
                </div>
              </div>
            ))}
          </div>
        </div>

        <div />
      </div>

    {/ * Form panel */}
    <div className="flex items-center justify-center bg-white px-6 py-12 sm:px-12">
      <div className="w-full max-w-[400px]">
        <div className="mb-9 lg:hidden">

```

```

        <span className="grid h-12 w-12 place-items-center overflow-hidden rounded-xl bg-white shadow-sm
ring-1 ring-slate-200">
          
        </span>
      </div>

      <h2 className="font-display text-[30px] font-extrabold tracking-tight text-slate-900">Welcome
back</h2>
      <p className="mt-2 text-[15px] text-slate-500">
        Sign in to the Backoffice & Grid Operator console.
      </p>

      <form onSubmit={handleSubmit} className="mt-9 space-y-5">
        {error && (
          <div className="flex items-start gap-3 rounded-xl border border-rose-200 bg-rose-50 p-3.5">
            <AlertIcon className="mt-0.5 h-4 w-4 shrink-0 text-rose-600" />
            <p className="text-[13px] font-medium text-rose-700">{error}</p>
          </div>
        )}

        <Field label="Username" required>
          <Input
            value={identifier}
            onChange={(e) => setIdentifier(e.target.value)}
            placeholder="e.g. admin"
            autoComplete="username"
            required
          />
        </Field>

        <Field label="Password" required>
          <Input
            type="password"
            value={password}
            onChange={(e) => setPassword(e.target.value)}
            placeholder="....."
            autoComplete="current-password"
            required
          />
        </Field>

        <Button type="submit" size="lg" className="w-full" disabled={loading}>
          {loading ? "Signing in..." : "Sign in to console"}
        </Button>
      </form>
    </div>
  </div>
</div>
);
}

```

web/src/pages/backoffice/PendingActivationsPage.jsx

```

// -----
// File: PendingActivationsPage.jsx
// Purpose: Backoffice review queue for accounts awaiting activation (mostly
//         mobile prosumer registrations), with an approve action per row.
// Module owner: Member A (Identity & Access)
// -----
import { useEffect, useState } from "react";
import apiClient from "../../api/client";
import { Card, CardHeader } from "../../components/ui/Card";
import DataTable from "../../components/ui/DataTable";
import Button from "../../components/ui/Button";
import { StatusBadge } from "../../components/ui/Badge";
import { useToast } from "../../components/ui/Toast";
import { CheckIcon, UserCheckIcon } from "../../components/ui/Icons";

// Renders the PendingActivation queue and approves accounts in place.
export default function PendingActivationsPage() {
  const [pending, setPending] = useState([]);
  const [loading, setLoading] = useState(true);
  const [busyId, setBusyId] = useState(null);

```

```

const toast = useToast();

// Loads every account currently in PendingActivation.
async function loadPending() {
  setLoading(true);
  try {
    const { data } = await apiClient.get("/users/pending");
    setPending(data);
  } finally {
    setLoading(false);
  }
}

useEffect(() => { loadPending(); }, []);

// Approves one account and refreshes the queue.
async function handleApprove(user) {
  setBusyId(user.id);
  try {
    await apiClient.put(`/users/${user.id}/approve`);
    toast.success(`${user.fullName} activated.`);
    await loadPending();
  } catch (err) {
    toast.error(err.response?.data?.message || "Could not approve this account.");
  } finally {
    setBusyId(null);
  }
}

const columns = [
  {
    key: "fullName",
    header: "Applicant",
    render: (u) => (
      <div>
        <p className="font-semibold text-slate-900">{u.fullName}</p>
        <p className="text-xs text-slate-500">{u.email}</p>
      </div>
    ),
  },
  { key: "userType", header: "Type", render: (u) => <span className="text-slate-600">{u.userType}</span> },
  { key: "nic", header: "NIC", render: (u) => <span className="font-mono text-[13px]">{u.nic} || "–"</span> },
  { key: "phone", header: "Phone", render: (u) => u.phone || "–" },
  {
    key: "createdAt",
    header: "Registered",
    render: (u) => new Date(u.createdAt).toLocaleString(undefined, { dateStyle: "medium", timeStyle: "short" })
  },
  { key: "status", header: "Status", render: (u) => <StatusBadge status={u.status} /> },
  {
    key: "actions",
    header: "",
    className: "text-right",
    render: (u) => (
      <Button size="sm" icon={CheckIcon} disabled={busyId === u.id} onClick={() => handleApprove(u)}>
        {busyId === u.id ? "Approving..." : "Approve"}
      </Button>
    ),
  },
];

return (
  <Card>
    <CardHeader>
      icon={UserCheckIcon}
      title="Pending activations"
      description="Accounts registered from the mobile app stay inactive until a Backoffice officer approves them."
      actions={
        !loading && (
          <span className="rounded-full bg-solar-50 px-3 py-1.5 text-xs font-bold text-solar-600 ring-1 ring-inset ring-solar-200">
            {pending.length} waiting
          </span>
        )
      }
    </CardHeader>
  </Card>
);

```

```

    )
  }
/>
<DataTable
  columns={columns}
  rows={pending}
  loading={loading}
  empty={{
    icon: UserCheckIcon,
    title: "Nothing awaiting activation",
    description: "New prosumer registrations from the Android app will appear here for approval.",
  }}
/>
</Card>
);
}

```

web/src/pages/backoffice/ReservationsPage.jsx

```

// -----
// File: ReservationsPage.jsx
// Purpose: Backoffice/Grid Operator screen to book an energy slot on a
//          prosumer's behalf and to approve or cancel pending reservations.
//          The 7-day window and 12-hour notice rules are enforced by the API;
//          this screen surfaces whatever it returns.
// Module owner: Member C (Reservations & Booking Workflow)
// -----
import { useEffect, useState } from "react";
import apiClient from "../../api/client";
import StatCard from "../../components/ui/StatCard";
import { Card, CardHeader } from "../../components/ui/Card";
import DataTable from "../../components/ui/DataTable";
import Button from "../../components/ui/Button";
import { Field, Input, Select } from "../../components/ui/Field";
import { StatusBadge } from "../../components/ui/Badge";
import Modal from "../../components/ui/Modal";
import Drawer from "../../components/ui/Drawer";
import { useToast } from "../../components/ui/Toast";
import {
  AlertIcon, BoltIcon, CalendarIcon, CheckIcon, ClockIcon, PlusIcon, StationIcon,
} from "../../components/ui/Icons";

const EMPTY_FORM = { prosumerNic: "", stationId: "", slotId: "", scheduledDateTime: "", energyAmountKWh: "" };

// Renders the pending-reservation queue and the book-on-behalf dialog.
export default function ReservationsPage() {
  const [stations, setStations] = useState([]);
  const [slots, setSlots] = useState([]);
  const [pending, setPending] = useState([]);
  const [form, setForm] = useState(EMPTY_FORM);
  const [loading, setLoading] = useState(true);
  const [saving, setSaving] = useState(false);
  const [formOpen, setFormOpen] = useState(false);
  const [busyId, setBusyId] = useState(null);
  const [cancelTarget, setCancelTarget] = useState(null);
  const [cancelReason, setCancelReason] = useState("");
  const toast = useToast();

  // Loads active stations and the pending-reservations queue together.
  async function loadInitial() {
    setLoading(true);
    try {
      const [stationsRes, pendingRes] = await Promise.all([
        apiClient.get("/stations"),
        apiClient.get("/reservations/pending"),
      ]);
      setStations(stationsRes.data.filter((s) => s.status === "Active"));
      setPending(pendingRes.data);
    } finally {
      setLoading(false);
    }
  }

  useEffect(() => { loadInitial(); }, []);

```

```

// Fetches bookable slots whenever the chosen station changes.
async function handleStationChange(stationId) {
  setForm({ ...form, stationId, slotId: "" });
  if (!stationId) { setSlots([]); return; }
  const { data } = await apiClient.get(`/stations/${stationId}/slots`);
  setSlots(data.filter((s) => s.status !== "Closed"));
}

// Creates a reservation for the entered prosumer NIC.
async function handleCreate(e) {
  e.preventDefault();
  setSaving(true);
  try {
    await apiClient.post("/reservations", {
      prosumerNic: form.prosumerNic,
      stationId: form.stationId,
      slotId: form.slotId,
      scheduledDateTime: new Date(form.scheduledDateTime).toISOString(),
      energyAmountKWh: parseFloat(form.energyAmountKWh),
    });
    toast.success("Reservation created.");
    setForm(EMPTY_FORM);
    setSlots([]);
    setFormOpen(false);
    await loadInitial();
  } catch (err) {
    toast.error(err.response?.data?.message || "Could not create this reservation.");
  } finally {
    setSaving(false);
  }
}

// Approves a pending reservation, which issues its QR token.
async function handleApprove(reservation) {
  setBusyId(reservation.id);
  try {
    await apiClient.put(`/reservations/${reservation.id}/approve`);
    toast.success(`${reservation.reservationCode} approved, QR issued.`);
    await loadInitial();
  } catch (err) {
    toast.error(err.response?.data?.message || "Could not approve this reservation.");
  } finally {
    setBusyId(null);
  }
}

// Submits the cancellation confirmed in the dialog.
async function applyCancel() {
  const reservation = cancelTarget;
  setCancelTarget(null);
  try {
    await apiClient.put(`/reservations/${reservation.id}/cancel`, { reason: cancelReason });
    toast.success(`${reservation.reservationCode} cancelled.`);
    setCancelReason("");
    await loadInitial();
  } catch (err) {
    toast.error(err.response?.data?.message || "Could not cancel this reservation.");
  }
}

const stationName = (id) => stations.find((s) => s.id === id)?.stationName || "-";

const columns = [
  {
    key: "reservationCode",
    header: "Reservation",
    className: "min-w-[200px]",
    render: (r) => (
      <div className="flex items-center gap-3">
        <span className="grid h-10 w-10 shrink-0 place-items-center rounded-xl bg-solar-50 text-solar-600 ring-1 ring-inset ring-solar-200">
          <CalendarIcon className="h-5 w-5" />
        </span>
        <div>

```

```

        <p className="font-mono text-[13px] font-bold text-slate-900">{r.reservationCode}</p>
        <p className="text-xs text-slate-500">NIC {r.prosumerNic}</p>
      </div>
    </div>
  ),
},
{
  key: "station",
  header: "Node",
  className: "whitespace-nowrap",
  render: (r) => (
    <span className="inline-flex items-center gap-1.5 text-slate-700">
      <StationIcon className="h-4 w-4 text-slate-400" />
      {stationName(r.stationId)}
    </span>
  ),
},
{
  key: "scheduledDateTime",
  header: "Scheduled",
  className: "whitespace-nowrap",
  render: (r) => (
    <div className="flex items-center gap-2">
      <ClockIcon className="h-4 w-4 shrink-0 text-slate-400" />
      <span>{new Date(r.scheduledDateTime).toLocaleString(undefined, { dateStyle: "medium", timeStyle:
"short" })}</span>
    </div>
  ),
},
{
  key: "energyAmountKWh",
  header: "Energy",
  className: "whitespace-nowrap",
  render: (r) => (
    <span className="inline-flex items-center gap-1.5 font-semibold text-slate-800">
      <BoltIcon className="h-4 w-4 text-solar-500" />
      {r.energyAmountKWh} kWh
    </span>
  ),
},
{
  key: "status", header: "Status", render: (r) => <StatusBadge status={r.status} /> },
{
  key: "actions",
  header: "",
  className: "text-right whitespace-nowrap",
  render: (r) => (
    <div className="flex items-center justify-end gap-2">
      <Button size="sm" icon={CheckIcon} disabled={busyId === r.id} onClick={() => handleApprove(r)}>
        {busyId === r.id ? "Approving..." : "Approve"}
      </Button>
      <Button size="sm" variant="dangerGhost" onClick={() => setCancelTarget(r)}>Cancel</Button>
    </div>
  ),
},
];

const totalEnergy = pending.reduce((sum, r) => sum + r.energyAmountKWh, 0);
const next = [...pending].sort((a, b) => new Date(a.scheduledDateTime) - new Date(b.scheduledDateTime))[0];

return (
  <div className="space-y-6">
    <section className="grid gap-5 sm:grid-cols-3">
      <StatCard label="Pending approval" value={pending.length} hint="Bookings waiting on an operator"
icon={ClockIcon} tone="amber" loading={loading} />
      <StatCard label="Energy requested" value={`\${totalEnergy.toFixed(1)} kWh`} hint="Across all pending
bookings" icon={BoltIcon} tone="emerald" loading={loading} />
      <StatCard
        label="Next transfer"
        value={next ? new Date(next.scheduledDateTime).toLocaleDateString(undefined, { day: "numeric",
month: "short" }) : "-"}
        hint={next ? `\${next.reservationCode} · \${new Date(next.scheduledDateTime).toLocaleTimeString(unde-
fined, { timeStyle: "short" })}` : "Nothing scheduled"}
        icon={CalendarIcon}
        tone="sky"
        loading={loading}
      />
    </section>
  </div>
);

```

```

<Card>
  <CardHeader
    icon={CalendarIcon}
    title="Pending reservations"
    description="Bookings waiting for approval before a transaction QR token is issued."
    actions={<Button icon={PlusIcon} onClick={() => setFormOpen(true)}>Book on behalf</Button>}
  />
  <DataTable
    columns={columns}
    rows={pending}
    loading={loading}
    empty={{
      icon: CheckIcon,
      title: "Queue is clear",
      description: "Approved, completed and cancelled bookings no longer appear in this queue.",
      action: <Button icon={PlusIcon} onClick={() => setFormOpen(true)}>Book on behalf</Button>,
    }}
  />
</Card>

{/* Booking panel */}
<Drawer
  open={formOpen}
  onClose={() => setFormOpen(false)}
  size="lg"
  icon={PlusIcon}
  title="Book an energy slot on behalf"
  description="For prosumers who call in rather than booking through the mobile app."
  footer={
    <>
      <Button variant="subtle" onClick={() => setFormOpen(false)}>Cancel</Button>
      <Button type="submit" form="reservation-form" icon={PlusIcon} disabled={saving}>
        {saving ? "Creating..." : "Create reservation"}
      </Button>
    </>
  }
>
  <form id="reservation-form" onSubmit={handleCreate} className="space-y-4 pb-2">
    <Field label="Prosumer NIC" required hint="The prosumer's account must already be active.">
      <Input value={form.prosumerNic} onChange={(e) => setForm({ ...form, prosumerNic: e.target.value
    }}} placeholder="199512345678" required />
    </Field>

    <div className="grid gap-4 sm:grid-cols-2">
      <Field label="Station" required>
        <Select value={form.stationId} onChange={(e) => handleStationChange(e.target.value)} required>
          <option value="">Select a station...</option>
          {stations.map((s) => <option key={s.id} value={s.id}>{s.stationName}</option>)}
        </Select>
      </Field>
      <Field label="Available slot" required>
        <Select value={form.slotId} onChange={(e) => setForm({ ...form, slotId: e.target.value })} re-
quired disabled={!form.stationId}>
          <option value="">{form.stationId ? "Select a slot..." : "Choose a station first"}</option>
          {slots.map((s) => (
            <option key={s.id} value={s.id}>
              {new Date(s.date).toLocaleDateString()} · {s.startTime}-{s.endTime} ({s.capacityAvailable}
left)
            </option>
          ))}
        </Select>
      </Field>
      <Field label="Scheduled date & time" required hint="Must fall within the next 7 days.">
        <Input type="datetime-local" value={form.scheduledDateTime} onChange={(e) => setForm({ ...form,
scheduledDateTime: e.target.value })} required />
      </Field>
      <Field label="Energy amount (kWh)" required>
        <Input type="number" step="0.1" value={form.energyAmountKWh} onChange={(e) => setForm({ ...form,
energyAmountKWh: e.target.value })} placeholder="6.5" required />
      </Field>
    </div>

    <div className="flex items-start gap-3 rounded-xl border border-solar-200 bg-solar-50 p-3.5">
      <AlertIcon className="mt-0.5 h-4 w-4 shrink-0 text-solar-600" />
      <p className="text-[13px] leading-relaxed text-solar-600">

```

```

        The service enforces a 7-day booking window, and later updates or cancellations need at
        least 12 hours' notice before the scheduled time.
    </p>
</div>
</form>
</Drawer>

{/* Cancellation confirmation */}
<Modal
  open={Boolean(cancelTarget)}
  onClose={() => setCancelTarget(null)}
  icon={AlertIcon}
  tone="danger"
  title="Cancel this reservation?"
  description={` ${cancelTarget?.reservationCode} will be released back to the slot. The API rejects this
if less than 12 hours remain before the scheduled time.`}
  footer={
    <>
      <Button variant="subtle" onClick={() => setCancelTarget(null)}>Keep reservation</Button>
      <Button variant="danger" onClick={applyCancel}>Cancel reservation</Button>
    </>
  }
>
  <Field label="Reason">
    <Input value={cancelReason} onChange={(e) => setCancelReason(e.target.value)} placeholder="Prosumer
requested cancellation" />
  </Field>
</Modal>
</div>
);
}

```

web/src/pages/backoffice/StationsPage.jsx

```

// -----
// File: StationsPage.jsx
// Purpose: Backoffice screen to register microgrid node hubs (GPS, capacity,
//          battery slots, weekly operating schedule) and to deactivate them.
//          Deactivation is refused server-side while active reservations exist.
// Module owner: Member B (Microgrid Nodes & Maps)
// -----
import { useEffect, useState } from "react";
import apiClient from "../../api/client";
import StatCard from "../../components/ui/StatCard";
import { Card, CardHeader } from "../../components/ui/Card";
import DataTable from "../../components/ui/DataTable";
import Button from "../../components/ui/Button";
import { Field, Input, Select } from "../../components/ui/Field";
import { StatusBadge } from "../../components/ui/Badge";
import Modal from "../../components/ui/Modal";
import Drawer from "../../components/ui/Drawer";
import { useToast } from "../../components/ui/Toast";
import {
  AlertIcon, BatteryIcon, BoltIcon, CalendarIcon, MapPinIcon, PlusIcon, StationIcon, XIcon,
} from "../../components/ui/Icons";

const DAYS = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"];
const EMPTY_FORM = {
  stationName: "", lat: "", lng: "", capacityKWh: "", totalBatterySlots: "",
  schedule: [{ day: "Monday", openTime: "08:00", closeTime: "18:00" }],
};

// Renders the node directory plus the registration dialog.
export default function StationsPage() {
  const [stations, setStations] = useState([]);
  const [form, setForm] = useState(EMPTY_FORM);
  const [loading, setLoading] = useState(true);
  const [saving, setSaving] = useState(false);
  const [formOpen, setFormOpen] = useState(false);
  const [confirm, setConfirm] = useState(null);
  const [generatingId, setGeneratingId] = useState(null);
  const toast = useToast();

  // Loads every station, active and deactivated.

```

```

async function loadStations() {
  setLoading(true);
  try {
    const { data } = await apiClient.get("/stations");
    setStations(data);
  } finally {
    setLoading(false);
  }
}

useEffect(() => { loadStations(); }, []);

// Updates one field of one schedule row.
function updateScheduleRow(index, field, value) {
  setForm({ ...form, schedule: form.schedule.map((row, i) => (i === index ? { ...row, [field]: value } :
row)) });
}

// Appends another operating-hours row.
function addScheduleRow() {
  setForm({ ...form, schedule: [...form.schedule, { day: "Monday", openTime: "08:00", closeTime: "18:00" }]
});
}

// Removes an operating-hours row.
function removeScheduleRow(index) {
  setForm({ ...form, schedule: form.schedule.filter((_, i) => i !== index) });
}

// Creates the station from the dialog form values.
async function handleCreate(e) {
  e.preventDefault();
  setSaving(true);
  try {
    await apiClient.post("/stations", {
      stationName: form.stationName,
      lat: parseFloat(form.lat),
      lng: parseFloat(form.lng),
      capacityKWh: parseFloat(form.capacityKWh),
      totalBatterySlots: parseInt(form.totalBatterySlots, 10),
      schedule: form.schedule,
    });
    toast.success(`${form.stationName} registered.`);
    setForm(EMPTY_FORM);
    setFormOpen(false);
    await loadStations();
  } catch (err) {
    toast.error(err.response?.data?.message || "Could not create this station.");
  } finally {
    setSaving(false);
  }
}

// Deactivates the station confirmed in the dialog.
async function applyDeactivate() {
  const station = confirm;
  setConfirm(null);
  try {
    await apiClient.put(`/stations/${station.id}/deactivate`);
    toast.success(`${station.stationName} deactivated.`);
    await loadStations();
  } catch (err) {
    toast.error(err.response?.data?.message || "Could not deactivate this station.");
  }
}

// Tops up bookable slots for a station from its schedule (also covers stations
// registered before slot auto-generation, and extends the rolling window over time).
async function handleGenerateSlots(station) {
  setGeneratingId(station.id);
  try {
    const { data } = await apiClient.post(`/stations/${station.id}/generate-slots`);
    toast.success(
      data.created > 0
      ? `${data.created} new slot${data.created === 1 ? "" : "s"} generated for ${station.stationName}.`
    );
  }
}

```

```

        : `${station.stationName} is already up to date – no new slots needed.`
    );
} catch (err) {
    toast.error(err.response?.data?.message || "Could not generate slots for this station.");
} finally {
    setGeneratingId(null);
}
}

const columns = [
    {
        key: "stationName",
        header: "Node",
        className: "min-w-[240px]",
        render: (s) => (
            <div className="flex items-center gap-3">
                <span className="grid h-10 w-10 shrink-0 place-items-center rounded-xl bg-brand-50 text-brand-700 ring-1 ring-inset ring-brand-100">
                    <StationIcon className="h-5 w-5" />
                </span>
                <div className="min-w-0">
                    <p className="truncate font-semibold text-slate-900">{s.stationName}</p>
                    <p className="mt-0.5 flex items-center gap-1 text-xs text-slate-500">
                        <MapPinIcon className="h-3 w-3 shrink-0" />
                        {s.lat.toFixed(4)}, {s.lng.toFixed(4)}
                    </p>
                </div>
            </div>
        ),
    },
    {
        key: "capacityKWh",
        header: "Capacity",
        className: "whitespace-nowrap",
        render: (s) => (
            <span className="inline-flex items-center gap-1.5 font-semibold text-slate-800">
                <BoltIcon className="h-4 w-4 text-solar-500" />
                {s.capacityKWh} kWh
            </span>
        ),
    },
    {
        key: "battery",
        header: "Battery slots",
        className: "min-w-[180px]",
        render: (s) => {
            const pct = s.totalBatterySlots ? (s.availableBatterySlots / s.totalBatterySlots) * 100 : 0;
            return (
                <div className="w-40">
                    <div className="mb-1.5 flex items-center justify-between text-xs">
                        <span className="font-semibold text-slate-700">{s.availableBatterySlots}/{s.totalBatterySlots}</span>
                        <span className="text-slate-400">{Math.round(pct)}%</span>
                    </div>
                    <div className="h-1.5 overflow-hidden rounded-full bg-slate-100">
                        <div
                            className={`h-full rounded-full bg-gradient-to-r ${pct > 50 ? "from-brand-400 to-brand-600" :
                                pct > 20 ? "from-solar-300 to-solar-500" : "from-rose-400 to-rose-600"}`}
                            style={{ width: `${pct}%` }}
                        </div>
                    </div>
                </div>
            );
        },
    },
    {
        key: "schedule",
        header: "Operating hours",
        className: "whitespace-nowrap",
        render: (s) =>
            s.schedule?.length ? (
                <div>
                    <p className="font-medium text-slate-700">{s.schedule[0].openTime} – {s.schedule[0].closeTime}</p>
                    <p className="text-xs text-slate-500">{s.schedule.map((d) => d.day.slice(0, 3)).join(", ")}</p>
                </div>
            ) : (
                <span className="text-slate-400">—</span>
            )
    },
];

```

```

    ),
    { key: "status", header: "Status", render: (s) => <StatusBadge status={s.status} /> },
    {
      key: "actions",
      header: "",
      className: "text-right whitespace-nowrap",
      render: (s) =>
        s.status === "Active" ? (
          <div className="flex items-center justify-end gap-2">
            <Button
              size="sm"
              variant="subtle"
              icon={CalendarIcon}
              disabled={generatingId === s.id}
              onClick={() => handleGenerateSlots(s)}
            />
            {generatingId === s.id ? "Generating..." : "Generate slots"}
          </Button>
          <Button size="sm" variant="dangerGhost" onClick={() => setConfirm(s)}>Deactivate</Button>
        </div>
        ) : null,
    }
  ],

  const active = stations.filter((s) => s.status === "Active");
  const totalCapacity = stations.reduce((sum, s) => sum + s.capacityKWh, 0);
  const freeSlots = stations.reduce((sum, s) => sum + s.availableBatterySlots, 0);
  const totalSlots = stations.reduce((sum, s) => sum + s.totalBatterySlots, 0);

  return (
    <div className="space-y-6">
      <section className="grid gap-5 sm:grid-cols-3">
        <StatCard label="Active nodes" value={active.length} hint={` ${stations.length} registered in total`}
          icon={StationIcon} tone="emerald" loading={loading} />
        <StatCard label="Network capacity" value={`${totalCapacity} kWh`} hint="Combined across every node"
          icon={BoltIcon} tone="amber" loading={loading} />
        <StatCard label="Battery slots free" value={freeSlots} hint={`of ${totalSlots} installed`} icon={BatteryIcon}
          tone="sky" loading={loading} />
      </section>

      <Card>
        <CardHeader>
          icon={StationIcon}
          title="Microgrid nodes"
          description="Solar hubs available for prosumer energy drop-off and charging."
          actions=<Button icon={PlusIcon} onClick={() => setFormOpen(true)}>Register node</Button>
        </CardHeader>
        <DataTable>
          columns={columns}
          rows={stations}
          loading={loading}
          empty={{
            icon: StationIcon,
            title: "No microgrid nodes yet",
            description: "Register the first solar hub to start accepting prosumer energy bookings.",
            action: <Button icon={PlusIcon} onClick={() => setFormOpen(true)}>Register node</Button>,
          }}
        </DataTable>
      </Card>

      <\/Registration panel *\/>
      <Drawer>
        open={formOpen}
        onClose={() => setFormOpen(false)}
        size="lg"
        icon={StationIcon}
        title="Register a microgrid node"
        description="GPS position drives the nearby-stations map in the prosumer mobile app."
        footer={
          <>
            <Button variant="subtle" onClick={() => setFormOpen(false)}>Cancel</Button>
            <Button type="submit" form="station-form" icon={PlusIcon} disabled={saving}>
              {saving ? "Registering..." : "Register node"}
            </Button>
          <\/>
        }
      </Drawer>
    <\/>
  );

```

```

    }
  >
  <form id="station-form" onSubmit={handleCreate} className="space-y-4 pb-2">
    <Field label="Station name" required>
      <Input value={form.stationName} onChange={(e) => setForm({ ...form, stationName: e.target.value
    }}} placeholder="Colombo Fort Hub" required />
    </Field>

    <div className="grid gap-4 sm:grid-cols-2">
      <Field label="Latitude" required>
        <Input type="number" step="0.000001" value={form.lat} onChange={(e) => setForm({ ...form, lat:
e.target.value }}} placeholder="6.9344" required />
      </Field>
      <Field label="Longitude" required>
        <Input type="number" step="0.000001" value={form.lng} onChange={(e) => setForm({ ...form, lng:
e.target.value }}} placeholder="79.8428" required />
      </Field>
      <Field label="Capacity (kWh)" required>
        <Input type="number" step="0.1" value={form.capacityKWh} onChange={(e) => setForm({ ...form, ca-
capacityKWh: e.target.value }}} placeholder="75" required />
      </Field>
      <Field label="Total battery slots" required>
        <Input type="number" value={form.totalBatterySlots} onChange={(e) => setForm({ ...form, total-
BatterySlots: e.target.value }}} placeholder="12" required />
      </Field>
    </div>

    <div className="rounded-xl border border-slate-200 bg-slate-50/60 p-4">
      <p className="mb-3 flex items-center gap-1.5 text-[13px] font-semibold text-slate-700">
        <BatteryIcon className="h-4 w-4 text-slate-400" />
        Operating schedule
      </p>
      <div className="space-y-2">
        {form.schedule.map((row, i) => (
          <div key={i} className="flex items-center gap-2">
            <Select value={row.day} onChange={(e) => updateScheduleRow(i, "day", e.target.value)} class-
Name="h-10 flex-1 text-[13px]">
              {DAYS.map((d) => <option key={d} value={d}>{d}</option>)}
            </Select>
            <Input type="time" value={row.openTime} onChange={(e) => updateScheduleRow(i, "openTime",
e.target.value)} className="h-10 w-[120px] px-2.5 text-[13px]" />
            <Input type="time" value={row.closeTime} onChange={(e) => updateScheduleRow(i, "closeTime",
e.target.value)} className="h-10 w-[120px] px-2.5 text-[13px]" />
            <button
              type="button"
              onClick={() => removeScheduleRow(i)}
              disabled={form.schedule.length === 1}
              className="rounded-lg p-2 text-slate-400 transition-colors hover:bg-rose-50 hover:text-
rose-600 disabled:opacity-30 disabled:hover:bg-transparent disabled:hover:text-slate-400"
              aria-label="Remove row"
            >
              <XIcon className="h-4 w-4" />
            </button>
          </div>
        ))}
      </div>
      <button type="button" onClick={addScheduleRow} className="mt-3 inline-flex items-center gap-1.5
text-[13px] font-semibold text-brand-700 hover:text-brand-800">
        <PlusIcon className="h-3.5 w-3.5" /> Add another day
      </button>
    </div>
  </form>
</Drawer>

  {/* Deactivation confirmation */}
  <Modal
    open={Boolean(confirm)}
    onClose={() => setConfirm(null)}
    icon={AlertIcon}
    tone="danger"
    title="Deactivate this node?"
    description={` ${confirm?.stationName} will stop accepting new bookings. The API refuses this if the
node still has pending or approved reservations.`}
    footer={
      <>
        <Button variant="subtle" onClick={() => setConfirm(null)}>Cancel</Button>
        <Button variant="danger" onClick={applyDeactivate}>Deactivate</Button>
      </>
    }
  >

```

```

    </>
  }
  </div>
);
}

```

web/src/pages/backoffice/UsersPage.jsx

```

// -----
// File: UsersPage.jsx
// Purpose: Backoffice screen to create Backoffice/Grid Operator staff accounts
//          and to deactivate or reactivate existing ones.
// Module owner: Member A (Identity & Access)
// -----
import { useEffect, useState } from "react";
import apiClient from "../../api/client";
import StatCard from "../../components/ui/StatCard";
import { Card, CardHeader } from "../../components/ui/Card";
import DataTable from "../../components/ui/DataTable";
import Button from "../../components/ui/Button";
import { Field, Input, Select } from "../../components/ui/Field";
import Badge, { StatusBadge } from "../../components/ui/Badge";
import Modal from "../../components/ui/Modal";
import Drawer from "../../components/ui/Drawer";
import { useToast } from "../../components/ui/Toast";
import { AlertIcon, GaugeIcon, PlusIcon, ShieldIcon, UsersIcon } from "../../components/ui/Icons";

const EMPTY_FORM = { userType: "GridOperator", fullName: "", email: "", phone: "", username: "", password: "" };

// Returns a person's initials for the avatar chip.
function initials(name = "") {
  return name.split(" ").filter(Boolean).slice(0, 2).map((w) => w[0]).join("").toUpperCase();
}

// Renders the staff directory and the create-account dialog.
export default function UsersPage() {
  const [users, setUsers] = useState([]);
  const [form, setForm] = useState(EMPTY_FORM);
  const [loading, setLoading] = useState(true);
  const [saving, setSaving] = useState(false);
  const [formOpen, setFormOpen] = useState(false);
  const [confirm, setConfirm] = useState(null);
  const toast = useToast();

  // Loads all Backoffice/GridOperator accounts.
  async function loadUsers() {
    setLoading(true);
    try {
      const { data } = await apiClient.get("/users");
      setUsers(data);
    } finally {
      setLoading(false);
    }
  }

  useEffect(() => { loadUsers(); }, []);

  // Creates a staff account from the dialog form.
  async function handleCreate(e) {
    e.preventDefault();
    setSaving(true);
    try {
      await apiClient.post("/users", form);
      toast.success(`${form.fullName} added as ${form.userType}.`);
      setForm(EMPTY_FORM);
      setFormOpen(false);
      await loadUsers();
    } catch (err) {
      toast.error(err.response?.data?.message || "Could not create this account.");
    } finally {
      setSaving(false);
    }
  }

```

```

    }
  }

  // Applies the deactivate/reactivate action confirmed in the dialog.
  async function applyStatusChange() {
    const user = confirm;
    const action = user.status === "Deactivated" ? "reactivate" : "deactivate";
    setConfirm(null);
    try {
      await apiClient.put(`/users/${user.id}/${action}`);
      toast.success(`${user.fullName} ${action === "reactivate" ? "reactivated" : "deactivated"}.`);
      await loadUsers();
    } catch (err) {
      toast.error(err.response?.data?.message || "Could not update this account.");
    }
  }

  const columns = [
    {
      key: "fullName",
      header: "Staff member",
      className: "min-w-[260px]",
      render: (u) => (
        <div className="flex items-center gap-3">
          <span className="grid h-10 w-10 shrink-0 place-items-center rounded-xl bg-gradient-to-br from-brand-500 to-brand-700 text-xs font-bold text-white">
            {initials(u.fullName)}
          </span>
          <div className="min-w-0">
            <p className="truncate font-semibold text-slate-900">{u.fullName}</p>
            <p className="truncate text-xs text-slate-500">{u.email}</p>
          </div>
        </div>
      ),
    },
    {
      key: "userType",
      header: "Role",
      className: "whitespace-nowrap",
      render: (u) => (
        <Badge tone={u.userType === "Backoffice" ? "sky" : "emerald"}>
          {u.userType === "Backoffice" ? <ShieldIcon className="h-3 w-3" /> : <GaugeIcon className="h-3 w-3" /> }
        </Badge>
      ),
    },
    { key: "username", header: "Username", render: (u) => <span className="font-mono text-[13px] text-slate-600">{u.username}</span> },
    { key: "phone", header: "Phone", className: "whitespace-nowrap", render: (u) => u.phone || "-" },
    { key: "status", header: "Status", render: (u) => <StatusBadge status={u.status} /> },
    {
      key: "actions",
      header: "",
      className: "text-right whitespace-nowrap",
      render: (u) => (
        <Button size="sm" variant={u.status === "Deactivated" ? "subtle" : "dangerGhost"} onClick={() => setConfirm(u)}>
          {u.status === "Deactivated" ? "Reactivate" : "Deactivate"}
        </Button>
      ),
    },
  ];

  const backoffice = users.filter((u) => u.userType === "Backoffice").length;
  const operators = users.filter((u) => u.userType === "GridOperator").length;
  const deactivated = users.filter((u) => u.status === "Deactivated").length;

  return (
    <div className="space-y-6">
      <section className="grid gap-5 sm:grid-cols-3">
        <StatCard label="Backoffice officers" value={backoffice} hint="Full administrative access" icon={ShieldIcon} tone="sky" loading={loading} />
        <StatCard label="Grid operators" value={operators} hint="Web console + mobile scanner" icon={GaugeIcon} tone="emerald" loading={loading} />
      </section>
    </div>
  );

```

```

        <StatCard label="Deactivated" value={deactivated} hint="No longer able to sign in" icon={AlertIcon}
tone="slate" loading={loading} />
    </section>

    <Card>
        <CardHeader
            icon={UsersIcon}
            title="Staff directory"
            description="Backoffice and Grid Operator accounts with console access."
            actions={<Button icon={PlusIcon} onClick={() => setFormOpen(true)}>New staff account</Button>}
        />
        <DataTable
            columns={columns}
            rows={users}
            loading={loading}
            empty={{
                icon: UsersIcon,
                title: "No staff accounts yet",
                description: "Create the first Backoffice or Grid Operator account to give your team console ac-
cess.",
                action: <Button icon={PlusIcon} onClick={() => setFormOpen(true)}>New staff account</Button>,
            }}
        />
    </Card>

    {/* Create panel */}
    <Drawer
        open={formOpen}
        onClose={() => setFormOpen(false)}
        size="lg"
        icon={UsersIcon}
        title="New staff account"
        description="The account is created active and can sign in immediately."
        footer={
            <>
                <Button variant="subtle" onClick={() => setFormOpen(false)}>Cancel</Button>
                <Button type="submit" form="user-form" icon={PlusIcon} disabled={saving}>
                    {saving ? "Creating..." : "Create account"}
                </Button>
            </>
        }
    >
        <form id="user-form" onSubmit={handleCreate} className="space-y-4 pb-2">
            <Field label="Role" required>
                <Select value={form.userType} onChange={(e) => setForm({ ...form, userType: e.target.value })}>
                    <option value="GridOperator">Grid Operator (operations & QR scanning)</option>
                    <option value="Backoffice">Backoffice (full administration)</option>
                </Select>
            </Field>

            <div className="grid gap-4 sm:grid-cols-2">
                <Field label="Full name" required>
                    <Input value={form.fullName} onChange={(e) => setForm({ ...form, fullName: e.target.value })}
placeholder="Kasun Silva" required />
                </Field>
                <Field label="Email" required>
                    <Input type="email" value={form.email} onChange={(e) => setForm({ ...form, email: e.target.value
})} placeholder="kasun@solarmicrogrid.lk" required />
                </Field>
                <Field label="Phone" required>
                    <Input value={form.phone} onChange={(e) => setForm({ ...form, phone: e.target.value })} place-
holder="0771112222" required />
                </Field>
                <Field label="Username" required>
                    <Input value={form.username} onChange={(e) => setForm({ ...form, username: e.target.value })}
placeholder="operator1" required />
                </Field>
            </div>

            <Field label="Password" required hint="Share this with the staff member securely. They can sign in
right away.">
                <Input type="password" value={form.password} onChange={(e) => setForm({ ...form, password: e.tar-
get.value })} placeholder="....." required />
            </Field>
        </form>
    </Drawer>

```

```

    /* Status change confirmation */
    <Modal
      open={Boolean(confirm)}
      onClose={() => setConfirm(null)}
      icon={AlertIcon}
      tone={confirm?.status === "Deactivated" ? "brand" : "danger"}
      title={confirm?.status === "Deactivated" ? "Reactivate account?" : "Deactivate account?"}
      description={
        confirm?.status === "Deactivated"
          ? `${confirm?.fullName} will be able to sign in again immediately.`
          : `${confirm?.fullName} will lose console access until a Backoffice officer reactivates the ac-
count.`
      }
      footer={
        <>
          <Button variant="subtle" onClick={() => setConfirm(null)}>Cancel</Button>
          <Button variant={confirm?.status === "Deactivated" ? "primary" : "danger"} onClick={applySta-
tusChange}>
            {confirm?.status === "Deactivated" ? "Reactivate" : "Deactivate"}
          </Button>
        </>
      }
    />
  </div>
);
}

```

web/src/pages/operator/OperatorDashboardPage.jsx

```

// -----
// File: OperatorDashboardPage.jsx
// Purpose: Grid Operator console – live pending/approved counters, an inline
//          battery-slot availability editor per node, and the queue of
//          reservations awaiting approval.
// Module owner: Member D (Grid Operator Ops & Service Integration)
// -----
import { useEffect, useState } from "react";
import apiClient from "../../api/client";
import StatCard from "../../components/ui/StatCard";
import { Card, CardHeader } from "../../components/ui/Card";
import DataTable from "../../components/ui/DataTable";
import Button from "../../components/ui/Button";
import { Input } from "../../components/ui/Field";
import { StatusBadge } from "../../components/ui/Badge";
import { useToast } from "../../components/ui/Toast";
import {
  BatteryIcon, CalendarIcon, CheckIcon, ClockIcon, GaugeIcon, StationIcon,
} from "../../components/ui/Icons";

// Renders the operator's counters, battery editor and approval queue.
export default function OperatorDashboardPage() {
  const [dashboard, setDashboard] = useState(null);
  const [stations, setStations] = useState([]);
  const [loading, setLoading] = useState(true);
  const [savingId, setSavingId] = useState(null);
  const toast = useToast();

  // Loads the dashboard summary and the station list for the battery editor.
  async function loadData() {
    setLoading(true);
    try {
      const [dashboardRes, stationsRes] = await Promise.all([
        apiClient.get("/dashboard/operator"),
        apiClient.get("/stations"),
      ]);
      setDashboard(dashboardRes.data);
      setStations(stationsRes.data);
    } finally {
      setLoading(false);
    }
  }

  useEffect(() => { loadData(); }, []);

```

```

// Persists a new available-battery-slot count for one station.
async function handleBatteryUpdate(station, newAvailable) {
  if (Number.isNaN(newAvailable) || newAvailable === station.availableBatterySlots) return;
  setSavingId(station.id);
  try {
    await apiClient.put(`/stations/${station.id}`, {
      stationName: station.stationName,
      lat: station.lat,
      lng: station.lng,
      capacityKWh: station.capacityKWh,
      totalBatterySlots: station.totalBatterySlots,
      availableBatterySlots: newAvailable,
      schedule: station.schedule,
    });
    toast.success(`${station.stationName} updated to ${newAvailable} free slots.`);
    await loadData();
  } catch (err) {
    toast.error(err.response?.data?.message || "Could not update battery availability.");
  } finally {
    setSavingId(null);
  }
}

// Approves a pending reservation straight from the queue.
async function handleApprove(reservation) {
  try {
    await apiClient.put(`/reservations/${reservation.id}/approve`);
    toast.success(`${reservation.reservationCode} approved, QR issued.`);
    await loadData();
  } catch (err) {
    toast.error(err.response?.data?.message || "Could not approve this reservation.");
  }
}

const batteryColumns = [
  {
    key: "stationName",
    header: "Node",
    render: (s) => (
      <div className="flex items-center gap-3">
        <span className="grid h-9 w-9 shrink-0 place-items-center rounded-lg bg-brand-50 text-brand-700 ring-1 ring-inset ring-brand-100">
          <StationIcon className="h-[18px] w-[18px]" />
        </span>
        <div>
          <p className="font-semibold text-slate-900">{s.stationName}</p>
          <p className="text-xs text-slate-500">{s.capacityKWh} kWh capacity</p>
        </div>
      </div>
    ),
  },
  {
    key: "availability",
    header: "Availability",
    render: (s) => {
      const pct = s.totalBatterySlots ? (s.availableBatterySlots / s.totalBatterySlots) * 100 : 0;
      return (
        <div className="w-40">
          <div className="mb-1.5 flex items-center justify-between text-xs">
            <span className="font-semibold text-slate-700">{s.availableBatterySlots} free</span>
            <span className="text-slate-400">of {s.totalBatterySlots}</span>
          </div>
          <div className="h-1.5 overflow-hidden rounded-full bg-slate-100">
            <div
              className={`h-full rounded-full bg-gradient-to-r ${pct > 50 ? "from-brand-400 to-brand-600" :
pct > 20 ? "from-solar-300 to-solar-500" : "from-rose-400 to-rose-600"}`}
              style={{ width: `${pct}%` }}
            />
          </div>
        </div>
      );
    },
  },
  { key: "status", header: "Status", render: (s) => <StatusBadge status={s.status} /> },
  {
    key: "update",

```

```

header: "Set free slots",
className: "text-right",
render: (s) => (
  <div className="flex items-center justify-end gap-2">
    <Input
      type="number"
      min={0}
      max={s.totalBatterySlots}
      defaultValue={s.availableBatterySlots}
      disabled={savingId === s.id}
      onBlur={(e) => handleBatteryUpdate(s, parseInt(e.target.value, 10))}
      className="h-10 w-24 text-center"
    />
    {savingId === s.id && <span className="text-xs text-slate-400">saving...</span>}
  </div>
),
},
];

const queueColumns = [
  {
    key: "reservationCode",
    header: "Reservation",
    render: (r) => (
      <div>
        <p className="font-mono text-[13px] font-bold text-slate-900">{r.reservationCode}</p>
        <p className="text-xs text-slate-500">NIC {r.prosumerNic}</p>
      </div>
    ),
  },
  {
    key: "scheduledDateTime",
    header: "Scheduled",
    render: (r) => (
      <div className="flex items-center gap-2">
        <ClockIcon className="h-4 w-4 text-slate-400" />
        {new Date(r.scheduledDateTime).toLocaleString(undefined, { dateStyle: "medium", timeStyle: "short"
      </div>
    ),
  },
  { key: "energyAmountKWh", header: "Energy", render: (r) => <span className="font-semibold text-slate-800">{r.energyAmountKWh} kWh</span> },
  {
    key: "actions",
    header: "",
    className: "text-right",
    render: (r) => <Button size="sm" icon={CheckIcon} onClick={() => handleApprove(r)}>Approve</Button>,
  },
];

const totalFree = stations.reduce((sum, s) => sum + s.availableBatterySlots, 0);
const totalSlots = stations.reduce((sum, s) => sum + s.totalBatterySlots, 0);

return (
  <div className="space-y-6">
    <section className="grid gap-5 sm:grid-cols-2 xl:grid-cols-4">
      <StatCard label="Pending reservations" value={dashboard?.pendingReservationsCount ?? 0} hint="Waiting on your approval" icon={ClockIcon} tone="amber" loading={loading} />
      <StatCard label="Approved (future)" value={dashboard?.approvedFutureReservationsCount ?? 0} hint="Confirmed upcoming transfers" icon={CalendarIcon} tone="emerald" loading={loading} />
      <StatCard label="Battery slots free" value={totalFree} hint={`of ${totalSlots} across all nodes`} icon={BatteryIcon} tone="sky" loading={loading} />
      <StatCard label="Active nodes" value={stations.filter((s) => s.status === "Active").length} hint={` ${stations.length} registered in total`} icon={StationIcon} tone="emerald" loading={loading} />
    </section>

    <Card>
      <CardHeader icon={GaugeIcon} title="Battery slot availability" description="Update free slot counts as batteries are swapped in and out at each node." />
      <DataTable
        columns={batteryColumns}
        rows={stations}
        loading={loading}
        empty={{ icon: StationIcon, title: "No nodes registered", description: "Ask a Backoffice officer to register a microgrid node first." }}
      </DataTable>
    </Card>
  </div>
);

```

```

        />
    </Card>

    <Card>
        <CardHeader
            icon={CalendarIcon}
            title="Reservations awaiting approval"
            description="Approving a booking issues the prosumer's transaction QR code."
            actions={
                !loading && dashboard?.pendingReservations?.length > 0 && (
                    <span className="rounded-full bg-solar-50 px-3 py-1.5 text-xs font-bold text-solar-600 ring-1
ring-inset ring-solar-200">
                        {dashboard.pendingReservations.length} pending
                    </span>
                )
            }
        />
        <DataTable
            columns={queueColumns}
            rows={dashboard?.pendingReservations || []}
            loading={loading}
            empty={{ icon: CheckIcon, title: "Queue is clear", description: "Every booking has been actioned.
Nothing is waiting for approval." }}
        />
    </Card>
</div>
);
}

```

Appendix C: Mobile Application Source Code (Kotlin)

25 Kotlin files, plus AndroidManifest.xml and 12 layout XML files.

mobile/app/src/main/AndroidManifest.xml

```

<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <uses-permission android:name="android.permission.INTERNET" />
    <uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
    <uses-permission android:name="android.permission.CAMERA" />
    <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
    <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
    <uses-feature android:name="android.hardware.camera" android:required="false" />

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher"
        android:theme="@style/Theme.SolarMicrogrid"
        android:usesCleartextTraffic="true">

        <!-- Replace with a real key before using MapActivity / running on a device with Play Services. -->
        <meta-data
            android:name="com.google.android.geo.API_KEY"
            android:value="@string/google_maps_api_key" />

        <activity
            android:name=".auth.LoginActivity"
            android:exported="true"
            android:theme="@style/Theme.SolarMicrogrid.Dark">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

        <activity android:name=".auth.RegisterActivity" android:exported="false"
            android:theme="@style/Theme.SolarMicrogrid.Dark" />
    </application>

```

```

<activity android:name=".prosumer.ProsumerHomeActivity" android:exported="false"
    android:theme="@style/Theme.SolarMicrogrid.Dark" />
<activity android:name=".prosumer.MapActivity" android:exported="false" />
<activity android:name=".prosumer.HistoryActivity" android:exported="false"
    android:theme="@style/Theme.SolarMicrogrid.Dark" />
<activity android:name=".prosumer.ProfileActivity" android:exported="false"
    android:theme="@style/Theme.SolarMicrogrid.Dark" />
<activity android:name=".prosumer.ReservationFormActivity" android:exported="false"
    android:theme="@style/Theme.SolarMicrogrid.Dark" />
<activity android:name=".prosumer.ReservationDetailActivity" android:exported="false"
    android:theme="@style/Theme.SolarMicrogrid.Dark" />

<activity android:name=".operator.OperatorHomeActivity" android:exported="false"
    android:theme="@style/Theme.SolarMicrogrid.Dark" />
<activity android:name=".operator.ScanActivity" android:exported="false"
    android:theme="@style/Theme.SolarMicrogrid.Dark" />
<activity android:name=".operator.TransferConfirmActivity" android:exported="false"
    android:theme="@style/Theme.SolarMicrogrid.Dark" />

</application>
</manifest>

```

mobile/app/src/main/java/com/solarmicrogrid/app/auth/LoginActivity.kt

```

// -----
// File: LoginActivity.kt
// Purpose: App entry point. Skips straight to the role-appropriate home
//          screen if a session is already saved in SQLite; otherwise
//          authenticates against the API and routes by returned userType.
// Module owner: Member A (Identity & Access)
// -----
package com.solarmicrogrid.app.auth

import android.content.Intent
import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity
import androidx.core.view.WindowCompat
import androidx.lifecycle.LifecycleScope
import com.solarmicrogrid.app.data.local.SessionManager
import com.solarmicrogrid.app.data.model.LoginRequest
import com.solarmicrogrid.app.data.remote.RetrofitClient
import com.solarmicrogrid.app.databinding.ActivityLoginBinding
import com.solarmicrogrid.app.operator.OperatorHomeActivity
import com.solarmicrogrid.app.prosumer.ProsumerHomeActivity
import com.solarmicrogrid.app.util.errorMessageOrDefault
import kotlinx.coroutines.launch

class LoginActivity : AppCompatActivity() {

    private lateinit var binding: ActivityLoginBinding
    private lateinit var sessionManager: SessionManager

    // Sets up bindings, redirects if already logged in, and wires the form.
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityLoginBinding.inflate(layoutInflater)
        setContentView(binding.root)
        WindowCompat.setDecorFitsSystemWindows(window, false)
        sessionManager = SessionManager(this)

        sessionManager.current()?.let { session ->
            routeToHome(session.userType)
            return
        }

        binding.buttonLogin.setOnClickListener { attemptLogin() }
        binding.textGoRegister.setOnClickListener {
            startActivity(Intent(this, RegisterActivity::class.java))
        }
    }
}

```

```

// Submits the identifier/password to the API and saves the session on success.
private fun attemptLogin() {
    val identifier = binding.inputIdentifier.text.toString().trim()
    val password = binding.inputPassword.text.toString()
    if (identifier.isEmpty() || password.isEmpty()) {
        showError("Please enter both fields.")
        return
    }

    binding.buttonLogin.isEnabled = false
    lifecycleScope.launch {
        try {
            val response = RetrofitClient.getApiService(this@LoginActivity).login(LoginRequest(identifier,
password))
            if (response.isSuccessful && response.body() != null) {
                val login = response.body()!!
                sessionManager.save(login)
                routeToHome(login.userType)
            } else {
                showError(response.errorMessageOrDefault("Invalid credentials."))
            }
        } catch (e: Exception) {
            showError("Could not reach the server. Check your connection.")
        } finally {
            binding.buttonLogin.isEnabled = true
        }
    }
}

// Navigates to the Prosumer or Grid Operator home screen based on role.
private fun routeToHome(userType: String) {
    val destination = if (userType == "Prosumer") ProsumerHomeActivity::class.java else OperatorHomeActiv-
ity::class.java
    startActivity(Intent(this, destination))
    finish()
}

// Shows the inline error banner above the form.
private fun showError(message: String) {
    binding.textError.text = message
    binding.errorBanner.visibility = android.view.View.VISIBLE
}
}

```

mobile/app/src/main/java/com/solarmicrogrid/app/auth/RegisterActivity.kt

```

// -----
// File: RegisterActivity.kt
// Purpose: Prosumer self-registration screen – NIC as primary key, submits
//          to the API and starts the account in PendingActivation until a
//          Backoffice officer approves it from the web app.
// Module owner: Member A (Identity & Access)
// -----
package com.solarmicrogrid.app.auth

import android.os.Bundle
import android.view.View
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
import androidx.lifecycle.Lifecycle
import androidx.lifecycle.LifecycleScope
import com.solarmicrogrid.app.data.model.RegisterProsumerRequest
import com.solarmicrogrid.app.data.remote.RetrofitClient
import com.solarmicrogrid.app.databinding.ActivityRegisterBinding
import com.solarmicrogrid.app.util.applyEdgeToEdgeInsets
import com.solarmicrogrid.app.util.errorMessageOrDefault
import kotlinx.coroutines.launch

class RegisterActivity : AppCompatActivity() {

    private lateinit var binding: ActivityRegisterBinding
}

```

```

// Wires the registration form submit action.
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    binding = ActivityRegisterBinding.inflate(layoutInflater)
    setContentView(binding.root)
    applyEdgeToEdgeInsets(topInsetView = binding.heroSection)

    binding.buttonRegister.setOnClickListener { submitRegistration() }
}

// Validates the form and posts the registration request to the API.
private fun submitRegistration() {
    val nic = binding.inputNic.text.toString().trim()
    val fullName = binding.inputFullName.text.toString().trim()
    val email = binding.inputEmail.text.toString().trim()
    val phone = binding.inputPhone.text.toString().trim()
    val address = binding.inputAddress.text.toString().trim()
    val password = binding.inputPassword.text.toString()

    if (listOf(nic, fullName, email, phone, address, password).any { it.isEmpty() }) {
        showError("Please fill in every field.")
        return
    }

    binding.buttonRegister.isEnabled = false
    lifecycleScope.launch {
        try {
            val response = RetrofitClient.getApiService(this@RegisterActivity)
                .registerProsumer(RegisterProsumerRequest(nic, fullName, email, phone, address, password))
            if (response.isSuccessful) {
                Toast.makeText(
                    this@RegisterActivity,
                    "Registered! Your account is pending Backoffice activation.",
                    Toast.LENGTH_LONG
                ).show()
                finish()
            } else {
                showError(response.errorMessageOrDefault("Registration failed."))
            }
        } catch (e: Exception) {
            showError("Could not reach the server. Check your connection.")
        } finally {
            binding.buttonRegister.isEnabled = true
        }
    }
}

// Shows the inline error banner above the form.
private fun showError(message: String) {
    binding.textError.text = message
    binding.errorBanner.visibility = View.VISIBLE
}
}

```

mobile/app/src/main/java/com/solarmicrogrid/app/data/local/DbHelper.kt

```

// -----
// File: DbHelper.kt
// Purpose: SQLiteOpenHelper defining the local database schema – a
//          single-row session table (JWT + logged-in profile) and a
//          station cache table for offline browsing of the map screen.
//          This is local persistence only; MongoDB via the API remains
//          the single source of truth for every business record.
// Module owner: Member D (Service Integration, Local Persistence)
// -----
package com.solarmicrogrid.app.data.local

import android.content.Context
import android.database.sqlite.SQLiteDatabase
import android.database.sqlite.SQLiteOpenHelper

class DbHelper(context: Context) : SQLiteOpenHelper(context, DATABASE_NAME, null, DATABASE_VERSION) {

```

```

companion object {
    private const val DATABASE_NAME = "solar_microgrid_local.db"
    private const val DATABASE_VERSION = 1

    const val TABLE_SESSION = "session"
    const val COL_TOKEN = "token"
    const val COL_USER_ID = "user_id"
    const val COL_USER_TYPE = "user_type"
    const val COL_FULL_NAME = "full_name"
    const val COL_NIC = "nic"

    const val TABLE_STATION_CACHE = "station_cache"
    const val COL_STATION_ID = "id"
    const val COL_STATION_NAME = "station_name"
    const val COL_LAT = "lat"
    const val COL_LNG = "lng"
    const val COL_AVAILABLE_SLOTS = "available_slots"
    const val COL_TOTAL_SLOTS = "total_slots"
}

// Creates the session and station-cache tables on first run.
override fun onCreate(db: SQLiteDatabase) {
    db.execSQL(
        """
        CREATE TABLE $TABLE_SESSION (
            $COL_TOKEN TEXT PRIMARY KEY,
            $COL_USER_ID TEXT NOT NULL,
            $COL_USER_TYPE TEXT NOT NULL,
            $COL_FULL_NAME TEXT NOT NULL,
            $COL_NIC TEXT
        )
        """
    )
    db.execSQL(
        """
        CREATE TABLE $TABLE_STATION_CACHE (
            $COL_STATION_ID TEXT PRIMARY KEY,
            $COL_STATION_NAME TEXT NOT NULL,
            $COL_LAT REAL NOT NULL,
            $COL_LNG REAL NOT NULL,
            $COL_AVAILABLE_SLOTS INTEGER NOT NULL,
            $COL_TOTAL_SLOTS INTEGER NOT NULL
        )
        """
    )
}

// Drops and recreates both tables on a schema version bump.
override fun onUpgrade(db: SQLiteDatabase, oldVersion: Int, newVersion: Int) {
    db.execSQL("DROP TABLE IF EXISTS $TABLE_SESSION")
    db.execSQL("DROP TABLE IF EXISTS $TABLE_STATION_CACHE")
    onCreate(db)
}
}

```

mobile/app/src/main/java/com/solarmicrogrid/app/data/local/SessionManager.kt

```

// -----
// File: SessionManager.kt
// Purpose: Reads/writes the single logged-in session row in SQLite, so the
//          JWT and profile survive an app restart without re-authenticating
//          against the API every launch.
// Module owner: Member D (Service Integration, Local Persistence)
// -----
package com.solarmicrogrid.app.data.local

import android.content.ContentValues
import android.content.Context
import com.solarmicrogrid.app.data.model.LoginResponse

data class Session(
    val token: String,

```

```

        val userId: String,
        val userType: String,
        val fullName: String,
        val nic: String?
    )

class SessionManager(context: Context) {

    private val dbHelper = DbHelper(context.applicationContext)

    // Persists the freshly-issued login response as the (single) active session.
    fun save(login: LoginResponse) {
        val db = dbHelper.writableDatabase
        db.delete(DbHelper.TABLE_SESSION, null, null)
        val values = ContentValues().apply {
            put(DbHelper.COL_TOKEN, login.token)
            put(DbHelper.COL_USER_ID, login.userId)
            put(DbHelper.COL_USER_TYPE, login.userType)
            put(DbHelper.COL_FULL_NAME, login.fullName)
            put(DbHelper.COL_NIC, login.nic)
        }
        db.insert(DbHelper.TABLE_SESSION, null, values)
    }

    // Reads the currently saved session, or null if nobody is logged in.
    fun current(): Session? {
        val db = dbHelper.readableDatabase
        db.query(DbHelper.TABLE_SESSION, null, null, null, null, null, null).use { cursor ->
            if (!cursor.moveToFirst()) return null
            return Session(
                token = cursor.getString(cursor.getColumnIndexOrThrow(DbHelper.COL_TOKEN)),
                userId = cursor.getString(cursor.getColumnIndexOrThrow(DbHelper.COL_USER_ID)),
                userType = cursor.getString(cursor.getColumnIndexOrThrow(DbHelper.COL_USER_TYPE)),
                fullName = cursor.getString(cursor.getColumnIndexOrThrow(DbHelper.COL_FULL_NAME)),
                nic = cursor.getString(cursor.getColumnIndexOrThrow(DbHelper.COL_NIC))
            )
        }
    }

    // Clears the session row on logout.
    fun clear() {
        dbHelper.writableDatabase.delete(DbHelper.TABLE_SESSION, null, null)
    }
}

```

mobile/app/src/main/java/com/solarmicrogrid/app/data/local/StationCacheDao.kt

```

// -----
// File: StationCacheDao.kt
// Purpose: Caches the last-fetched nearby-station list in SQLite so the map
//          screen has something to show immediately (and offline) before
//          the live API response arrives.
// Module owner: Member B (Microgrid Nodes & Maps)
// -----
package com.solarmicrogrid.app.data.local

import android.content.ContentValues
import android.content.Context
import com.solarmicrogrid.app.data.model.StationResponse

data class CachedStation(
    val id: String,
    val stationName: String,
    val lat: Double,
    val lng: Double,
    val availableSlots: Int,
    val totalSlots: Int
)

class StationCacheDao(context: Context) {

    private val dbHelper = DbHelper(context.applicationContext)
}

```

```

// Replaces the cached station list with the latest data fetched from the API.
fun replaceAll(stations: List<StationResponse>) {
    val db = dbHelper.writableDatabase
    db.beginTransaction()
    try {
        db.delete(DbHelper.TABLE_STATION_CACHE, null, null)
        stations.forEach { station ->
            val values = ContentValues().apply {
                put(DbHelper.COL_STATION_ID, station.id)
                put(DbHelper.COL_STATION_NAME, station.stationName)
                put(DbHelper.COL_LAT, station.lat)
                put(DbHelper.COL_LNG, station.lng)
                put(DbHelper.COL_AVAILABLE_SLOTS, station.availableBatterySlots)
                put(DbHelper.COL_TOTAL_SLOTS, station.totalBatterySlots)
            }
            db.insert(DbHelper.TABLE_STATION_CACHE, null, values)
        }
        db.setTransactionSuccessful()
    } finally {
        db.endTransaction()
    }
}

// Reads whatever was last cached, for offline/instant display.
fun getAll(): List<CachedStation> {
    val db = dbHelper.readableDatabase
    val result = mutableListOf<CachedStation>()
    db.query(DbHelper.TABLE_STATION_CACHE, null, null, null, null, null, null).use { cursor ->
        while (cursor.moveToNext()) {
            result.add(
                CachedStation(
                    id = cursor.getString(cursor.getColumnIndexOrThrow(DbHelper.COL_STATION_ID)),
                    stationName = cursor.getString(cursor.getColumnIndexOrThrow(DbHelper.COL_STATION_NAME)),
                    lat = cursor.getDouble(cursor.getColumnIndexOrThrow(DbHelper.COL_LAT)),
                    lng = cursor.getDouble(cursor.getColumnIndexOrThrow(DbHelper.COL_LNG)),
                    availableSlots = cursor.getInt(cursor.getColumnIndexOrThrow(DbHelper.COL_AVAILABLE_SLOTS)),
                    totalSlots = cursor.getInt(cursor.getColumnIndexOrThrow(DbHelper.COL_TOTAL_SLOTS))
                )
            )
        }
    }
    return result
}

```

mobile/app/src/main/java/com/solarmicrogrid/app/data/model/AuthModels.kt

```

// -----
// File: AuthModels.kt
// Purpose: Request/response data classes for the shared login endpoint and
//          the account-management payloads (register/update/error body),
//          mirroring the API's Dtos/AuthDtos.cs, ProsumerDtos.cs, UserDtos.cs.
// Module owner: Member A (Identity & Access)
// -----
package com.solarmicrogrid.app.data.model

data class LoginRequest(
    val identifier: String,
    val password: String
)

data class LoginResponse(
    val token: String,
    val userId: String,
    val userType: String,
    val fullName: String,
    val nic: String?
)

data class RegisterProsumerRequest(

```

```

        val nic: String,
        val fullName: String,
        val email: String,
        val phone: String,
        val address: String,
        val password: String
    )

    data class UpdateProsumerRequest(
        val fullName: String,
        val email: String,
        val phone: String,
        val address: String
    )

    data class UserResponse(
        val id: String,
        val userType: String,
        val fullName: String,
        val email: String,
        val phone: String,
        val username: String?,
        val nic: String?,
        val address: String?,
        val photoUrl: String?,
        val status: String,
        val createdAt: String
    )

    // Shape of the JSON error body written by the API's GlobalExceptionHandler.
    data class ApiErrorBody(
        val message: String?
    )

```

mobile/app/src/main/java/com/solarmicrogrid/app/data/model/ReservationModels.kt

```

// -----
// File: ReservationModels.kt
// Purpose: Request/response data classes for the reservation workflow (create,
//          update, cancel, approve, QR verify/complete) and the dashboard
//          aggregations, mirroring the API's ReservationDtos.cs.
// Module owner: Member C (Reservations & Booking Workflow) /
//               Member D (QR verification)
// -----
package com.solarmicrogrid.app.data.model

data class CreateReservationRequest(
    val prosumerNic: String? = null,
    val stationId: String,
    val slotId: String,
    val scheduledDateTime: String,
    val energyAmountKWh: Double
)

data class UpdateReservationRequest(
    val slotId: String,
    val scheduledDateTime: String,
    val energyAmountKWh: Double
)

data class CancelReservationRequest(
    val reason: String
)

data class VerifyQrRequest(
    val qrToken: String
)

data class ReservationResponse(
    val id: String,
    val reservationCode: String,
    val prosumerNic: String,

```

```

        val stationId: String,
        val slotId: String,
        val scheduledDateTime: String,
        val energyAmountKWh: Double,
        val status: String,
        val qrToken: String?,
        val createdAt: String
    )

    data class ProsumerDashboardResponse(
        val activeCount: Int,
        val pendingCount: Int,
        val upcomingReservations: List<ReservationResponse>
    )

    data class OperatorDashboardResponse(
        val pendingReservationsCount: Int,
        val approvedFutureReservationsCount: Int,
        val pendingReservations: List<ReservationResponse>
    )

```

mobile/app/src/main/java/com/solarmicrogrid/app/data/model/StationModels.kt

```

// -----
// File: StationModels.kt
// Purpose: Response data classes for microgrid nodes (stations) and their
//          bookable energy slots, mirroring the API's StationDtos.cs /
//          SlotDtos.cs.
// Module owner: Member B (Microgrid Nodes & Maps)
// -----
package com.solarmicrogrid.app.data.model

data class ScheduleEntry(
    val day: String,
    val openTime: String,
    val closeTime: String
)

data class StationResponse(
    val id: String,
    val stationName: String,
    val lat: Double,
    val lng: Double,
    val capacityKWh: Double,
    val totalBatterySlots: Int,
    val availableBatterySlots: Int,
    val schedule: List<ScheduleEntry>,
    val status: String
)

data class SlotResponse(
    val id: String,
    val stationId: String,
    val date: String,
    val startTime: String,
    val endTime: String,
    val slotType: String,
    val capacityTotal: Int,
    val capacityAvailable: Int,
    val status: String
)

```

mobile/app/src/main/java/com/solarmicrogrid/app/data/remote/ApiService.kt

```

// -----
// File: ApiService.kt
// Purpose: Retrofit interface declaring every REST call the mobile app makes
//          against the Web API. No business logic lives here or anywhere
//          else in the app – every rule (7-day window, 12-hour notice, QR
//          verification) is enforced server-side, per the FAT-service
//          architecture.

```

```

// Module owner: shared infrastructure
// -----
package com.solarmicrogrid.app.data.remote

import com.solarmicrogrid.app.data.model.*
import okhttp3.MultipartBody
import retrofit2.Response
import retrofit2.http.*

interface ApiService {

    @POST("auth/login")
    suspend fun login(@Body request: LoginRequest): Response<LoginResponse>

    @POST("prosumers/register")
    suspend fun registerProsumer(@Body request: RegisterProsumerRequest): Response<UserResponse>

    @GET("prosumers/{nic}")
    suspend fun getProsumer(@Path("nic") nic: String): Response<UserResponse>

    @PUT("prosumers/{nic}")
    suspend fun updateProsumer(@Path("nic") nic: String, @Body request: UpdateProsumerRequest): Response<UserResponse>

    @PUT("prosumers/{nic}/request-deactivation")
    suspend fun requestDeactivation(@Path("nic") nic: String): Response<UserResponse>

    @Multipart
    @POST("prosumers/{nic}/photo")
    suspend fun uploadProsumerPhoto(@Path("nic") nic: String, @Part file: MultipartBody.Part): Response<UserResponse>

    @GET("stations")
    suspend fun getStations(): Response<List<StationResponse>>

    @GET("stations/nearby")
    suspend fun getNearbyStations(
        @Query("lat") lat: Double,
        @Query("lng") lng: Double,
        @Query("radiusKm") radiusKm: Double
    ): Response<List<StationResponse>>

    @GET("stations/{stationId}/slots")
    suspend fun getSlots(@Path("stationId") stationId: String): Response<List<SlotResponse>>

    @GET("reservations/{id}")
    suspend fun getReservation(@Path("id") id: String): Response<ReservationResponse>

    @POST("reservations")
    suspend fun createReservation(@Body request: CreateReservationRequest): Response<ReservationResponse>

    @PUT("reservations/{id}")
    suspend fun updateReservation(@Path("id") id: String, @Body request: UpdateReservationRequest): Response<ReservationResponse>

    @PUT("reservations/{id}/cancel")
    suspend fun cancelReservation(@Path("id") id: String, @Body request: CancelReservationRequest): Response<ReservationResponse>

    @GET("reservations/prosumer/{nic}")
    suspend fun getReservationsByProsumer(@Path("nic") nic: String): Response<List<ReservationResponse>>

    @PUT("reservations/{id}/approve")
    suspend fun approveReservation(@Path("id") id: String): Response<ReservationResponse>

    @GET("dashboard/prosumer/{nic}")
    suspend fun getProsumerDashboard(@Path("nic") nic: String): Response<ProsumerDashboardResponse>

    @GET("dashboard/operator")
    suspend fun getOperatorDashboard(): Response<OperatorDashboardResponse>

```

```

@POST("reservations/verify-qr")
suspend fun verifyQr(@Body request: VerifyQrRequest): Response<ReservationResponse>

@PUT("reservations/{id}/complete")
suspend fun completeReservation(@Path("id") id: String): Response<ReservationResponse>
}

```

mobile/app/src/main/java/com/solarmicrogrid/app/data/remote/AuthInterceptor.kt

```

// -----
// File: AuthInterceptor.kt
// Purpose: OkHttp interceptor that attaches the saved JWT (if any) to every
//          outgoing request's Authorization header, so individual API
//          calls never have to handle auth headers themselves.
// Module owner: Member A (Identity & Access)
// -----
package com.solarmicrogrid.app.data.remote

import com.solarmicrogrid.app.data.local.SessionManager
import okhttp3.Interceptor
import okhttp3.Response

class AuthInterceptor(private val sessionManager: SessionManager) : Interceptor {

    // Adds "Authorization: Bearer <token>" to the request when a session exists.
    override fun intercept(chain: Interceptor.Chain): Response {
        val token = sessionManager.current()?.token
        val request = chain.request().newBuilder().apply {
            if (!token.isNullOrEmpty()) {
                addHeader("Authorization", "Bearer $token")
            }
        }.build()
        return chain.proceed(request)
    }
}

```

mobile/app/src/main/java/com/solarmicrogrid/app/data/remote/RetrofitClient.kt

```

// -----
// File: RetrofitClient.kt
// Purpose: Builds a single shared Retrofit/OkHttp instance (with the auth
//          interceptor and request logging) pointed at the Web API base URL.
// Module owner: shared infrastructure
// -----
package com.solarmicrogrid.app.data.remote

import android.content.Context
import com.solarmicrogrid.app.data.local.SessionManager
import com.solarmicrogrid.app.util.Constants
import okhttp3.OkHttpClient
import okhttp3.logging.HttpLoggingInterceptor
import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory

object RetrofitClient {

    @Volatile
    private var apiService: ApiService? = null

    // Returns the shared ApiService, building it on first use.
    fun getApiService(context: Context): ApiService {
        return apiService ?: synchronized(this) {
            apiService ?: build(context).also { apiService = it }
        }
    }

    // Assembles the OkHttp client (auth + logging) and the Retrofit/Gson instance.
    private fun build(context: Context): ApiService {

```

```

        val sessionManager = SessionManager(context.applicationContext)

        val logging = HttpLoggingInterceptor().apply { level = HttpLoggingInterceptor.Level.BASIC }

        val client = OkHttpClient.Builder()
            .addInterceptor(AuthInterceptor(sessionManager))
            .addInterceptor(logging)
            .build()

        val retrofit = Retrofit.Builder()
            .baseUrl(Constants.API_BASE_URL)
            .client(client)
            .addConverterFactory(GsonConverterFactory.create())
            .build()

        return retrofit.create(ApiService::class.java)
    }
}

```

mobile/app/src/main/java/com/solarmicrogrid/app/operator/OperatorHomeActivity.kt

```

// -----
// File: OperatorHomeActivity.kt
// Purpose: Grid Operator mobile dashboard – pending/approved-future counts
//          and the pending-reservation queue (tap a row to approve it),
//          plus the entry point into QR scanning mode.
// Module owner: Member D (Grid Operator Ops & Service Integration)
// -----
package com.solarmicrogrid.app.operator

import android.app.AlertDialog
import android.content.Intent
import android.os.Bundle
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
import androidx.lifecycle.lifecycleScope
import androidx.recyclerview.widget.LinearLayoutManager
import com.solarmicrogrid.app.auth.LoginActivity
import com.solarmicrogrid.app.data.local.SessionManager
import com.solarmicrogrid.app.data.remote.RetrofitClient
import com.solarmicrogrid.app.databinding.ActivityOperatorHomeBinding
import com.solarmicrogrid.app.prosumer.ReservationAdapter
import com.solarmicrogrid.app.util.applyEdgeToEdgeInsets
import com.solarmicrogrid.app.util.errorMessageOrDefault
import kotlinx.coroutines.launch

class OperatorHomeActivity : AppCompatActivity() {

    private lateinit var binding: ActivityOperatorHomeBinding
    private lateinit var sessionManager: SessionManager
    private lateinit var adapter: ReservationAdapter

    // Sets up the dashboard, its pending-reservations list and the scan/logout actions.
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityOperatorHomeBinding.inflate(layoutInflater)
        setContentView(binding.root)
        applyEdgeToEdgeInsets(topInsetView = binding.heroSection)
        sessionManager = SessionManager(this)

        val session = sessionManager.current() ?: return redirectToLogin()
        binding.textWelcome.text = session.fullName

        adapter = ReservationAdapter(emptyList()) { reservation -> confirmApprove(reservation.id, reservation.reservationCode) }
        binding.recyclerPending.layoutManager = LinearLayoutManager(this)
        binding.recyclerPending.adapter = adapter
        binding.recyclerPending.isNestedScrollingEnabled = false

        binding.swipeRefresh.setColorSchemeResources(com.solarmicrogrid.app.R.color.brand_600)
        binding.swipeRefresh.setOnRefreshListener { loadDashboard() }
    }
}

```

```

        binding.buttonScan.setOnClickListener { startActivity(Intent(this, ScanActivity::class.java)) }
        binding.buttonLogout.setOnClickListener { logout() }
    }

    // Refreshes the dashboard every time this screen regains focus.
    override fun onResume() {
        super.onResume()
        loadDashboard()
    }

    // Fetches the operator dashboard summary from the API.
    private fun loadDashboard() {
        binding.swipeRefresh.isRefreshing = true
        lifecycleScope.launch {
            try {
                val response = RetrofitClient.getApiService(this@OperatorHomeActivity).getOperatorDashboard()
                if (response.isSuccessful && response.body() != null) {
                    val dashboard = response.body()!!
                    binding.textPendingCount.text = dashboard.pendingReservationsCount.toString()
                    binding.textApprovedCount.text = dashboard.approvedFutureReservationsCount.toString()
                    adapter.submitList(dashboard.pendingReservations)
                    binding.emptyState.visibility =
                        if (dashboard.pendingReservations.isEmpty()) android.view.View.VISIBLE else an-
droid.view.View.GONE
                }
            } catch (e: Exception) {
                Toast.makeText(this@OperatorHomeActivity, "Could not reach the server. Check your connec-
tion.", Toast.LENGTH_SHORT).show()
            } finally {
                binding.swipeRefresh.isRefreshing = false
            }
        }
    }

    // Confirms and submits approval for a pending reservation (issues its QR token).
    private fun confirmApprove(reservationId: String, code: String) {
        AlertDialog.Builder(this)
            .setTitle("Approve Reservation")
            .setMessage("Approve $code? This issues its transaction QR code to the prosumer.")
            .setPositiveButton("Approve") { _, _ -> submitApprove(reservationId) }
            .setNegativeButton("Cancel", null)
            .show()
    }

    // Calls the approve endpoint and refreshes the dashboard on success.
    private fun submitApprove(reservationId: String) {
        lifecycleScope.launch {
            try {
                val response = RetrofitClient.getApiService(this@OperatorHomeActivity).approveReservation(res-
ervationId)
                if (response.isSuccessful) {
                    Toast.makeText(this@OperatorHomeActivity, "Reservation approved.",
Toast.LENGTH_SHORT).show()
                    loadDashboard()
                } else {
                    Toast.makeText(this@OperatorHomeActivity, response.errorMessageOrDefault("Approval
failed."), Toast.LENGTH_LONG).show()
                }
            } catch (e: Exception) {
                Toast.makeText(this@OperatorHomeActivity, "Could not reach the server. Check your connec-
tion.", Toast.LENGTH_SHORT).show()
            }
        }
    }

    // Clears the local session and returns to the login screen.
    private fun logout() {
        sessionManager.clear()
        redirectToLogin()
    }

    // Sends the user back to LoginActivity, clearing the back stack.
    private fun redirectToLogin() {
        val intent = Intent(this, LoginActivity::class.java)
        intent.flags = Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_CLEAR_TASK
    }

```

```

        startActivity(intent)
        finish()
    }
}

```

mobile/app/src/main/java/com/solarmicrogrid/app/operator/ScanActivity.kt

```

// -----
// File: ScanActivity.kt
// Purpose: Grid Operator camera screen – live-scans a prosumer's QR code
//           with CameraX + ML Kit, sends the decoded token to the API for
//           verification, and opens the transfer-confirmation screen once
//           the server confirms it matches an Approved reservation.
// Module owner: Member D (Grid Operator Verification)
// -----
package com.solarmicrogrid.app.operator

import android.Manifest
import android.content.Intent
import android.content.pm.PackageManager
import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity
import androidx.camera.core.CameraSelector
import androidx.camera.core.ImageAnalysis
import androidx.camera.core.ImageProxy
import androidx.camera.core.Preview
import androidx.camera.lifecycle.ProcessCameraProvider
import androidx.core.app.ActivityCompat
import androidx.core.content.ContextCompat
import androidx.lifecycle.lifecycleScope
import com.google.mlkit.vision.barcode.BarcodeScannerOptions
import com.google.mlkit.vision.barcode.BarcodeScanning
import com.google.mlkit.vision.barcode.common.Barcode
import com.google.mlkit.vision.common.InputImage
import com.solarmicrogrid.app.data.model.VerifyQrRequest
import com.solarmicrogrid.app.data.remote.RetrofitClient
import com.solarmicrogrid.app.databinding.ActivityScanBinding
import com.solarmicrogrid.app.util.Constants
import com.solarmicrogrid.app.util.errorMessageOrDefault
import kotlinx.coroutines.launch
import java.util.concurrent.Executors

class ScanActivity : AppCompatActivity() {

    private lateinit var binding: ActivityScanBinding
    private val cameraPermissionCode = 2001
    private var isProcessing = false

    // Requests camera permission (if needed) and starts the preview.
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityScanBinding.inflate(layoutInflater)
        setContentView(binding.root)

        if (ContextCompat.checkSelfPermission(this, Manifest.permission.CAMERA) == Pack-
ageManager.PERMISSION_GRANTED) {
            startCamera()
        } else {
            ActivityCompat.requestPermissions(this, arrayOf(Manifest.permission.CAMERA), cameraPermissionCode)
        }
    }

    // Reacts to the camera permission prompt's result.
    override fun onRequestPermissionsResult(requestCode: Int, permissions: Array<out String>, grantResults:
IntArray) {
        super.onRequestPermissionsResult(requestCode, permissions, grantResults)
        if (requestCode == cameraPermissionCode && grantResults.isNotEmpty() && grantResults[0] == Pack-
ageManager.PERMISSION_GRANTED) {
            startCamera()
        } else {
            binding.textStatus.text = "Camera permission is required to scan QR codes."
        }
    }
}

```

```

// Binds the CameraX preview + ML Kit barcode analyzer to this activity's lifecycle.
private fun startCamera() {
    val cameraProviderFuture = ProcessCameraProvider.getInstance(this)
    cameraProviderFuture.addListener({
        val cameraProvider = cameraProviderFuture.get()

        val preview = Preview.Builder().build().also {
            it.setSurfaceProvider(binding.previewView.surfaceProvider)
        }

        val scannerOptions = BarcodeScannerOptions.Builder()
            .setBarcodeFormats(Barcode.FORMAT_QR_CODE)
            .build()
        val scanner = BarcodeScanning.getClient(scannerOptions)

        val analysis = ImageAnalysis.Builder().build()
        analysis.setAnalyzer(Executors.newSingleThreadExecutor()) { imageProxy ->
            processFrame(imageProxy, scanner)
        }

        cameraProvider.unbindAll()
        cameraProvider.bindToLifecycle(this, CameraSelector.DEFAULT_BACK_CAMERA, preview, analysis)
    }, ContextCompat.getMainExecutor(this))
}

// Runs ML Kit barcode detection on one camera frame and hands off the first QR match.
@androidx.camera.core.ExperimentalGetImage
private fun processFrame(imageProxy: ImageProxy, scanner: com.google.mlkit.vision.barcode.BarcodeScanner)
{
    val mediaImage = imageProxy.image
    if (mediaImage == null || isProcessing) {
        imageProxy.close()
        return
    }

    val image = InputImage.fromMediaImage(mediaImage, imageProxy.imageInfo.rotationDegrees)
    scanner.process(image)
        .addOnSuccessListener { barcodes ->
            val token = barcodes.firstOrNull()?.rawValue
            if (token != null && !isProcessing) {
                isProcessing = true
                verifyToken(token)
            }
        }
        .addOnCompleteListener { imageProxy.close() }
}

// Sends the scanned token to the API; opens the confirmation screen if it verifies.
private fun verifyToken(token: String) {
    binding.textStatus.text = "Verifying..."
    lifecycleScope.launch {
        try {
            val response = RetrofitClient.getApiService(this@ScanActivity).verifyQr(VerifyQrRequest(token))

            if (response.isSuccessful && response.body() != null) {
                val reservation = response.body()!!
                val intent = Intent(this@ScanActivity, TransferConfirmActivity::class.java)
                intent.putExtra(Constants.EXTRA_RESERVATION_ID, reservation.id)
                startActivity(intent)
                finish()
            } else {
                binding.textStatus.text = response.errorMessageOrDefault("QR verification failed.")
                isProcessing = false
            }
        } catch (e: Exception) {
            binding.textStatus.text = "Could not reach the server. Check your connection."
            isProcessing = false
        }
    }
}
}

```

mobile/app/src/main/java/com/solarmicrogrid/app/operator/TransferConfirmActivity.kt

```
// -----  
// File: TransferConfirmActivity.kt  
// Purpose: Shows the QR-verified reservation's details and finalises the  
//           energy transfer business logic (marks it Completed server-side)  
//           once the Grid Operator confirms.  
// Module owner: Member D (Grid Operator Verification)  
// -----  
package com.solarmicrogrid.app.operator  
  
import android.os.Bundle  
import android.view.View  
import android.widget.Toast  
import androidx.appcompat.app.AppCompatActivity  
import androidx.lifecycle.LifecycleScope  
import com.solarmicrogrid.app.data.remote.RetrofitClient  
import com.solarmicrogrid.app.databinding.ActivityTransferConfirmBinding  
import com.solarmicrogrid.app.util.Constants  
import com.solarmicrogrid.app.util.applyEdgeToEdgeInsets  
import com.solarmicrogrid.app.util.errorMessageOrDefault  
import kotlinx.coroutines.launch  
  
class TransferConfirmActivity : AppCompatActivity() {  
  
    private lateinit var binding: ActivityTransferConfirmBinding  
    private lateinit var reservationId: String  
  
    // Loads the verified reservation's details and wires the Complete action.  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        binding = ActivityTransferConfirmBinding.inflate(layoutInflater)  
        setContentView(binding.root)  
        applyEdgeToEdgeInsets(topInsetView = binding.heroSection)  
  
        reservationId = intent.getStringExtra(Constants.EXTRA_RESERVATION_ID) ?: return finish()  
        binding.buttonComplete.setOnClickListener { completeTransfer() }  
  
        loadReservation()  
    }  
  
    // Fetches the reservation's current details for display.  
    private fun loadReservation() {  
        lifecycleScope.launch {  
            try {  
                val response = RetrofitClient.getApiService(this@TransferConfirmActivity).getReservation(reservationId)  
                if (response.isSuccessful && response.body() != null) {  
                    val reservation = response.body()!!  
                    binding.textCode.text = reservation.reservationCode  
                    binding.textDetails.text =  
                        "Prosumer NIC: ${reservation.prosumerNic}\nScheduled: ${reservation.scheduledDateTime.replace("T", " ").take(16)}\nEnergy: ${reservation.energyAmountKWh} kWh"  
                }  
            } catch (e: Exception) {  
                binding.textError.text = "Could not reach the server. Check your connection."  
                binding.errorBanner.visibility = View.VISIBLE  
            }  
        }  
    }  
  
    // Marks the reservation Completed on the server, finalising the transfer.  
    private fun completeTransfer() {  
        binding.buttonComplete.isEnabled = false  
        lifecycleScope.launch {  
            try {  
                val response = RetrofitClient.getApiService(this@TransferConfirmActivity).completeReservation(reservationId)  
                if (response.isSuccessful) {  
                    Toast.makeText(this@TransferConfirmActivity, "Energy transfer completed.",  
                        Toast.LENGTH_LONG).show()  
                    finish()  
                } else {  
                    binding.textError.text = response.errorMessageOrDefault("Failed to complete transfer.")  
                    binding.errorBanner.visibility = View.VISIBLE  
                }  
            }  
        }  
    }  
}
```



```

        binding.bottomNav.selectedItemId = R.id.nav_history
        binding.bottomNav.setOnItemSelectedListener { item ->
            when (item.itemId) {
                R.id.nav_home -> { startActivity(Intent(this, ProsumerHomeActivity::class.java)); false }
                R.id.nav_map -> { startActivity(Intent(this, MapActivity::class.java)); false }
                R.id.nav_profile -> { startActivity(Intent(this, ProfileActivity::class.java)); false }
                else -> true
            }
        }
    }

    // Reloads history whenever the screen regains focus (e.g. after a cancel).
    override fun onResume() {
        super.onResume()
        binding.bottomNav.selectedItemId = R.id.nav_history
        loadHistory()
    }

    // Applies a status chip as the active filter.
    private fun wireChip(chip: TextView, status: String?) {
        chip.setOnClickListener {
            statusFilter = if (statusFilter == status) null else status
            applyFilter()
        }
    }

    // Fetches the prosumer's full booking history from the API.
    private fun loadHistory() {
        val nic = SessionManager(this).current()?.nic ?: return
        lifecycleScope.launch {
            try {
                val response = RetrofitClient.getApiService(this@HistoryActivity).getReservationsByPro-
sumer(nic)
                if (response.isSuccessful && response.body() != null) {
                    allReservations = response.body()!!
                    applyFilter()
                }
            } catch (e: Exception) {
                Toast.makeText(this@HistoryActivity, "Could not reach the server. Check your connection.",
Toast.LENGTH_SHORT).show()
            }
        }
    }

    // Filters the in-memory list by the search text and the active status chip.
    private fun applyFilter() {
        val query = binding.inputSearch.text.toString().trim()
        val filtered = allReservations.filter { reservation ->
            val matchesQuery = query.isBlank() ||
                reservation.reservationCode.contains(query, ignoreCase = true) ||
                reservation.status.contains(query, ignoreCase = true)
            val matchesStatus = statusFilter == null || reservation.status == statusFilter
            matchesQuery && matchesStatus
        }

        adapter.submitList(filtered)
        binding.emptyState.visibility = if (filtered.isEmpty()) View.VISIBLE else View.GONE
        binding.recyclerHistory.visibility = if (filtered.isEmpty()) View.GONE else View.VISIBLE
    }
}

```

mobile/app/src/main/java/com/solarmicrogrid/app/prosumer/MapActivity.kt

```

// -----
// File: MapActivity.kt
// Purpose: Plots nearby microgrid stations on Google Maps around the
//          device's current location, with a station-info marker popup.
//          Falls back to the SQLite station cache (StationCacheDao) so the
//          screen still shows something when the API call is slow/offline.
// Module owner: Member B (Microgrid Nodes & Maps)
// -----
package com.solarmicrogrid.app.prosumer

```

```

import android.Manifest
import android.content.Intent
import android.content.pm.PackageManager
import android.os.Bundle
import android.view.ViewGroup
import androidx.appcompat.app.AppCompatActivity
import androidx.core.app.ActivityCompat
import androidx.core.content.ContextCompat
import androidx.core.view.ViewCompat
import androidx.core.view.WindowCompat
import androidx.core.view.WindowInsetsCompat
import androidx.lifecycle.LifecycleScope
import com.google.android.gms.maps.CameraUpdateFactory
import com.google.android.gms.maps.GoogleMap
import com.google.android.gms.maps.OnMapReadyCallback
import com.google.android.gms.maps.SupportMapFragment
import com.google.android.gms.maps.model.LatLng
import com.google.android.gms.maps.model.MarkerOptions
import com.google.android.gms.location.LocationServices
import com.google.android.material.bottomnavigation.BottomNavigationView
import com.solarmicrogrid.app.R
import com.solarmicrogrid.app.data.local.StationCacheDao
import com.solarmicrogrid.app.data.remote.RetrofitClient
import kotlinx.coroutines.launch

class MapActivity : AppCompatActivity(), OnMapReadyCallback {

    private lateinit var googleMap: GoogleMap
    private lateinit var stationCacheDao: StationCacheDao
    private val locationPermissionCode = 1001

    // Sets up the map fragment and local cache.
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_map)
        stationCacheDao = StationCacheDao(this)

        val mapFragment = supportFragmentManager.findFragmentById(R.id.mapFragment) as SupportMapFragment
        mapFragment.getMapAsync(this)

        setupBottomNav()
        setupEdgeToEdge()
    }

    // Pushes the floating header card below the status bar (by margin, since it
    // floats rather than sitting flush) and the bottom nav above the gesture bar.
    private fun setupEdgeToEdge() {
        WindowCompat.setDecorFitsSystemWindows(window, false)
        val headerCard = findViewById<ViewGroup>(R.id.headerCard)
        val bottomNav = findViewById<BottomNavigationView>(R.id.bottomNav)
        val baseTopMargin = (headerCard.layoutParams as ViewGroup.MarginLayoutParams).topMargin
        ViewCompat.setOnApplyWindowInsetsListener(headerCard.rootView) { _, insets ->
            val bars = insets.getInsets(WindowInsetsCompat.Type.systemBars())
            (headerCard.layoutParams as ViewGroup.MarginLayoutParams).topMargin = baseTopMargin + bars.top
            headerCard.requestLayout()
            bottomNav.setPadding(bottomNav.paddingLeft, bottomNav.paddingTop, bottomNav.paddingRight,
bars.bottom)
            insets
        }
    }

    // Wires the shared bottom tab bar, highlighting this screen's own tab.
    private fun setupBottomNav() {
        val bottomNav = findViewById<BottomNavigationView>(R.id.bottomNav)
        bottomNav.selectedItemId = R.id.nav_map
        bottomNav.setOnItemSelectedListener { item ->
            when (item.itemId) {
                R.id.nav_home -> { startActivity(Intent(this, ProsumerHomeActivity::class.java)); false }
                R.id.nav_history -> { startActivity(Intent(this, HistoryActivity::class.java)); false }
                R.id.nav_profile -> { startActivity(Intent(this, ProfileActivity::class.java)); false }
                else -> true
            }
        }
    }
}

```

```

}

// Called once the map is ready to receive markers/camera moves.
override fun onMapReady(map: GoogleMap) {
    googleMap = map

    // Show whatever was cached last, immediately.
    stationCacheDao.getAll().forEach { station ->
        googleMap.addMarker(
            MarkerOptions().position(LatLng(station.lat, station.lng))
                .title(station.stationName)
                .snippet("${station.availableSlots}/${station.totalSlots} battery slots available")
        )
    }

    if (hasLocationPermission()) {
        loadNearbyStations()
    } else {
        ActivityCompat.requestPermissions(
            this, arrayOf(Manifest.permission.ACCESS_FINE_LOCATION), locationPermissionCode
        )
    }
}

// Checks whether fine-location permission has already been granted.
private fun hasLocationPermission(): Boolean {
    return ContextCompat.checkSelfPermission(this, Manifest.permission.ACCESS_FINE_LOCATION) == PackageManager.PERMISSION_GRANTED
}

// Reacts to the location permission prompt's result.
override fun onRequestPermissionsResult(requestCode: Int, permissions: Array<out String>, grantResults: IntArray) {
    super.onRequestPermissionsResult(requestCode, permissions, grantResults)
    if (requestCode == locationPermissionCode && grantResults.isNotEmpty() && grantResults[0] == PackageManager.PERMISSION_GRANTED) {
        loadNearbyStations()
    }
}

// Gets the last-known location, then queries the API for nearby stations around it.
@android.annotation.SuppressLint("MissingPermission")
private fun loadNearbyStations() {
    val fusedClient = LocationServices.getFusedLocationProviderClient(this)
    fusedClient.lastLocation.addOnSuccessListener { location ->
        val lat = location?.latitude ?: 6.9271 // Falls back to Colombo if location is unavailable.
        val lng = location?.longitude ?: 79.8612

        googleMap.moveCamera(CameraUpdateFactory.newLatLngZoom(LatLng(lat, lng), 11f))

        lifecycleScope.launch {
            try {
                val response = RetrofitClient.getApiService(this@MapActivity).getNearbyStations(lat, lng, 50.0)

                if (response.isSuccessful && response.body() != null) {
                    val stations = response.body()!!
                    stationCacheDao.replaceAll(stations)
                    googleMap.clear()
                    stations.forEach { station ->
                        googleMap.addMarker(
                            MarkerOptions().position(LatLng(station.lat, station.lng))
                                .title(station.stationName)
                                .snippet("${station.availableBatterySlots}/${station.totalBatterySlots} battery slots available")
                        )
                    }
                }
            } catch (e: Exception) {
                // Keep whatever markers are already on screen (loaded from the cache above).
            }
        }
    }
}
}

```

mobile/app/src/main/java/com/solarmicrogrid/app/prosumer/ProfileActivity.kt

```
// -----
// File: ProfileActivity.kt
// Purpose: Prosumer self-service profile – view/edit contact details and
//          request account deactivation (Backoffice-only reactivation
//          afterwards, per the API's business rule).
// Module owner: Member A (Identity & Access)
// -----
package com.solarmicrogrid.app.prosumer

import android.app.AlertDialog
import android.content.Intent
import android.net.Uri
import android.os.Bundle
import android.view.View
import android.widget.Toast
import androidx.activity.result.PickVisualMediaRequest
import androidx.activity.result.contract.ActivityResultContracts
import androidx.appcompat.app.AppCompatActivity
import androidx.lifecycle.LifecycleScope
import com.bumptech.glide.Glide
import com.bumptech.glide.load.resource.bitmap.CenterCrop
import com.bumptech.glide.load.resource.bitmap.RoundedCorners
import com.solarmicrogrid.app.R
import com.solarmicrogrid.app.auth.LoginActivity
import com.solarmicrogrid.app.data.local.SessionManager
import com.solarmicrogrid.app.data.model.UpdateProsumerRequest
import com.solarmicrogrid.app.data.model.UserResponse
import com.solarmicrogrid.app.data.remote.RetrofitClient
import com.solarmicrogrid.app.databinding.ActivityProfileBinding
import com.solarmicrogrid.app.util.Constants
import com.solarmicrogrid.app.util.applyEdgeToEdgeInsets
import com.solarmicrogrid.app.util.errorMessageOrDefault
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.launch
import kotlinx.coroutines.withContext
import okhttp3.MediaType.Companion.toMediaTypeOrNull
import okhttp3.MultipartBody
import okhttp3.RequestBody.Companion.toRequestBody
import java.text.SimpleDateFormat
import java.util.Locale
import java.util.TimeZone

class ProfileActivity : AppCompatActivity() {

    private lateinit var binding: ActivityProfileBinding
    private lateinit var sessionManager: SessionManager

    // Registered up front (before STARTED) as required by the Activity Result API.
    private val pickImage = registerForActivityResult(ActivityResultContracts.PickVisualMedia()) { uri ->
        uri?.let { uploadPhoto(it) }
    }

    // Loads the prosumer's current profile and wires save/deactivate actions.
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityProfileBinding.inflate(layoutInflater)
        setContentView(binding.root)
        applyEdgeToEdgeInsets(topInsetView = binding.heroSection, bottomInsetView = binding.bottomNav)
        sessionManager = SessionManager(this)

        binding.buttonSave.setOnClickListener { saveProfile() }
        binding.buttonDeactivate.setOnClickListener { confirmDeactivate() }
        binding.buttonEditPhoto.setOnClickListener {
            pickImage.launch(PickVisualMediaRequest(ActivityResultContracts.PickVisualMedia.ImageOnly))
        }

        binding.bottomNav.selectedItemId = R.id.nav_profile
        binding.bottomNav.setOnItemSelectedListener { item ->
            when (item.itemId) {
                R.id.nav_home -> { startActivity(Intent(this, ProsumerHomeActivity::class.java)); false }
            }
        }
    }
}
```

```

        R.id.nav_map -> { startActivity(Intent(this, MapActivity::class.java)); false }
        R.id.nav_history -> { startActivity(Intent(this, HistoryActivity::class.java)); false }
        else -> true
    }
}

loadProfile()
}

// Fetches the prosumer's profile from the API and fills the form.
private fun loadProfile() {
    val nic = sessionManager.current()?.nic ?: return
    lifecycleScope.launch {
        try {
            val response = RetrofitClient.getApiService(this@ProfileActivity).getProsumer(nic)
            if (response.isSuccessful && response.body() != null) {
                val profile = response.body()!!
                binding.textName.text = profile.fullName
                binding.textNic.text = "NIC ${profile.nic}"
                binding.textMemberSince.text = "Member since ${formatMemberSince(profile.createdAt)}"
                binding.inputFullName.setText(profile.fullName)
                binding.inputEmail.setText(profile.email)
                binding.inputPhone.setText(profile.phone)
                binding.inputAddress.setText(profile.address)
                renderAvatar(profile)
                renderStatus(profile.status)
            }
        } catch (e: Exception) {
            showError("Could not reach the server. Check your connection.")
        }
    }
}

// Shows the profile photo if one is set, otherwise falls back to initials.
private fun renderAvatar(profile: UserResponse) {
    binding.textAvatar.text = profile.fullName
        .split(" ").filter { it.isNotBlank() }.take(2)
        .joinToString("") { it.first().uppercase() }

    if (profile.photoUrl != null) {
        val radiusPx = (14 * resources.displayMetrics.density).toInt()
        Glide.with(this)
            .load(Constants.SERVER_BASE_URL.trimEnd('/') + profile.photoUrl)
            .transform(CenterCrop(), RoundedCorners(radiusPx))
            .into(binding.imageAvatar)
        binding.imageAvatar.visibility = View.VISIBLE
    } else {
        binding.imageAvatar.visibility = View.GONE
    }
}

// Colours the status pill to match the account's actual state.
private fun renderStatus(status: String) {
    val (icon, tone, label) = when (status) {
        "Active" -> Triple(R.drawable.ic_check_circle, R.color.brand_300, "Active")
        "PendingActivation" -> Triple(R.drawable.ic_clock, R.color.solar_400, "Pending activation")
        else -> Triple(R.drawable.ic_alert, R.color.rose_500, "Deactivated")
    }
    binding.iconStatus.setImageResource(icon)
    binding.iconStatus.imageTintList = android.content.res.ColorStateList.valueOf(getColor(tone))
    binding.textStatus.text = label
    binding.textStatus.setTextColor(getColor(tone))
}

// Turns the API's ISO createdAt timestamp into a "Month yyyy" caption.
private fun formatMemberSince(iso: String): String {
    return try {
        val parser = SimpleDateFormat("yyyy-MM-dd'T'HH:mm:ss", Locale.US).apply {
            timeZone = TimeZone.getTimeZone("UTC")
        }
        val parsed = parser.parse(iso.take(19)) ?: return iso.take(7)
        SimpleDateFormat("MMMM yyyy", Locale.US).format(parsed)
    } catch (e: Exception) {
        iso.take(7)
    }
}

```

```

}

// Uploads the picked gallery image as the prosumer's new profile picture.
private fun uploadPhoto(uri: Uri) {
    val nic = sessionManager.current()?.nic ?: return
    lifecycleScope.launch {
        try {
            val mimeType = contentResolver.getType(uri) ?: "image/jpeg"
            val extension = when {
                mimeType.contains("png") -> "png"
                mimeType.contains("webp") -> "webp"
                else -> "jpg"
            }
            val bytes = withContext(Dispatchers.IO) {
                contentResolver.openInputStream(uri)?.use { it.readBytes() }
            } ?: throw IllegalStateException("Could not read the selected image.")

            val requestBody = bytes.toRequestBody(mimeType.toMediaTypeOrNull())
            val part = MultipartBody.Part.createFormData("file", "photo.$extension", requestBody)

            val response = RetrofitClient.getApiService(this@ProfileActivity).uploadProsumerPhoto(nic,
part)

            if (response.isSuccessful && response.body() != null) {
                renderAvatar(response.body()!!)
                Toast.makeText(this@ProfileActivity, "Profile picture updated.",
Toast.LENGTH_SHORT).show()
            } else {
                showError(response.errorMessageOrDefault("Failed to update profile picture."))
            }
        } catch (e: Exception) {
            showError(e.message ?: "Failed to update profile picture.")
        }
    }
}

// Submits the edited profile fields to the API.
private fun saveProfile() {
    val nic = sessionManager.current()?.nic ?: return
    val request = UpdateProsumerRequest(
        fullName = binding.inputFullName.text.toString().trim(),
        email = binding.inputEmail.text.toString().trim(),
        phone = binding.inputPhone.text.toString().trim(),
        address = binding.inputAddress.text.toString().trim()
    )

    lifecycleScope.launch {
        try {
            val response = RetrofitClient.getApiService(this@ProfileActivity).updateProsumer(nic, request)
            if (response.isSuccessful) {
                Toast.makeText(this@ProfileActivity, "Profile updated.", Toast.LENGTH_SHORT).show()
            } else {
                showError(response.errorMessageOrDefault("Failed to update profile."))
            }
        } catch (e: Exception) {
            showError("Could not reach the server. Check your connection.")
        }
    }
}

// Confirms and submits an account-deactivation request, then logs the user out.
private fun confirmDeactivate() {
    AlertDialog.Builder(this)
        .setTitle("Deactivate Account")
        .setMessage("Your account will be deactivated immediately. A Backoffice officer must reactivate it before you can log in again. Continue?")
        .setPositiveButton("Deactivate") { _, _ -> deactivate() }
        .setNegativeButton("Cancel", null)
        .show()
}

// Calls the deactivation endpoint and returns to the login screen.
private fun deactivate() {
    val nic = sessionManager.current()?.nic ?: return
    lifecycleScope.launch {
        try {

```

```

        val response = RetrofitClient.getApiService(this@ProfileActivity).requestDeactivation(nic)
        if (response.isSuccessful) {
            sessionManager.clear()
            Toast.makeText(this@ProfileActivity, "Account deactivated.", Toast.LENGTH_LONG).show()
            val intent = Intent(this@ProfileActivity, LoginActivity::class.java)
            intent.flags = Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_CLEAR_TASK
            startActivity(intent)
            finish()
        } else {
            showError(response.errorMessageOrDefault("Failed to deactivate account."))
        }
    } catch (e: Exception) {
        showError("Could not reach the server. Check your connection.")
    }
}

// Shows the inline error banner.
private fun showError(message: String) {
    binding.textError.text = message
    binding.errorBanner.visibility = View.VISIBLE
}
}

```

mobile/app/src/main/java/com/solarmicrogrid/app/prosumer/ProsumerHomeActivity.kt

```

// -----
// File: ProsumerHomeActivity.kt
// Purpose: Prosumer dashboard – approved/pending reservation counts and the
//           upcoming-bookings list read live from the API, plus bottom
//           navigation to the map, history and profile screens.
// Module owner: Member C (Booking Views & Operational Dashboards)
// -----
package com.solarmicrogrid.app.prosumer

import android.content.Intent
import android.os.Bundle
import android.view.View
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
import androidx.lifecycle.lifecycleScope
import androidx.recyclerview.widget.LinearLayoutManager
import com.bumptech.glide.Glide
import com.bumptech.glide.load.resource.bitmap.CenterCrop
import com.bumptech.glide.load.resource.bitmap.RoundedCorners
import com.solarmicrogrid.app.R
import com.solarmicrogrid.app.auth.LoginActivity
import com.solarmicrogrid.app.data.local.SessionManager
import com.solarmicrogrid.app.data.remote.RetrofitClient
import com.solarmicrogrid.app.databinding.ActivityProsumerHomeBinding
import com.solarmicrogrid.app.util.Constants
import com.solarmicrogrid.app.util.applyEdgeToEdgeInsets
import kotlinx.coroutines.launch

class ProsumerHomeActivity : AppCompatActivity() {

    private lateinit var binding: ActivityProsumerHomeBinding
    private lateinit var sessionManager: SessionManager
    private lateinit var adapter: ReservationAdapter

    // Sets up the dashboard, its list adapter and the bottom navigation.
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityProsumerHomeBinding.inflate(layoutInflater)
        setContentView(binding.root)
        applyEdgeToEdgeInsets(topInsetView = binding.heroSection, bottomInsetView = binding.bottomNav)
        sessionManager = SessionManager(this)

        val session = sessionManager.current() ?: return redirectToLogin()
        binding.textWelcome.text = session.fullName
        binding.textAvatar.text = initials(session.fullName)

        adapter = ReservationAdapter(emptyList()) { reservation ->

```

```

        val intent = Intent(this, ReservationDetailActivity::class.java)
        intent.putExtra(Constants.EXTRA_RESERVATION_ID, reservation.id)
        startActivity(intent)
    }
    binding.recyclerUpcoming.layoutManager = LinearLayoutManager(this)
    binding.recyclerUpcoming.adapter = adapter
    binding.recyclerUpcoming.isNestedScrollingEnabled = false

    binding.swipeRefresh.setColorSchemeResources(R.color.brand_600)
    binding.swipeRefresh.setOnRefreshListener { session.nic?.let { loadDashboard(it) } }
    binding.buttonNewReservation.setOnClickListener {
        startActivity(Intent(this, ReservationFormActivity::class.java))
    }
    binding.textSeeAll.setOnClickListener { startActivity(Intent(this, HistoryActivity::class.java)) }
    binding.buttonLogout.setOnClickListener { logout() }

    binding.bottomNav.selectedItemId = R.id.nav_home
    binding.bottomNav.setOnItemSelectedListener { item ->
        when (item.itemId) {
            R.id.nav_map -> { startActivity(Intent(this, MapActivity::class.java)); false }
            R.id.nav_history -> { startActivity(Intent(this, HistoryActivity::class.java)); false }
            R.id.nav_profile -> { startActivity(Intent(this, ProfileActivity::class.java)); false }
            else -> true
        }
    }
}

// Refreshes the dashboard every time the screen becomes visible again.
override fun onResume() {
    super.onResume()
    binding.bottomNav.selectedItemId = R.id.nav_home
    sessionManager.current()?.nic?.let {
        loadDashboard(it)
        loadAvatar(it)
    }
}

// Fetches the prosumer's dashboard summary from the API.
private fun loadDashboard(nic: String) {
    binding.swipeRefresh.isRefreshing = true
    lifecycleScope.launch {
        try {
            val response = RetrofitClient.getApiService(this@ProsumerHomeActivity).getProsumerDashboard(nic)

            if (response.isSuccessful && response.body() != null) {
                val dashboard = response.body()!!
                binding.textActiveCount.text = dashboard.activeCount.toString()
                binding.textPendingCount.text = dashboard.pendingCount.toString()
                adapter.submitList(dashboard.upcomingReservations)
                binding.emptyState.visibility =
                    if (dashboard.upcomingReservations.isEmpty()) View.VISIBLE else View.GONE
            }
        } catch (e: Exception) {
            Toast.makeText(this@ProsumerHomeActivity, "Could not reach the server. Check your connection.", Toast.LENGTH_SHORT).show()
        } finally {
            binding.swipeRefresh.isRefreshing = false
        }
    }
}

// Builds two-letter initials for the avatar chip.
private fun initials(name: String): String =
    name.split(" ").filter { it.isNotBlank() }.take(2).joinToString("") { it.first().uppercase() }

// Shows the prosumer's uploaded profile photo in the avatar chip, if one is set.
private fun loadAvatar(nic: String) {
    lifecycleScope.launch {
        try {
            val response = RetrofitClient.getApiService(this@ProsumerHomeActivity).getProsumer(nic)
            val photoUrl = response.body()?.photoUrl
            if (response.isSuccessful && photoUrl != null) {
                val radiusPx = (10 * resources.displayMetrics.density).toInt()
                Glide.with(this@ProsumerHomeActivity)
                    .load(Constants.SERVER_BASE_URL.trimEnd('/') + photoUrl)
            }
        }
    }
}

```

```

                .transform(CenterCrop(), RoundedCorners(radiusPx))
                .into(binding.imageAvatar)
                binding.imageAvatar.visibility = View.VISIBLE
            }
        } catch (e: Exception) {
            // Non-critical: the initials chip stays as the fallback.
        }
    }
}

// Clears the local session and returns to the login screen.
private fun logout() {
    sessionManager.clear()
    redirectToLogin()
}

// Sends the user back to LoginActivity, clearing the back stack.
private fun redirectToLogin() {
    val intent = Intent(this, LoginActivity::class.java)
    intent.flags = Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_CLEAR_TASK
    startActivity(intent)
    finish()
}
}

```

mobile/app/src/main/java/com/solarmicrogrid/app/prosumer/ReservationAdapter.kt

```

// -----
// File: ReservationAdapter.kt
// Purpose: RecyclerView adapter shared by the prosumer dashboard/history and the
//          grid operator's approval queue. Renders one reservation card per row
//          and colours the status pill to match the web console's palette.
// Module owner: Member C (Booking Views & Operational Dashboards)
// -----
package com.solarmicrogrid.app.prosumer

import android.view.LayoutInflater
import android.view.ViewGroup
import androidx.core.content.ContextCompat
import androidx.recyclerview.widget.RecyclerView
import com.solarmicrogrid.app.R
import com.solarmicrogrid.app.data.model.ReservationResponse
import com.solarmicrogrid.app.databinding.ItemReservationBinding
import java.text.SimpleDateFormat
import java.util.Locale
import java.util.TimeZone

class ReservationAdapter(
    private var items: List<ReservationResponse>,
    private val onClick: (ReservationResponse) -> Unit
) : RecyclerView.Adapter<ReservationAdapter.ReservationViewHolder>() {

    class ReservationViewHolder(val binding: ItemReservationBinding) : RecyclerView.ViewHolder(binding.root)

    // Inflates a single reservation card.
    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): ReservationViewHolder {
        val binding = ItemReservationBinding.inflate(LayoutInflater.from(parent.context), parent, false)
        return ReservationViewHolder(binding)
    }

    // Binds one reservation's fields and applies its status colours.
    override fun onBindViewHolder(holder: ReservationViewHolder, position: Int) {
        val reservation = items[position]
        val context = holder.itemView.context

        holder.binding.textCode.text = reservation.reservationCode
        holder.binding.textSchedule.text = formatSchedule(reservation.scheduledDateTime)
        holder.binding.textEnergy.text = "${reservation.energyAmountKWh} kWh"

        val (pillBackground, pillText) = statusColours(reservation.status)
        holder.binding.textStatus.text = reservation.status
        holder.binding.textStatus.setBackgroundResource(pillBackground)
    }
}

```

```

        holder.binding.textStatus.setTextColor(ContextCompat.getColor(context, pillText))

        holder.itemView.setOnClickListener { onClick(reservation) }
    }

    override fun getItemCount(): Int = items.size

    // Replaces the backing list and refreshes the view.
    fun submitList(newItems: List<ReservationResponse>) {
        items = newItems
        notifyDataSetChanged()
    }

    // Maps a reservation status onto its pill background and text colour.
    private fun statusColours(status: String): Pair<Int, Int> = when (status) {
        "Approved" -> R.drawable.bg_pill_emerald to R.color.brand_700
        "Pending" -> R.drawable.bg_pill_amber to R.color.solar_600
        "Completed" -> R.drawable.bg_pill_sky to R.color.sky_700
        "Cancelled" -> R.drawable.bg_pill_rose to R.color.rose_700
        else -> R.drawable.bg_pill_slate to R.color.slate_600
    }

    // Turns the API's ISO timestamp into a short, readable local date/time.
    private fun formatSchedule(iso: String): String {
        return try {
            val parser = SimpleDateFormat("yyyy-MM-dd'T'HH:mm:ss", Locale.US).apply {
                timeZone = TimeZone.getTimeZone("UTC")
            }
            val parsed = parser.parse(iso.take(19)) ?: return iso.take(16).replace("T", " ")
            SimpleDateFormat("dd MMM yyyy · HH:mm", Locale.US).format(parsed)
        } catch (e: Exception) {
            iso.take(16).replace("T", " ")
        }
    }
}

```

mobile/app/src/main/java/com/solarmicrogrid/app/prosumer/ReservationDetailActivity.kt

```

// -----
// File: ReservationDetailActivity.kt
// Purpose: Shows one reservation's full details, renders its transaction QR
//          code once approved, and offers Modify/Cancel actions (the
//          12-hour notice rule is enforced server-side; this screen just
//          surfaces the API's rejection message when it fires).
// Module owner: Member C (Reservation Workflow & Booking Management)
// -----
package com.solarmicrogrid.app.prosumer

import android.app.AlertDialog
import android.content.Intent
import android.os.Bundle
import android.view.View
import android.widget.EditText
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
import androidx.lifecycle.lifecycleScope
import com.solarmicrogrid.app.data.model.CancelReservationRequest
import com.solarmicrogrid.app.data.model.ReservationResponse
import com.solarmicrogrid.app.data.remote.RetrofitClient
import com.solarmicrogrid.app.databinding.ActivityReservationDetailBinding
import com.solarmicrogrid.app.util.Constants
import com.solarmicrogrid.app.util.QrCodeGenerator
import com.solarmicrogrid.app.util.applyEdgeToEdgeInsets
import com.solarmicrogrid.app.util.errorMessageOrDefault
import kotlinx.coroutines.launch
import java.text.SimpleDateFormat
import java.util.Locale
import java.util.TimeZone

class ReservationDetailActivity : AppCompatActivity() {

    private lateinit var binding: ActivityReservationDetailBinding
}

```

```

private lateinit var reservationId: String

// Loads the reservation and wires the Modify/Cancel actions.
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    binding = ActivityReservationDetailBinding.inflate(layoutInflater)
    setContentView(binding.root)
    applyEdgeToEdgeInsets(topInsetView = binding.heroSection)

    reservationId = intent.getStringExtra(Constants.EXTRA_RESERVATION_ID) ?: return finish()

    binding.buttonEdit.setOnClickListener {
        val intent = Intent(this, ReservationFormActivity::class.java)
        intent.putExtra(Constants.EXTRA_RESERVATION_ID, reservationId)
        startActivity(intent)
    }
    binding.buttonCancel.setOnClickListener { promptCancel() }

    loadReservation()
}

// Reloads the reservation whenever this screen regains focus (e.g. after an edit).
override fun onResume() {
    super.onResume()
    loadReservation()
}

// Fetches the current reservation state from the API and renders it.
private fun loadReservation() {
    lifecycleScope.launch {
        try {
            val response = RetrofitClient.getApiService(this@ReservationDetailActivity).getReservation(reservationId)
            if (response.isSuccessful && response.body() != null) {
                render(response.body()!!)
            }
        } catch (e: Exception) {
            showError("Could not load reservation.")
        }
    }
}

// Populates the screen with the reservation's fields and QR code (if approved).
private fun render(reservation: ReservationResponse) {
    binding.textCode.text = reservation.reservationCode
    binding.textStatus.text = reservation.status.uppercase()
    binding.textScheduled.text = formatSchedule(reservation.scheduledDateTime)
    binding.textEnergy.text = "${reservation.energyAmountKWh} kWh"

    val canModify = reservation.status == "Pending" || reservation.status == "Approved"
    binding.buttonEdit.visibility = if (canModify) View.VISIBLE else View.GONE
    binding.buttonCancel.visibility = if (canModify) View.VISIBLE else View.GONE

    if (reservation.status == "Approved" && !reservation.qrToken.isNullOrEmpty()) {
        binding.imageQr.setImageBitmap(QrCodeGenerator.generate(reservation.qrToken))
        binding.qrCard.visibility = View.VISIBLE
    } else {
        binding.qrCard.visibility = View.GONE
    }
}

// Turns the API's ISO timestamp into a readable local date and time.
private fun formatSchedule(iso: String): String {
    return try {
        val parser = SimpleDateFormat("yyyy-MM-dd'T'HH:mm:ss", Locale.US).apply {
            timeZone = TimeZone.getTimeZone("UTC")
        }
        val parsed = parser.parse(iso.take(19)) ?: return iso.take(16).replace("T", " ")
        SimpleDateFormat("dd MMM yyyy 'at' HH:mm", Locale.US).format(parsed)
    } catch (e: Exception) {
        iso.take(16).replace("T", " ")
    }
}

```

```

// Prompts for a cancellation reason, then submits it to the API.
private fun promptCancel() {
    val input = EditText(this)
    input.hint = "Reason for cancellation"
    AlertDialog.Builder(this)
        .setTitle("Cancel Reservation")
        .setView(input)
        .setPositiveButton("Cancel Reservation") { _, _ -> submitCancel(input.text.toString()) }
        .setNegativeButton("Keep Reservation", null)
        .show()
}

// Sends the cancellation request; the API rejects it if under 12 hours' notice remains.
private fun submitCancel(reason: String) {
    lifecycleScope.launch {
        try {
            val response = RetrofitClient.getApiService(this@ReservationDetailActivity)
                .cancelReservation(reservationId, CancelReservationRequest(reason))
            if (response.isSuccessful) {
                Toast.makeText(this@ReservationDetailActivity, "Reservation cancelled.",
                    Toast.LENGTH_SHORT).show()
                finish()
            } else {
                showError(response.errorMessageOrDefault("Could not cancel this reservation."))
            }
        } catch (e: Exception) {
            showError("Could not reach the server. Check your connection.")
        }
    }
}

// Shows the inline error banner.
private fun showError(message: String) {
    binding.textError.text = message
    binding.errorBanner.visibility = View.VISIBLE
}
}

```

mobile/app/src/main/java/com/solarmicrogrid/app/prosumer/ReservationFormActivity.kt

```

// -----
// File: ReservationFormActivity.kt
// Purpose: Create a new reservation, or edit an existing one when launched
//          with EXTRA_RESERVATION_ID. The 7-day booking window and 12-hour
//          update notice rules are enforced by the API; this screen simply
//          surfaces whatever validation message comes back.
// Module owner: Member C (Reservation Workflow & Booking Management)
// -----
package com.solarmicrogrid.app.prosumer

import android.app.DatePickerDialog
import android.app.TimePickerDialog
import android.os.Bundle
import android.view.View
import android.widget.AdapterView
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
import androidx.lifecycle.LifecycleScope
import com.solarmicrogrid.app.data.local.SessionManager
import com.solarmicrogrid.app.data.model.CreateReservationRequest
import com.solarmicrogrid.app.data.model.SlotResponse
import com.solarmicrogrid.app.data.model.StationResponse
import com.solarmicrogrid.app.data.model.UpdateReservationRequest
import com.solarmicrogrid.app.data.remote.RetrofitClient
import com.solarmicrogrid.app.databinding.ActivityReservationFormBinding
import com.solarmicrogrid.app.util.Constants
import com.solarmicrogrid.app.util.applyEdgeToEdgeInsets
import com.solarmicrogrid.app.util.errorMessageOrDefault
import kotlinx.coroutines.launch
import java.text.SimpleDateFormat
import java.util.Calendar
import java.util.Locale
import java.util.TimeZone

```

```

class ReservationFormActivity : AppCompatActivity() {

    private lateinit var binding: ActivityReservationFormBinding
    private lateinit var sessionManager: SessionManager

    private var stations: List<StationResponse> = emptyList()
    private var slots: List<SlotResponse> = emptyList()
    private var selectedCalendar: Calendar? = null
    private var editingReservationId: String? = null

    private val isoFormat = SimpleDateFormat("yyyy-MM-dd'T'HH:mm:ss'Z'", Locale.US).apply {
        timeZone = TimeZone.getTimeZone("UTC")
    }

    // Loads stations (and the reservation being edited, if any) and wires the form.
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityReservationFormBinding.inflate(layoutInflater)
        setContentView(binding.root)
        applyEdgeToEdgeInsets(topInsetView = binding.heroSection)
        sessionManager = SessionManager(this)

        editingReservationId = intent.getStringExtra(Constants.EXTRA_RESERVATION_ID)
        if (editingReservationId != null) {
            binding.textTitle.text = "Edit Reservation"
        }

        binding.buttonPickDateTime.setOnClickListener { showDateTimePicker() }
        binding.buttonSubmit.setOnClickListener { submit() }

        loadStations()
    }

    // Fetches the active station list for the dropdown.
    private fun loadStations() {
        lifecycleScope.launch {
            try {
                val response = RetrofitClient.getApiService(this@ReservationFormActivity).getStations()
                if (response.isSuccessful && response.body() != null) {
                    stations = response.body()!!.filter { it.status == "Active" }
                    binding.spinnerStation.adapter = ArrayAdapter(
                        this@ReservationFormActivity, android.R.layout.simple_spinner_dropdown_item,
                        stations.map { it.stationName }
                    )
                    binding.spinnerStation.setOnItemSelectedListener(object : android.widget.AdapterView-
View.OnItemSelectedListener {
                        override fun onItemSelected(parent: android.widget.AdapterView<*>?, view: View?, posi-
tion: Int, id: Long) {
                            loadSlots(stations[position].id)
                        }
                        override fun onNothingSelected(parent: android.widget.AdapterView<*>?) {}
                    })
                } catch (e: Exception) {
                    showError("Could not load stations.")
                }
            }
        }
    }

    // Fetches the bookable slots for the chosen station.
    private fun loadSlots(stationId: String) {
        lifecycleScope.launch {
            try {
                val response = RetrofitClient.getApiService(this@ReservationFormActivity).getSlots(stationId)
                if (response.isSuccessful && response.body() != null) {
                    slots = response.body()!!.filter { it.status != "Closed" }
                    binding.spinnerSlot.adapter = ArrayAdapter(
                        this@ReservationFormActivity, android.R.layout.simple_spinner_dropdown_item,
                        slots.map { "${it.date.take(10)} ${it.startTime}-${it.endTime} (${it.capacityAvaila-
ble} left)" }
                    )
                } catch (e: Exception) {

```

```

        showError("Could not load slots.")
    }
}

// Opens a date picker followed by a time picker, storing the combined UTC instant.
private fun showDateTimePicker() {
    val now = Calendar.getInstance()
    DatePickerDialog(this, { _, year, month, day ->
        TimePickerDialog(this, { _, hour, minute ->
            val calendar = Calendar.getInstance()
            calendar.set(year, month, day, hour, minute, 0)
            selectedCalendar = calendar
            binding.buttonPickDateTime.text = SimpleDateFormat("dd MMM yyyy, HH:mm", Lo-
cale.US).format(calendar.time)
        }, now.get(Calendar.HOUR_OF_DAY), now.get(Calendar.MINUTE), true).show()
    }, now.get(Calendar.YEAR), now.get(Calendar.MONTH), now.get(Calendar.DAY_OF_MONTH)).show()
}

// Validates the form and creates or updates the reservation via the API.
private fun submit() {
    val nic = sessionManager.current()?.nic ?: return
    if (stations.isEmpty() || slots.isEmpty() || selectedCalendar == null) {
        showError("Please select a station, slot and date/time.")
        return
    }
    val energyText = binding.inputEnergyAmount.text.toString()
    val energyAmount = energyText.toDoubleOrNull()
    if (energyAmount == null || energyAmount <= 0) {
        showError("Enter a valid energy amount.")
        return
    }

    val station = stations[binding.spinnerStation.selectedItemPosition]
    val slot = slots[binding.spinnerSlot.selectedItemPosition]
    val scheduledIso = isoFormat.format(selectedCalendar!!.time)

    binding.buttonSubmit.isEnabled = false
    lifecycleScope.launch {
        try {
            val response = if (editingReservationId == null) {
                RetrofitClient.getApiService(this@ReservationFormActivity).createReservation(
                    CreateReservationRequest(nic, station.id, slot.id, scheduledIso, energyAmount)
                )
            } else {
                RetrofitClient.getApiService(this@ReservationFormActivity).updateReservation(
                    editingReservationId!!, UpdateReservationRequest(slot.id, scheduledIso, energyAmount)
                )
            }

            if (response.isSuccessful) {
                Toast.makeText(this@ReservationFormActivity, "Reservation saved.",
Toast.LENGTH_SHORT).show()
                finish()
            } else {
                showError(response.errorMessageOrDefault("Failed to save reservation."))
            }
        } catch (e: Exception) {
            showError("Could not reach the server. Check your connection.")
        } finally {
            binding.buttonSubmit.isEnabled = true
        }
    }
}

// Shows the inline error banner above the form.
private fun showError(message: String) {
    binding.textError.text = message
    binding.errorBanner.visibility = View.VISIBLE
}
}

```

mobile/app/src/main/java/com/solarmicrogrid/app/util/ApiErrorParser.kt

```
// -----  
// File: ApiErrorParser.kt  
// Purpose: Extracts a human-readable message from a failed Retrofit  
//           Response's error body (the {"message": "..."} JSON written by  
//           the API's GlobalExceptionHandler).  
// Module owner: shared infrastructure  
// -----  
package com.solarmicrogrid.app.util  
  
import com.google.gson.Gson  
import com.solarmicrogrid.app.data.model.ApiErrorBody  
import retrofit2.Response  
  
// Reads the error body of a failed response, falling back to a generic message.  
fun <T> Response<T>.errorMessageOrDefault(default: String = "Something went wrong. Please try again."): String  
{  
    val body = errorBody()?.string()  
    if (body.isNullOrBlank()) return default  
    return try {  
        Gson().fromJson(body, ApiErrorBody::class.java)?.message ?: default  
    } catch (e: Exception) {  
        default  
    }  
}
```

mobile/app/src/main/java/com/solarmicrogrid/app/util/Constants.kt

```
// -----  
// File: Constants.kt  
// Purpose: App-wide constants – the Web API base URL and shared Intent extra  
//           keys. On the emulator, 10.0.2.2 aliases the host machine's  
//           localhost. On a physical device over USB, `adb reverse tcp:5080  
//           tcp:5080` forwards the device's own localhost to the host, so  
//           127.0.0.1 works there too without needing a shared Wi-Fi network.  
// Module owner: shared infrastructure  
// -----  
package com.solarmicrogrid.app.util  
  
object Constants {  
    // Points at the ASP.NET Core Web API. Swap for the IIS-hosted URL when deploying.  
    const val API_BASE_URL = "http://127.0.0.1:5080/api/"  
  
    // Same host without the "api/" suffix, for loading static assets like uploaded photos.  
    const val SERVER_BASE_URL = "http://127.0.0.1:5080/"  
  
    const val EXTRA_RESERVATION_ID = "extra_reservation_id"  
    const val EXTRA_STATION_ID = "extra_station_id"  
    const val EXTRA_QR_TOKEN = "extra_qr_token"  
}
```

mobile/app/src/main/java/com/solarmicrogrid/app/util/EdgeToEdge.kt

```
// -----  
// File: EdgeToEdge.kt  
// Purpose: Draws an activity edge-to-edge and manually pads specific views by  
//           the actual system-bar insets. Needed because the legacy  
//           android:fitsSystemWindows XML attribute stops reliably reaching a  
//           nested view once a SwipeRefreshLayout/ScrollView sits in between:  
//           it gets consumed by whichever ancestor sees it first (often the  
//           activity's root layout), leaving a blank gap above content that  
//           expects to bleed under the status bar instead.  
// Module owner: shared infrastructure  
// -----  
package com.solarmicrogrid.app.util  
  
import android.app.Activity  
import android.view.View  
import androidx.core.view.ViewCompat  
import androidx.core.view.WindowCompat  
import androidx.core.view.WindowInsetsCompat
```

```

// `topInsetView` keeps its own background bleeding under the status bar while its
// content is pushed below it. `bottomInsetView` (e.g. a BottomNavigationView) is
// pushed above the gesture/nav bar the same way.
fun Activity.applyEdgeToEdgeInsets(topInsetView: View? = null, bottomInsetView: View? = null) {
    WindowCompat.setDecorFitsSystemWindows(window, false)
    val root = topInsetView?.rootView ?: bottomInsetView?.rootView ?: return
    ViewCompat.setOnApplyWindowInsetsListener(root) { _, insets ->
        val bars = insets.getInsets(WindowInsetsCompat.Type.systemBars())
        topInsetView?.setPadding(topInsetView.paddingLeft, bars.top, topInsetView.paddingRight, top-
InsetView.paddingBottom)
        bottomInsetView?.setPadding(bottomInsetView.paddingLeft, bottomInsetView.paddingTop, bottom-
InsetView.paddingRight, bars.bottom)
        insets
    }
}

```

mobile/app/src/main/java/com/solarmicrogrid/app/util/QrCodeGenerator.kt

```

// -----
// File: QrCodeGenerator.kt
// Purpose: Renders the server-issued qrToken string into a scannable QR
//          bitmap on-device (ZXing), so the prosumer app never needs an
//          image endpoint from the API – just the token text.
// Module owner: Member C (Reservation & QR Dispatch)
// -----
package com.solarmicrogrid.app.util

import android.graphics.Bitmap
import com.google.zxing.BarcodeFormat
import com.google.zxing.qrcode.QRCodeWriter

object QrCodeGenerator {

    // Encodes the given text as a square black-and-white QR code bitmap.
    fun generate(content: String, sizePx: Int = 600): Bitmap {
        val bitMatrix = QRCodeWriter().encode(content, BarcodeFormat.QR_CODE, sizePx, sizePx)
        val bitmap = Bitmap.createBitmap(sizePx, sizePx, Bitmap.Config.RGB_565)
        for (x in 0 until sizePx) {
            for (y in 0 until sizePx) {
                bitmap.setPixel(x, y, if (bitMatrix[x, y]) android.graphics.Color.BLACK else an-
droid.graphics.Color.WHITE)
            }
        }
        return bitmap
    }
}

```

mobile/app/src/main/res/layout/activity_history.xml

```

<?xml version="1.0" encoding="utf-8"?>
<!-- Full booking history with a live search filter over code and status. -->
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:background="@color/page_background">

    <!-- Header -->
    <FrameLayout
        android:id="@+id/heroSection"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:background="@drawable/bg_hero_gradient_rounded">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:paddingStart="20dp"

```

```

        android:paddingEnd="20dp"
        android:paddingTop="24dp"
        android:paddingBottom="42dp">

        <FrameLayout
            android:layout_width="42dp"
            android:layout_height="42dp"
            android:background="@drawable/bg_tile_glass">

            <ImageView
                android:layout_width="20dp"
                android:layout_height="20dp"
                android:layout_gravity="center"
                android:src="@drawable/ic_history"
                app:tint="@color/brand_200"
                android:contentDescription="@null" />
        </FrameLayout>

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginTop="14dp"
            android:text="Booking history"
            android:textColor="@color/white"
            android:textSize="24sp"
            android:textStyle="bold" />

        <TextView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="4dp"
            android:text="Every energy transfer you have booked, past and upcoming."
            android:textColor="#99FFFFFF"
            android:textSize="13sp" />
    </LinearLayout>
</FrameLayout>

<!-- Search field pulled over the header -->
<com.google.android.material.card.MaterialCardView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="-26dp"
    android:layout_marginStart="20dp"
    android:layout_marginEnd="20dp"
    app:cardCornerRadius="16dp"
    app:cardElevation="6dp"
    app:cardBackgroundColor="@color/white">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:gravity="center_vertical"
        android:paddingStart="14dp"
        android:paddingEnd="14dp">

        <ImageView
            android:layout_width="18dp"
            android:layout_height="18dp"
            android:src="@drawable/ic_search"
            app:tint="@color/slate_400"
            android:contentDescription="@null" />

        <EditText
            android:id="@+id/inputSearch"
            android:layout_width="match_parent"
            android:layout_height="52dp"
            android:layout_marginStart="10dp"
            android:background="@android:color/transparent"
            android:hint="Search by code or status"
            android:inputType="text"
            android:maxLines="1"
            android:textColor="@color/slate_900"
            android:textColorHint="@color/slate_400"

```

```

        android:textSize="14sp"
        android:importantForAutofill="no" />
    </LinearLayout>
</com.google.android.material.card.MaterialCardView>

<!-- Status filter legend -->
<HorizontalScrollView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="14dp"
    android:paddingStart="20dp"
    android:paddingEnd="20dp"
    android:scrollbars="none"
    android:clipToPadding="false">

    <LinearLayout
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:orientation="horizontal">

        <TextView
            android:id="@+id/chipAll"
            style="@style/Text.Solar.Body"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:background="@drawable/bg_pill_slate"
            android:paddingStart="14dp" android:paddingEnd="14dp"
            android:paddingTop="7dp" android:paddingBottom="7dp"
            android:text="All"
            android:textColor="@color/slate_700"
            android:textSize="12sp"
            android:textStyle="bold" />

        <TextView
            android:id="@+id/chipPending"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginStart="8dp"
            android:background="@drawable/bg_pill_amber"
            android:paddingStart="14dp" android:paddingEnd="14dp"
            android:paddingTop="7dp" android:paddingBottom="7dp"
            android:text="Pending"
            android:textColor="@color/solar_600"
            android:textSize="12sp"
            android:textStyle="bold" />

        <TextView
            android:id="@+id/chipApproved"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginStart="8dp"
            android:background="@drawable/bg_pill_emerald"
            android:paddingStart="14dp" android:paddingEnd="14dp"
            android:paddingTop="7dp" android:paddingBottom="7dp"
            android:text="Approved"
            android:textColor="@color/brand_700"
            android:textSize="12sp"
            android:textStyle="bold" />

        <TextView
            android:id="@+id/chipCompleted"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginStart="8dp"
            android:background="@drawable/bg_pill_sky"
            android:paddingStart="14dp" android:paddingEnd="14dp"
            android:paddingTop="7dp" android:paddingBottom="7dp"
            android:text="Completed"
            android:textColor="@color/sky_700"
            android:textSize="12sp"
            android:textStyle="bold" />

        <TextView
            android:id="@+id/chipCancelled"
            android:layout_width="wrap_content"

```

```

        android:layout_height="wrap_content"
        android:layout_marginStart="8dp"
        android:background="@drawable/bg_pill_rose"
        android:paddingStart="14dp" android:paddingEnd="14dp"
        android:paddingTop="7dp" android:paddingBottom="7dp"
        android:text="Cancelled"
        android:textColor="@color/rose_700"
        android:textSize="12sp"
        android:textStyle="bold" />
    </LinearLayout>
</HorizontalScrollView>

<androidx.recyclerview.widget.RecyclerView
    android:id="@+id/recyclerHistory"
    android:layout_width="match_parent"
    android:layout_height="0dp"
    android:layout_weight="1"
    android:layout_marginTop="14dp"
    android:paddingStart="20dp"
    android:paddingEnd="20dp"
    android:paddingBottom="20dp"
    android:clipToPadding="false"
    android:scrollbars="none" />

<LinearLayout
    android:id="@+id/emptyState"
    android:layout_width="match_parent"
    android:layout_height="0dp"
    android:layout_weight="1"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="36dp"
    android:visibility="gone">

    <FrameLayout
        android:layout_width="60dp"
        android:layout_height="60dp"
        android:background="@drawable/bg_tile_slate">

        <ImageView
            android:layout_width="28dp"
            android:layout_height="28dp"
            android:layout_gravity="center"
            android:src="@drawable/ic_search"
            app:tint="@color/slate_400"
            android:contentDescription="@null" />
    </FrameLayout>

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="14dp"
        android:text="No reservations found"
        android:textColor="@color/slate_700"
        android:textSize="15sp"
        android:textStyle="bold" />

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="4dp"
        android:gravity="center"
        android:text="Try a different code or status filter."
        android:textColor="@color/slate_500"
        android:textSize="13sp" />
</LinearLayout>

<com.google.android.material.bottomnavigation.BottomNavigationView
    android:id="@+id/bottomNav"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:background="@color/white"
    app:menu="@menu/bottom_nav_prosumer"
    app:itemIconTint="@color/bottom_nav_item"
    app:itemTextColor="@color/bottom_nav_item"

```



```

        android:textColor="#8CFFFFFF"
        android:textSize="12sp" />
    </LinearLayout>
</LinearLayout>

<TextView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="28dp"
    android:text="Trade stored sunlight across the grid."
    android:textColor="@color/white"
    android:textSize="19sp"
    android:textStyle="bold"
    android:lineSpacingExtra="2dp" />

<!-- Credential card -->
<com.google.android.material.card.MaterialCardView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="24dp"
    app:cardCornerRadius="24dp"
    app:cardElevation="0dp"
    app:cardBackgroundColor="@color/white">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="24dp">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Welcome back"
            android:textColor="@color/slate_900"
            android:textSize="22sp"
            android:textStyle="bold" />

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginTop="4dp"
            android:layout_marginBottom="20dp"
            android:text="Sign in with your NIC or username"
            android:textColor="@color/slate_500"
            android:textSize="13sp" />

    <!-- Inline error banner -->
    <LinearLayout
        android:id="@+id/errorBanner"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:background="@drawable/bg_pill_rose"
        android:padding="12dp"
        android:layout_marginBottom="14dp"
        android:gravity="center_vertical"
        android:visibility="gone">

        <ImageView
            android:layout_width="18dp"
            android:layout_height="18dp"
            android:layout_marginEnd="10dp"
            android:src="@drawable/ic_alert"
            app:tint="@color/rose_600"
            android:contentDescription="@null" />

        <TextView
            android:id="@+id/textError"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:textColor="@color/rose_700"
            android:textSize="13sp" />
    </LinearLayout>

```

```

<com.google.android.material.textfield.TextInputLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Username or NIC"
    app:startIconDrawable="@drawable/ic_person"
    app:startIconTint="@color/slate_400">

    <com.google.android.material.textfield.TextInputEditText
        android:id="@+id/inputIdentifier"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:inputType="text"
        android:textSize="15sp" />
</com.google.android.material.textfield.TextInputLayout>

<com.google.android.material.textfield.TextInputLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="12dp"
    android:hint="Password"
    app:startIconDrawable="@drawable/ic_shield"
    app:startIconTint="@color/slate_400"
    app:endIconMode="password_toggle">

    <com.google.android.material.textfield.TextInputEditText
        android:id="@+id/inputPassword"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:inputType="textPassword"
        android:textSize="15sp" />
</com.google.android.material.textfield.TextInputLayout>

<com.google.android.material.button.MaterialButton
    android:id="@+id/buttonLogin"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="20dp"
    android:text="Sign in" />

<View
    android:layout_width="match_parent"
    android:layout_height="1dp"
    android:layout_marginTop="20dp"
    android:background="@color/slate_200" />

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="14dp"
    android:orientation="horizontal"
    android:gravity="center">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="New prosumer?"
        android:textColor="@color/slate_500"
        android:textSize="14sp" />

    <TextView
        android:id="@+id/textGoRegister"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginStart="6dp"
        android:padding="4dp"
        android:text="Create an account"
        android:textColor="@color/brand_700"
        android:textSize="14sp"
        android:textStyle="bold" />
</LinearLayout>
</LinearLayout>
</com.google.android.material.card.MaterialCardView>

```

```

        <TextView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="24dp"
            android:gravity="center"
            android:text="SE4040 · Smart Solar Microgrid Trading System"
            android:textColor="#59FFFFFF"
            android:textSize="11sp" />
    </LinearLayout>
</ScrollView>
</FrameLayout>

```

mobile/app/src/main/res/layout/activity_map.xml

```

<?xml version="1.0" encoding="utf-8"?>
<!-- Nearby microgrid nodes plotted on Google Maps from their stored GPS
coordinates, with a floating header over the map. -->
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/page_background">

    <fragment
        android:id="@+id/mapFragment"
        android:name="com.google.android.gms.maps.SupportMapFragment"
        android:layout_width="match_parent"
        android:layout_height="match_parent" />

    <!-- Floating header card -->
    <com.google.android.material.card.MaterialCardView
        android:id="@+id/headerCard"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_gravity="top"
        android:layout_margin="16dp"
        app:cardCornerRadius="18dp"
        app:cardElevation="8dp"
        app:cardBackgroundColor="@color/white">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="horizontal"
            android:gravity="center_vertical"
            android:padding="14dp">

            <FrameLayout
                android:layout_width="42dp"
                android:layout_height="42dp"
                android:background="@drawable/bg_tile_brand">

                <ImageView
                    android:layout_width="20dp"
                    android:layout_height="20dp"
                    android:layout_gravity="center"
                    android:src="@drawable/ic_map"
                    app:tint="@color/brand_700"
                    android:contentDescription="@null" />
            </FrameLayout>

            <LinearLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:layout_marginStart="14dp"
                android:orientation="vertical">

                <TextView
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:text="Nearby microgrid nodes"
                    android:textColor="@color/slate_900"
                    android:textSize="15sp"

```



```

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:gravity="center_vertical">

    <FrameLayout
        android:layout_width="44dp"
        android:layout_height="44dp"
        android:background="@drawable/bg_brand_gradient">

        <ImageView
            android:layout_width="22dp"
            android:layout_height="22dp"
            android:layout_gravity="center"
            android:src="@drawable/ic_gauge"
            app:tint="@color/white"
            android:contentDescription="@null" />
        </FrameLayout>

    <LinearLayout
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:layout_marginStart="12dp"
        android:orientation="vertical">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Grid operator"
            android:textColor="#8CFFFFFF"
            android:textSize="12sp" />

        <TextView
            android:id="@+id/textWelcome"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:maxLines="1"
            android:ellipsize="end"
            android:textColor="@color/white"
            android:textSize="18sp"
            android:textStyle="bold" />
        </LinearLayout>

    <ImageView
        android:id="@+id/buttonLogout"
        android:layout_width="40dp"
        android:layout_height="40dp"
        android:background="@drawable/bg_tile_glass"
        android:padding="10dp"
        android:src="@drawable/ic_logout"
        app:tint="#B3FFFFFF"
        android:contentDescription="Sign out" />
    </LinearLayout>

    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="22dp"
        android:text="Verify & finalise transfers"
        android:textColor="@color/white"
        android:textSize="24sp"
        android:textStyle="bold" />

    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="4dp"
        android:text="Scan a prosumer's QR code to confirm their booking at the node."
        android:textColor="#99FFFFFF"
        android:textSize="13sp"
        android:lineSpacingExtra="2dp" />
    </LinearLayout>

```

```

</FrameLayout>

<!-- KPI tiles -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:layout_marginTop="-34dp"
    android:paddingStart="20dp"
    android:paddingEnd="20dp">

    <com.google.android.material.card.MaterialCardView
        style="@style/Widget.Solar.Card"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:layout_marginEnd="6dp"
        app:cardElevation="6dp">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:padding="16dp">

            <FrameLayout
                android:layout_width="36dp"
                android:layout_height="36dp"
                android:background="@drawable/bg_tile_solar">

                <ImageView
                    android:layout_width="18dp"
                    android:layout_height="18dp"
                    android:layout_gravity="center"
                    android:src="@drawable/ic_clock"
                    app:tint="@color/solar_600"
                    android:contentDescription="@null" />
            </FrameLayout>

            <TextView
                android:id="@+id/textPendingCount"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:layout_marginTop="10dp"
                android:textColor="@color/slate_900"
                android:textSize="30sp"
                android:textStyle="bold" />

            <TextView
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="Pending"
                android:textColor="@color/slate_500"
                android:textSize="12sp" />
        </LinearLayout>
    </com.google.android.material.card.MaterialCardView>

    <com.google.android.material.card.MaterialCardView
        style="@style/Widget.Solar.Card"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:layout_marginStart="6dp"
        app:cardElevation="6dp">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:padding="16dp">

            <FrameLayout
                android:layout_width="36dp"
                android:layout_height="36dp"

```

```

        android:background="@drawable/bg_tile_brand">

        <ImageView
            android:layout_width="18dp"
            android:layout_height="18dp"
            android:layout_gravity="center"
            android:src="@drawable/ic_calendar"
            app:tint="@color/brand_700"
            android:contentDescription="@null" />
    </FrameLayout>

    <TextView
        android:id="@+id/textApprovedCount"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="10dp"
        android:textColor="@color/slate_900"
        android:textSize="30sp"
        android:textStyle="bold" />

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Approved"
        android:textColor="@color/slate_500"
        android:textSize="12sp" />
    </LinearLayout>
</com.google.android.material.card.MaterialCardView>
</LinearLayout>

<!-- Scan CTA -->
<com.google.android.material.button.MaterialButton
    android:id="@+id/buttonScan"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="18dp"
    android:layout_marginStart="20dp"
    android:layout_marginEnd="20dp"
    android:paddingTop="18dp"
    android:paddingBottom="18dp"
    android:text="Scan prosumer QR code"
    app:icon="@drawable/ic_scan"
    app:iconGravity="textStart"
    app:iconTint="@color/white" />

<TextView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="26dp"
    android:paddingStart="20dp"
    android:paddingEnd="20dp"
    android:text="Awaiting approval"
    android:textColor="@color/slate_900"
    android:textSize="16sp"
    android:textStyle="bold" />

<TextView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="2dp"
    android:paddingStart="20dp"
    android:paddingEnd="20dp"
    android:text="Tap a booking to approve it and issue its QR token."
    android:textColor="@color/slate_500"
    android:textSize="13sp" />

<androidx.recyclerview.widget.RecyclerView
    android:id="@+id/recyclerPending"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="12dp"
    android:paddingStart="20dp"
    android:paddingEnd="20dp"
    android:paddingBottom="24dp"
    android:clipToPadding="false" />

```

```

<LinearLayout
    android:id="@+id/emptyState"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="36dp"
    android:visibility="gone">

    <FrameLayout
        android:layout_width="60dp"
        android:layout_height="60dp"
        android:background="@drawable/bg_tile_brand">

        <ImageView
            android:layout_width="28dp"
            android:layout_height="28dp"
            android:layout_gravity="center"
            android:src="@drawable/ic_check_circle"
            app:tint="@color/brand_600"
            android:contentDescription="@null" />
    </FrameLayout>

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="14dp"
        android:text="Queue is clear"
        android:textColor="@color/slate_700"
        android:textSize="15sp"
        android:textStyle="bold" />

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="4dp"
        android:gravity="center"
        android:text="Every booking has been actioned."
        android:textColor="@color/slate_500"
        android:textSize="13sp" />
</LinearLayout>
</LinearLayout>
</ScrollView>
</androidx.swiperefreshlayout.widget.SwipeRefreshLayout>
</LinearLayout>

```

mobile/app/src/main/res/layout/activity_profile.xml

```

<?xml version="1.0" encoding="utf-8"?>
<!-- Prosumer self-service profile: editable contact details plus the
account-deactivation request. -->
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/page_background">

    <ScrollView
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:fillViewport="true"
        android:scrollbars="none">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical">

            <!-- Identity header -->
            <FrameLayout
                android:id="@+id/heroSection"

```

```

android:layout_width="match_parent"
android:layout_height="wrap_content"
android:background="@drawable/bg_hero_gradient_rounded">

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:paddingStart="20dp"
    android:paddingEnd="20dp"
    android:paddingTop="24dp"
    android:paddingBottom="34dp">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="PROSUMER ACCOUNT"
        android:textColor="#8CFFFFFF"
        android:textSize="11sp"
        android:textStyle="bold"
        android:letterSpacing="0.08" />

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="14dp"
        android:orientation="horizontal"
        android:gravity="center_vertical">

        <FrameLayout
            android:layout_width="72dp"
            android:layout_height="72dp">

            <FrameLayout
                android:layout_width="64dp"
                android:layout_height="64dp"
                android:layout_gravity="center"
                android:background="@drawable/bg_solar_gradient">

                <TextView
                    android:id="@+id/textAvatar"
                    android:layout_width="match_parent"
                    android:layout_height="match_parent"
                    android:gravity="center"
                    android:textColor="@color/ink_900"
                    android:textSize="22sp"
                    android:textStyle="bold" />

                <ImageView
                    android:id="@+id/imageAvatar"
                    android:layout_width="match_parent"
                    android:layout_height="match_parent"
                    android:scaleType="centerCrop"
                    android:visibility="gone"
                    android:contentDescription="@null" />
            </FrameLayout>

            <ImageView
                android:id="@+id/buttonEditPhoto"
                android:layout_width="26dp"
                android:layout_height="26dp"
                android:layout_gravity="bottom|end"
                android:background="@drawable/bg_circle_white"
                android:padding="6dp"
                android:src="@drawable/ic_edit"
                app:tint="@color/brand_700"
                android:contentDescription="Change profile picture" />
            </FrameLayout>

        <LinearLayout
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:layout_marginStart="16dp"

```

```

        android:orientation="vertical">

        <TextView
            android:id="@+id/textName"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:maxLines="1"
            android:ellipsize="end"
            android:textColor="@color/white"
            android:textSize="19sp"
            android:textStyle="bold" />

        <TextView
            android:id="@+id/textNic"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginTop="3dp"
            android:textColor="#B3FFFFFF"
            android:textSize="12sp" />

        <LinearLayout
            android:id="@+id/statusPill"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginTop="9dp"
            android:orientation="horizontal"
            android:gravity="center_vertical"
            android:background="@drawable/bg_pill_glass"
            android:paddingStart="10dp" android:paddingEnd="12dp"
            android:paddingTop="5dp" android:paddingBottom="5dp">

            <ImageView
                android:id="@+id/iconStatus"
                android:layout_width="12dp"
                android:layout_height="12dp"
                android:layout_marginEnd="5dp"
                android:src="@drawable/ic_check_circle"
                app:tint="@color/brand_300"
                android:contentDescription="@null" />

            <TextView
                android:id="@+id/textStatus"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:textColor="@color/brand_200"
                android:textSize="12sp"
                android:textStyle="bold" />
        </LinearLayout>
    </LinearLayout>

    <TextView
        android:id="@+id/textMemberSince"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="18dp"
        android:textColor="#8CFFFFFF"
        android:textSize="12sp" />
</LinearLayout>
</FrameLayout>

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:layout_marginTop="-30dp"
    android:paddingStart="20dp"
    android:paddingEnd="20dp"
    android:paddingTop="20dp"
    android:paddingBottom="96dp">

    <!-- Inline error banner -->
    <LinearLayout
        android:id="@+id/errorBanner"

```

```

        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:background="@drawable/bg_pill_rose"
        android:padding="12dp"
        android:layout_marginBottom="12dp"
        android:gravity="center_vertical"
        android:visibility="gone">

        <ImageView
            android:layout_width="18dp"
            android:layout_height="18dp"
            android:layout_marginEnd="10dp"
            android:src="@drawable/ic_alert"
            app:tint="@color/rose_600"
            android:contentDescription="@null" />

        <TextView
            android:id="@+id/textError"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:textColor="@color/rose_700"
            android:textSize="13sp" />
    </LinearLayout>

    <com.google.android.material.card.MaterialCardView
        style="@style/Widget.Solar.Card"
        android:layout_width="match_parent"
        android:layout_height="wrap_content">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:padding="20dp">

            <TextView
                style="@style/Text.Solar.Label"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:layout_marginBottom="12dp"
                android:text="Your details" />

            <com.google.android.material.textfield.TextInputLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:hint="Full name">

                <com.google.android.material.textfield.TextInputEditText
                    android:id="@+id/inputFullName"
                    android:layout_width="match_parent"
                    android:layout_height="wrap_content"
                    android:inputType="textPersonName" />
            </com.google.android.material.textfield.TextInputLayout>

            <com.google.android.material.textfield.TextInputLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:layout_marginTop="12dp"
                android:hint="Email">

                <com.google.android.material.textfield.TextInputEditText
                    android:id="@+id/inputEmail"
                    android:layout_width="match_parent"
                    android:layout_height="wrap_content"
                    android:inputType="textEmailAddress" />
            </com.google.android.material.textfield.TextInputLayout>

            <com.google.android.material.textfield.TextInputLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:layout_marginTop="12dp"
                android:hint="Phone">

```

```

        <com.google.android.material.textfield.TextInputEditText
            android:id="@+id/inputPhone"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:inputType="phone" />
    </com.google.android.material.textfield.TextInputLayout>

    <com.google.android.material.textfield.TextInputLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="12dp"
        android:hint="Address">

        <com.google.android.material.textfield.TextInputEditText
            android:id="@+id/inputAddress"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:inputType="textPostalAddress" />
    </com.google.android.material.textfield.TextInputLayout>

    <com.google.android.material.button.MaterialButton
        android:id="@+id/buttonSave"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="18dp"
        android:text="Save changes"
        app:icon="@drawable/ic_check"
        app:iconGravity="textStart"
        app:iconTint="@color/white" />
</LinearLayout>
</com.google.android.material.card.MaterialCardView>

```

```

<!-- Danger zone -->
<com.google.android.material.card.MaterialCardView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="16dp"
    app:cardCornerRadius="20dp"
    app:cardElevation="0dp"
    app:cardBackgroundColor="@color/rose_50"
    app:strokeColor="#FECDD3"
    app:strokeWidth="1dp">

```

```

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="20dp">

```

```

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Deactivate account"
            android:textColor="@color/rose_700"
            android:textSize="15sp"
            android:textStyle="bold" />

```

```

        <TextView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="4dp"
            android:text="You will be signed out immediately. Only a Backoffice officer can
reactivate your account."
            android:textColor="@color/rose_600"
            android:textSize="13sp"
            android:lineSpacingExtra="2dp" />

```

```

    <com.google.android.material.button.MaterialButton
        android:id="@+id/buttonDeactivate"
        style="@style/Widget.Solar.Button.Danger"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="16dp"

```

```

                android:text="Request deactivation" />
            </LinearLayout>
        </com.google.android.material.card.MaterialCardView>
    </LinearLayout>
</LinearLayout>
</ScrollView>

<com.google.android.material.bottomnavigation.BottomNavigationView
    android:id="@+id/bottomNav"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_gravity="bottom"
    android:background="@color/white"
    app:menu="@menu/bottom_nav_prosumer"
    app:itemIconTint="@color/bottom_nav_item"
    app:itemTextColor="@color/bottom_nav_item"
    app:itemActiveIndicatorStyle="@style/Widget.Solar.BottomNav.Indicator"
    app:elevation="0dp"
    app:labelVisibilityMode="labeled" />
</FrameLayout>

```

mobile/app/src/main/res/layout/activity_prosumer_home.xml

```

<?xml version="1.0" encoding="utf-8"?>
<!-- Prosumer dashboard: gradient hero, KPI tiles, upcoming bookings and bottom nav. -->
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:background="@color/page_background">

    <androidx.swiperefreshlayout.widget.SwipeRefreshLayout
        android:id="@+id/swipeRefresh"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1">

        <ScrollView
            android:layout_width="match_parent"
            android:layout_height="match_parent"
            android:scrollbars="none"
            android:clipToPadding="false">

            <LinearLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:orientation="vertical">

                <!-- Hero -->
                <FrameLayout
                    android:id="@+id/heroSection"
                    android:layout_width="match_parent"
                    android:layout_height="wrap_content"
                    android:background="@drawable/bg_hero_gradient_rounded">

                    <LinearLayout
                        android:layout_width="match_parent"
                        android:layout_height="wrap_content"
                        android:orientation="vertical"
                        android:paddingStart="20dp"
                        android:paddingEnd="20dp"
                        android:paddingTop="24dp"
                        android:paddingBottom="52dp">

                        <LinearLayout
                            android:layout_width="match_parent"
                            android:layout_height="wrap_content"
                            android:orientation="horizontal"
                            android:gravity="center_vertical">

                            <FrameLayout

```

```

        android:layout_width="44dp"
        android:layout_height="44dp"
        android:background="@drawable/bg_solar_gradient">

        <TextView
            android:id="@+id/textAvatar"
            android:layout_width="match_parent"
            android:layout_height="match_parent"
            android:gravity="center"
            android:textColor="@color/ink_900"
            android:textSize="15sp"
            android:textStyle="bold" />

        <ImageView
            android:id="@+id/imageAvatar"
            android:layout_width="match_parent"
            android:layout_height="match_parent"
            android:scaleType="centerCrop"
            android:visibility="gone"
            android:contentDescription="@null" />
    </FrameLayout>

    <LinearLayout
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:layout_marginStart="12dp"
        android:orientation="vertical">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Welcome back"
            android:textColor="#8CFFFFFF"
            android:textSize="12sp" />

        <TextView
            android:id="@+id/textWelcome"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:maxLines="1"
            android:ellipsize="end"
            android:textColor="@color/white"
            android:textSize="18sp"
            android:textStyle="bold" />
    </LinearLayout>

    <ImageView
        android:id="@+id/buttonLogout"
        android:layout_width="40dp"
        android:layout_height="40dp"
        android:background="@drawable/bg_tile_glass"
        android:padding="10dp"
        android:src="@drawable/ic_logout"
        app:tint="#B3FFFFFF"
        android:contentDescription="Sign out" />
</LinearLayout>

    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="22dp"
        android:text="Your energy bookings"
        android:textColor="@color/white"
        android:textSize="24sp"
        android:textStyle="bold" />

    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="4dp"
        android:text="Reserve a slot at a nearby microgrid node and track every transfer."
        android:textColor="#99FFFFFF"
        android:textSize="13sp"

```

```

        android:lineSpacingExtra="2dp" />
    </LinearLayout>
</FrameLayout>

<!-- KPI tiles (pulled up over the hero) -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:layout_marginTop="-34dp"
    android:paddingStart="20dp"
    android:paddingEnd="20dp">

    <com.google.android.material.card.MaterialCardView
        style="@style/Widget.Solar.Card"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:layout_marginEnd="6dp">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:padding="16dp">

            <FrameLayout
                android:layout_width="36dp"
                android:layout_height="36dp"
                android:background="@drawable/bg_tile_brand">

                <ImageView
                    android:layout_width="18dp"
                    android:layout_height="18dp"
                    android:layout_gravity="center"
                    android:src="@drawable/ic_check_circle"
                    app:tint="@color/brand_700"
                    android:contentDescription="@null" />
            </FrameLayout>

            <TextView
                android:id="@+id/textActiveCount"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:layout_marginTop="10dp"
                android:textColor="@color/slate_900"
                android:textSize="30sp"
                android:textStyle="bold" />

            <TextView
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="Approved"
                android:textColor="@color/slate_500"
                android:textSize="12sp" />
        </LinearLayout>
    </com.google.android.material.card.MaterialCardView>

    <com.google.android.material.card.MaterialCardView
        style="@style/Widget.Solar.Card"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:layout_marginStart="6dp">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:padding="16dp">

            <FrameLayout
                android:layout_width="36dp"
                android:layout_height="36dp"

```

```

        android:background="@drawable/bg_tile_solar">

        <ImageView
            android:layout_width="18dp"
            android:layout_height="18dp"
            android:layout_gravity="center"
            android:src="@drawable/ic_clock"
            app:tint="@color/solar_600"
            android:contentDescription="@null" />
    </FrameLayout>

    <TextView
        android:id="@+id/textPendingCount"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="10dp"
        android:textColor="@color/slate_900"
        android:textSize="30sp"
        android:textStyle="bold" />

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Pending"
        android:textColor="@color/slate_500"
        android:textSize="12sp" />
    </LinearLayout>
</com.google.android.material.card.MaterialCardView>
</LinearLayout>

<!-- Primary action -->
<com.google.android.material.button.MaterialButton
    android:id="@+id/buttonNewReservation"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="18dp"
    android:layout_marginStart="20dp"
    android:layout_marginEnd="20dp"
    android:text="Reserve an energy slot"
    app:icon="@drawable/ic_plus"
    app:iconGravity="textStart"
    app:iconTint="@color/white" />

<!-- Upcoming list -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:gravity="center_vertical"
    android:layout_marginTop="26dp"
    android:paddingStart="20dp"
    android:paddingEnd="20dp">

    <TextView
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:text="Upcoming reservations"
        android:textColor="@color/slate_900"
        android:textSize="16sp"
        android:textStyle="bold" />

    <TextView
        android:id="@+id/textSeeAll"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:padding="4dp"
        android:text="View history"
        android:textColor="@color/brand_700"
        android:textSize="13sp"
        android:textStyle="bold" />
</LinearLayout>

<androidx.recyclerview.widget.RecyclerView

```

```

        android:id="@+id/recyclerUpcoming"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="10dp"
        android:paddingStart="20dp"
        android:paddingEnd="20dp"
        android:paddingBottom="20dp"
        android:clipToPadding="false" />

<!-- Empty state -->
<LinearLayout
    android:id="@+id/emptyState"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="36dp"
    android:visibility="gone">

    <FrameLayout
        android:layout_width="60dp"
        android:layout_height="60dp"
        android:background="@drawable/bg_tile_slate">

        <ImageView
            android:layout_width="28dp"
            android:layout_height="28dp"
            android:layout_gravity="center"
            android:src="@drawable/ic_calendar"
            app:tint="@color/slate_400"
            android:contentDescription="@null" />
    </FrameLayout>

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="14dp"
        android:text="No upcoming reservations"
        android:textColor="@color/slate_700"
        android:textSize="15sp"
        android:textStyle="bold" />

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="4dp"
        android:gravity="center"
        android:text="Book a slot at a nearby node to start trading energy."
        android:textColor="@color/slate_500"
        android:textSize="13sp" />
    </LinearLayout>
</LinearLayout>
</ScrollView>
</androidx.swiperefreshlayout.widget.SwipeRefreshLayout>

<com.google.android.material.bottomnavigation.BottomNavigationView
    android:id="@+id/bottomNav"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:background="@color/white"
    app:menu="@menu/bottom_nav_prosumer"
    app:itemIconTint="@color/bottom_nav_item"
    app:itemTextColor="@color/bottom_nav_item"
    app:itemActiveIndicatorStyle="@style/Widget.Solar.BottomNav.Indicator"
    app:elevation="0dp"
    app:labelVisibilityMode="labeled" />
</LinearLayout>

```

mobile/app/src/main/res/layout/activity_register.xml

```

<?xml version="1.0" encoding="utf-8"?>
<!-- Prosumer self-registration. NIC is the account's primary key. -->
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"

```

```

xmlns:app="http://schemas.android.com/apk/res-auto"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:background="@color/page_background">

<ScrollView
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:fillViewport="true"
    android:scrollbars="none">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical">

        <!-- Hero header -->
        <FrameLayout
            android:id="@+id/heroSection"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:background="@drawable/bg_hero_gradient_rounded">

            <LinearLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:orientation="vertical"
                android:paddingStart="24dp"
                android:paddingEnd="24dp"
                android:paddingTop="28dp"
                android:paddingBottom="36dp">

                <FrameLayout
                    android:layout_width="44dp"
                    android:layout_height="44dp"
                    android:background="@drawable/bg_tile_glass">

                    <ImageView
                        android:layout_width="22dp"
                        android:layout_height="22dp"
                        android:layout_gravity="center"
                        android:src="@drawable/ic_person"
                        app:tint="@color/brand_200"
                        android:contentDescription="@null" />
                </FrameLayout>

                <TextView
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:layout_marginTop="16dp"
                    android:text="Create your account"
                    android:textColor="@color/white"
                    android:textSize="26sp"
                    android:textStyle="bold" />

                <TextView
                    android:layout_width="match_parent"
                    android:layout_height="wrap_content"
                    android:layout_marginTop="6dp"
                    android:text="Register as a solar prosumer to reserve energy slots at microgrid nodes."
                    android:textColor="#99FFFFFF"
                    android:textSize="13sp"
                    android:lineSpacingExtra="2dp" />
                </LinearLayout>
            </FrameLayout>

            <LinearLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:orientation="vertical"
                android:padding="20dp">

                <!-- Inline error banner -->
                <LinearLayout

```

```

        android:id="@+id/errorBanner"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:background="@drawable/bg_pill_rose"
        android:padding="12dp"
        android:layout_marginBottom="14dp"
        android:gravity="center_vertical"
        android:visibility="gone">

        <ImageView
            android:layout_width="18dp"
            android:layout_height="18dp"
            android:layout_marginEnd="10dp"
            android:src="@drawable/ic_alert"
            app:tint="@color/rose_600"
            android:contentDescription="@null" />

        <TextView
            android:id="@+id/textError"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:textColor="@color/rose_700"
            android:textSize="13sp" />
    </LinearLayout>

    <com.google.android.material.card.MaterialCardView
        style="@style/Widget.Solar.Card"
        android:layout_width="match_parent"
        android:layout_height="wrap_content">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:padding="20dp">

            <TextView
                style="@style/Text.Solar.Label"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:layout_marginBottom="12dp"
                android:text="Identity" />

            <com.google.android.material.textfield.TextInputLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:hint="NIC (National Identity Card)">

                <com.google.android.material.textfield.TextInputEditText
                    android:id="@+id/inputNic"
                    android:layout_width="match_parent"
                    android:layout_height="wrap_content"
                    android:inputType="text" />
            </com.google.android.material.textfield.TextInputLayout>

            <com.google.android.material.textfield.TextInputLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:layout_marginTop="12dp"
                android:hint="Full name">

                <com.google.android.material.textfield.TextInputEditText
                    android:id="@+id/inputFullName"
                    android:layout_width="match_parent"
                    android:layout_height="wrap_content"
                    android:inputType="textPersonName" />
            </com.google.android.material.textfield.TextInputLayout>

            <TextView
                style="@style/Text.Solar.Label"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:layout_marginTop="20dp"

```

```

        android:layout_marginBottom="12dp"
        android:text="Contact" />

<com.google.android.material.textfield.TextInputLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Email">

    <com.google.android.material.textfield.TextInputEditText
        android:id="@+id/inputEmail"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:inputType="textEmailAddress" />
</com.google.android.material.textfield.TextInputLayout>

<com.google.android.material.textfield.TextInputLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="12dp"
    android:hint="Phone">

    <com.google.android.material.textfield.TextInputEditText
        android:id="@+id/inputPhone"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:inputType="phone" />
</com.google.android.material.textfield.TextInputLayout>

<com.google.android.material.textfield.TextInputLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="12dp"
    android:hint="Address">

    <com.google.android.material.textfield.TextInputEditText
        android:id="@+id/inputAddress"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:inputType="textPostalAddress" />
</com.google.android.material.textfield.TextInputLayout>

<TextView
    style="@style/Text.Solar.Label"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="20dp"
    android:layout_marginBottom="12dp"
    android:text="Security" />

<com.google.android.material.textfield.TextInputLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Password"
    app:endIconMode="password_toggle">

    <com.google.android.material.textfield.TextInputEditText
        android:id="@+id/inputPassword"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:inputType="textPassword" />
</com.google.android.material.textfield.TextInputLayout>
</LinearLayout>
</com.google.android.material.card.MaterialCardView>

<!-- Activation notice -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="14dp"
    android:orientation="horizontal"
    android:background="@drawable/bg_pill_amber"
    android:padding="14dp">

```

```

        <ImageView
            android:layout_width="18dp"
            android:layout_height="18dp"
            android:layout_marginEnd="10dp"
            android:src="@drawable/ic_clock"
            app:tint="@color/solar_600"
            android:contentDescription="@null" />

        <TextView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Your account stays pending until a Backoffice officer activates it."
            android:textColor="@color/solar_600"
            android:textSize="13sp"
            android:lineSpacingExtra="2dp" />
    </LinearLayout>

    <com.google.android.material.button.MaterialButton
        android:id="@+id/buttonRegister"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="18dp"
        android:text="Create account" />
</LinearLayout>
</LinearLayout>
</ScrollView>
</FrameLayout>

```

mobile/app/src/main/res/layout/activity_reservation_detail.xml

```

<?xml version="1.0" encoding="utf-8"?>
<!-- One reservation in full: status, details, the transaction QR code once
      approved, and the modify/cancel actions. -->
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/page_background">

    <ScrollView
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:fillViewport="true"
        android:scrollbars="none">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical">

            <!-- Status hero -->
            <FrameLayout
                android:id="@+id/heroSection"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:background="@drawable/bg_hero_gradient_rounded">

                <LinearLayout
                    android:layout_width="match_parent"
                    android:layout_height="wrap_content"
                    android:orientation="vertical"
                    android:gravity="center_horizontal"
                    android:paddingStart="20dp"
                    android:paddingEnd="20dp"
                    android:paddingTop="28dp"
                    android:paddingBottom="46dp">

                    <TextView
                        android:id="@+id/textStatus"
                        android:layout_width="wrap_content"
                        android:layout_height="wrap_content"
                        android:background="@drawable/bg_pill_glass"
                        android:paddingStart="14dp" android:paddingEnd="14dp"

```

[illegible]

```

        android:text="Scheduled"
        android:textColor="@color/slate_500"
        android:textSize="12sp" />

        <TextView
            android:id="@+id/textScheduled"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:textColor="@color/slate_900"
            android:textSize="15sp"
            android:textStyle="bold" />
    </LinearLayout>
</LinearLayout>

<View
    android:layout_width="match_parent"
    android:layout_height="1dp"
    android:layout_marginTop="16dp"
    android:layout_marginBottom="16dp"
    android:background="@color/slate_200" />

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:gravity="center_vertical">

    <FrameLayout
        android:layout_width="40dp"
        android:layout_height="40dp"
        android:background="@drawable/bg_tile_solar">

        <ImageView
            android:layout_width="18dp"
            android:layout_height="18dp"
            android:layout_gravity="center"
            android:src="@drawable/ic_bolt"
            app:tint="@color/solar_600"
            android:contentDescription="@null" />
    </FrameLayout>

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginStart="14dp"
        android:orientation="vertical">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Energy"
            android:textColor="@color/slate_500"
            android:textSize="12sp" />

        <TextView
            android:id="@+id/textEnergy"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:textColor="@color/slate_900"
            android:textSize="15sp"
            android:textStyle="bold" />
    </LinearLayout>
    </LinearLayout>
</LinearLayout>
</com.google.android.material.card.MaterialCardView>

<!-- QR card -->
<com.google.android.material.card.MaterialCardView
    android:id="@+id/qrCard"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="16dp"
    android:visibility="gone"
    app:cardCornerRadius="20dp"

```

```
app:cardElevation="0dp"
app:cardBackgroundColor="@color/white"
app:strokeColor="@color/brand_200"
app:strokeWidth="1dp">
```

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:gravity="center_horizontal"
    android:padding="20dp">
```

```
    <LinearLayout
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:gravity="center_vertical"
        android:background="@drawable/bg_pill_emerald"
        android:paddingStart="12dp" android:paddingEnd="12dp"
        android:paddingTop="6dp" android:paddingBottom="6dp">
```

```
        <ImageView
            android:layout_width="14dp"
            android:layout_height="14dp"
            android:layout_marginEnd="6dp"
            android:src="@drawable/ic_qr"
            app:tint="@color/brand_700"
            android:contentDescription="@null" />
```

```
        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Transaction QR"
            android:textColor="@color/brand_700"
            android:textSize="11sp"
            android:textStyle="bold" />
```

```
    </LinearLayout>
```

```
    <ImageView
        android:id="@+id/imageQr"
        android:layout_width="230dp"
        android:layout_height="230dp"
        android:layout_marginTop="16dp"
        android:contentDescription="Transaction QR code" />
```

```
    <TextView
        android:id="@+id/textQrHint"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="14dp"
        android:gravity="center"
        android:text="Show this code to the Grid Operator at the station to complete your
```

energy transfer."

```
        android:textColor="@color/slate_500"
        android:textSize="13sp"
        android:lineSpacingExtra="2dp" />
```

```
    </LinearLayout>
```

```
</com.google.android.material.card.MaterialCardView>
```

```
<!-- Inline error banner -->
```

```
<LinearLayout
    android:id="@+id/errorBanner"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:background="@drawable/bg_pill_rose"
    android:padding="12dp"
    android:layout_marginTop="14dp"
    android:gravity="center_vertical"
    android:visibility="gone">
```

```
    <ImageView
        android:layout_width="18dp"
        android:layout_height="18dp"
```

```

        android:layout_marginEnd="10dp"
        android:src="@drawable/ic_alert"
        app:tint="@color/rose_600"
        android:contentDescription="@null" />

        <TextView
            android:id="@+id/textError"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:textColor="@color/rose_700"
            android:textSize="13sp" />
    </LinearLayout>

    <com.google.android.material.button.MaterialButton
        android:id="@+id/buttonEdit"
        style="@style/Widget.Solar.Button.Outlined"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="18dp"
        android:text="Modify reservation"
        app:icon="@drawable/ic_edit"
        app:iconGravity="textStart"
        app:iconTint="@color/slate_500" />

    <com.google.android.material.button.MaterialButton
        android:id="@+id/buttonCancel"
        style="@style/Widget.Solar.Button.Danger"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="10dp"
        android:text="Cancel reservation" />
</LinearLayout>
</LinearLayout>
</ScrollView>
</FrameLayout>

```

mobile/app/src/main/res/layout/activity_reservation_form.xml

```

<?xml version="1.0" encoding="utf-8"?>
<!-- Create or edit an energy slot reservation. The 7-day window and the
12-hour change rule are enforced by the API. -->
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/page_background">

    <ScrollView
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:fillViewport="true"
        android:scrollbars="none">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical">

            <!-- Header -->
            <FrameLayout
                android:id="@+id/heroSection"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:background="@drawable/bg_hero_gradient_rounded">

                <LinearLayout
                    android:layout_width="match_parent"
                    android:layout_height="wrap_content"
                    android:orientation="vertical"
                    android:paddingStart="20dp"
                    android:paddingEnd="20dp"
                    android:paddingTop="26dp"
                    android:paddingBottom="42dp">

```

```

<FrameLayout
    android:layout_width="42dp"
    android:layout_height="42dp"
    android:background="@drawable/bg_tile_glass">

    <ImageView
        android:layout_width="20dp"
        android:layout_height="20dp"
        android:layout_gravity="center"
        android:src="@drawable/ic_bolt"
        app:tint="@color/brand_200"
        android:contentDescription="@null" />
</FrameLayout>

<TextView
    android:id="@+id/textTitle"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="14dp"
    android:text="New reservation"
    android:textColor="@color/white"
    android:textSize="24sp"
    android:textStyle="bold" />

<TextView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="4dp"
    android:text="Pick a microgrid node and an available energy slot."
    android:textColor="#99FFFFFF"
    android:textSize="13sp" />
</LinearLayout>
</FrameLayout>

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:layout_marginTop="-28dp"
    android:padding="20dp">

<!-- Inline error banner -->
<LinearLayout
    android:id="@+id/errorBanner"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:background="@drawable/bg_pill_rose"
    android:padding="12dp"
    android:layout_marginBottom="12dp"
    android:gravity="center_vertical"
    android:visibility="gone">

    <ImageView
        android:layout_width="18dp"
        android:layout_height="18dp"
        android:layout_marginEnd="10dp"
        android:src="@drawable/ic_alert"
        app:tint="@color/rose_600"
        android:contentDescription="@null" />

    <TextView
        android:id="@+id/textError"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:textColor="@color/rose_700"
        android:textSize="13sp" />
</LinearLayout>

<com.google.android.material.card.MaterialCardView
    style="@style/Widget.Solar.Card"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"

```

```

app:cardElevation="6dp">

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="20dp">

    <TextView
        style="@style/Text.Solar.Label"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginBottom="8dp"
        android:text="Where" />

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginBottom="6dp"
        android:text="Station"
        android:textColor="@color/slate_700"
        android:textSize="13sp"
        android:textStyle="bold" />

    <Spinner
        android:id="@+id/spinnerStation"
        android:layout_width="match_parent"
        android:layout_height="52dp"
        android:background="@drawable/bg_search_field"
        android:paddingStart="12dp"
        android:paddingEnd="12dp" />

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="16dp"
        android:layout_marginBottom="6dp"
        android:text="Available slot"
        android:textColor="@color/slate_700"
        android:textSize="13sp"
        android:textStyle="bold" />

    <Spinner
        android:id="@+id/spinnerSlot"
        android:layout_width="match_parent"
        android:layout_height="52dp"
        android:background="@drawable/bg_search_field"
        android:paddingStart="12dp"
        android:paddingEnd="12dp" />

    <View
        android:layout_width="match_parent"
        android:layout_height="1dp"
        android:layout_marginTop="20dp"
        android:layout_marginBottom="16dp"
        android:background="@color/slate_200" />

    <TextView
        style="@style/Text.Solar.Label"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginBottom="8dp"
        android:text="When & how much" />

    <com.google.android.material.button.MaterialButton
        android:id="@+id/buttonPickDateTime"
        style="@style/Widget.Solar.Button.Outlined"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Pick date & time"
        app:icon="@drawable/ic_calendar"
        app:iconGravity="textStart"
        app:iconTint="@color/slate_500" />

```

```

        <com.google.android.material.textfield.TextInputLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="12dp"
            android:hint="Energy amount (kWh)"
            app:suffixText="kWh">

            <com.google.android.material.textfield.TextInputEditText
                android:id="@+id/inputEnergyAmount"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:inputType="numberDecimal" />
        </com.google.android.material.textfield.TextInputLayout>
    </LinearLayout>
</com.google.android.material.card.MaterialCardView>

<!-- Rules notice -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="14dp"
    android:orientation="horizontal"
    android:background="@drawable/bg_pill_amber"
    android:padding="14dp">

    <ImageView
        android:layout_width="18dp"
        android:layout_height="18dp"
        android:layout_marginEnd="10dp"
        android:src="@drawable/ic_clock"
        app:tint="@color/solar_600"
        android:contentDescription="@null" />

    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Bookings must be within 7 days. Changes and cancellations need at least
12 hours' notice."
        android:textColor="@color/solar_600"
        android:textSize="13sp"
        android:lineSpacingExtra="2dp" />
</LinearLayout>

<com.google.android.material.button.MaterialButton
    android:id="@+id/buttonSubmit"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="18dp"
    android:text="Submit reservation"
    app:icon="@drawable/ic_check"
    app:iconGravity="textStart"
    app:iconTint="@color/white" />
</LinearLayout>
</LinearLayout>
</ScrollView>
</FrameLayout>

```

mobile/app/src/main/res/layout/activity_scan.xml

```

<?xml version="1.0" encoding="utf-8"?>
<!-- Live QR scanner: full-bleed camera preview with a viewfinder frame and a
status strip that reports verification progress. -->
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/ink_900">

    <androidx.camera.view.PreviewView
        android:id="@+id/previewView"
        android:layout_width="match_parent"
        android:layout_height="match_parent" />

```

```

<!-- Viewfinder -->
<View
    android:layout_width="250dp"
    android:layout_height="250dp"
    android:layout_gravity="center"
    android:background="@drawable/bg_scan_frame" />

<!-- Top instruction -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_gravity="top"
    android:orientation="vertical"
    android:gravity="center_horizontal"
    android:fitsSystemWindows="true"
    android:paddingStart="24dp"
    android:paddingEnd="24dp"
    android:paddingTop="24dp">

    <LinearLayout
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:gravity="center_vertical"
        android:background="@drawable/bg_pill_glass"
        android:paddingStart="14dp" android:paddingEnd="14dp"
        android:paddingTop="8dp" android:paddingBottom="8dp">

            <ImageView
                android:layout_width="16dp"
                android:layout_height="16dp"
                android:layout_marginEnd="8dp"
                android:src="@drawable/ic_qr"
                app:tint="@color/brand_300"
                android:contentDescription="@null" />

            <TextView
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="Scan mode"
                android:textColor="@color/brand_200"
                android:textSize="12sp"
                android:textStyle="bold" />
        </LinearLayout>

        <TextView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="16dp"
            android:gravity="center"
            android:text="Point the camera at the prosumer's QR code"
            android:textColor="@color/white"
            android:textSize="17sp"
            android:textStyle="bold" />
    </LinearLayout>

<!-- Bottom status strip -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_gravity="bottom"
    android:orientation="vertical"
    android:background="@drawable/bg_scrip_bottom"
    android:gravity="center_horizontal"
    android:fitsSystemWindows="true"
    android:paddingStart="24dp"
    android:paddingEnd="24dp"
    android:paddingTop="48dp"
    android:paddingBottom="28dp">

    <TextView
        android:id="@+id/textStatus"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:gravity="center"

```



```

        android:layout_marginTop="6dp"
        android:gravity="center"
        android:text="This code was issued by the server and matches an approved booking."
        android:textColor="#99FFFFFF"
        android:textSize="13sp"
        android:lineSpacingExtra="2dp" />
    </LinearLayout>
</FrameLayout>

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:layout_marginTop="-30dp"
    android:padding="20dp">

    <com.google.android.material.card.MaterialCardView
        style="@style/Widget.Solar.Card"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        app:cardElevation="6dp">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:padding="20dp">

            <TextView
                style="@style/Text.Solar.Label"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="Reservation" />

            <TextView
                android:id="@+id/textCode"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:layout_marginTop="6dp"
                android:fontFamily="monospace"
                android:textColor="@color/slate_900"
                android:textSize="24sp"
                android:textStyle="bold" />

            <View
                android:layout_width="match_parent"
                android:layout_height="1dp"
                android:layout_marginTop="16dp"
                android:layout_marginBottom="16dp"
                android:background="@color/slate_200" />

            <TextView
                android:id="@+id/textDetails"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:textColor="@color/slate_600"
                android:textSize="14sp"
                android:lineSpacingExtra="6dp" />
        </LinearLayout>
    </com.google.android.material.card.MaterialCardView>

    <!-- Inline error banner -->
    <LinearLayout
        android:id="@+id/errorBanner"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:background="@drawable/bg_pill_rose"
        android:padding="12dp"
        android:layout_marginTop="14dp"
        android:gravity="center_vertical"
        android:visibility="gone">

    <ImageView

```

```

        android:layout_width="18dp"
        android:layout_height="18dp"
        android:layout_marginEnd="10dp"
        android:src="@drawable/ic_alert"
        app:tint="@color/rose_600"
        android:contentDescription="@null" />

        <TextView
            android:id="@+id/textError"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:textColor="@color/rose_700"
            android:textSize="13sp" />
    </LinearLayout>

    <com.google.android.material.button.MaterialButton
        android:id="@+id/buttonComplete"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="20dp"
        android:paddingTop="18dp"
        android:paddingBottom="18dp"
        android:text="Complete energy transfer"
        app:icon="@drawable/ic_bolt"
        app:iconGravity="textStart"
        app:iconTint="@color/white" />
</LinearLayout>
</LinearLayout>
</ScrollView>
</FrameLayout>

```

mobile/app/src/main/res/layout/item_reservation.xml

```

<?xml version="1.0" encoding="utf-8"?>
<!-- One reservation row, shared by the prosumer dashboard/history and the
    grid operator's approval queue. -->
<com.google.android.material.card.MaterialCardView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginBottom="10dp"
    app:cardCornerRadius="18dp"
    app:cardElevation="0dp"
    app:cardBackgroundColor="@color/white"
    app:strokeColor="@color/slate_200"
    app:strokeWidth="1dp"
    android:clickable="true"
    android:focusable="true"
    app:rippleColor="@color/brand_100">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:gravity="center_vertical"
        android:padding="14dp">

        <FrameLayout
            android:layout_width="44dp"
            android:layout_height="44dp"
            android:background="@drawable/bg_tile_brand">

            <ImageView
                android:layout_width="20dp"
                android:layout_height="20dp"
                android:layout_gravity="center"
                android:src="@drawable/ic_bolt"
                app:tint="@color/brand_700"
                android:contentDescription="@null" />
        </FrameLayout>

        <LinearLayout
            android:layout_width="0dp"

```

```

        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:layout_marginStart="14dp"
        android:orientation="vertical">

        <TextView
            android:id="@+id/textCode"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:maxLines="1"
            android:ellipsize="end"
            android:fontFamily="monospace"
            android:textColor="@color/slate_900"
            android:textSize="14sp"
            android:textStyle="bold" />

        <TextView
            android:id="@+id/textSchedule"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="3dp"
            android:maxLines="1"
            android:ellipsize="end"
            android:textColor="@color/slate_500"
            android:textSize="12sp" />

        <TextView
            android:id="@+id/textEnergy"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginTop="2dp"
            android:textColor="@color/slate_700"
            android:textSize="12sp"
            android:textStyle="bold" />
    </LinearLayout>

    <TextView
        android:id="@+id/textStatus"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginStart="10dp"
        android:background="@drawable/bg_pill_emerald"
        android:paddingStart="10dp"
        android:paddingEnd="10dp"
        android:paddingTop="5dp"
        android:paddingBottom="5dp"
        android:textColor="@color/brand_700"
        android:textSize="11sp"
        android:textStyle="bold" />
</LinearLayout>
</com.google.android.material.card.MaterialCardView>

```